

Master's Programme in Computer, Communication and Information Sciences

Identifying User Experience Patterns for the Design System

Aada Köpsi

© 2026

This work is licensed under a [Creative Commons](#)
“Attribution-NonCommercial-ShareAlike 4.0 International” license.



Author Aada Köpsi

Title Identifying User Experience Patterns for the Design System

Degree programme Computer, Communication and Information Sciences

Major Human-Computer Interaction

Supervisor Prof. Marko Nieminen

Advisor Mr Sauli Laitinen (MPsy)

Collaborative partner Vaisala

Date 27 April 2026 **Number of pages** 99+4 **Language** English

Abstract

A Design System (DS) provides reusable elements, standards and visual guidelines, serving as a single source of truth for cohesive and scalable product design. This study focuses on identifying and documenting User Experience (UX) patterns to improve system usability and maintain design integrity.

Conducted with a sensing and measurement solutions company, the work addresses the organization's goal of improving consistency across its digital products by integrating reusable UX patterns into the existing DS. UX patterns are first examined through a review of assumptions within the design team. A top-down analysis of selected software products is then used to extract shared patterns, while micro-level examination of user interactions enables the identification of related sub-patterns. These findings inform the development of a workflow for UX pattern identification.

A documentation structure for the patterns is proposed through benchmarking existing DSs to ensure coverage of functional, visual, and behavioral characteristics. This structure is evaluated through interviews with nine designers and developers, and the insights are analyzed using thematic analysis.

The main outcome of this study is the formal documentation of reusable UX patterns for the DS. The proposed documentation model includes visual overviews, usage examples, UX flows, and guidance for applying and configuring pattern elements, all linked to existing DS assets. There is strong emphasis in this thesis on developing a preliminary framework for extracting UX patterns from the artifacts of established analytical methods.

By formalizing UX patterns within the DS, this work enhances design coherence, reduces redundancy, and strengthens cross-functional collaboration. The proposed framework provides a replicable model for organizations seeking to advance their DSs through pattern-driven practices.

Keywords UX patterns, design patterns, design system, design guidelines, UX design, user experience, interaction design, pattern language

Tekijä Aada Köpsi

Työn nimi Käyttäjäkokeusmallien löytäminen ja sisällyttäminen osaksi suunnittelujärjestelmää

Koulutusohjelma Tietokone-, viestintä- ja informaatiotieteiden koulutusohjelma

Pääaine Ihmisen ja tietokoneen välinen vuorovaikutus

Työn valvoja Prof. Marko Nieminen

Työn ohjaaja PsM Sauli Laitinen

Yhteistyötaho Vaisala

Päivämäärä 27.4.2026

Sivumäärä 99+4

Kieli englanti

Tiivistelmä

Suunnittelujärjestelmät rakentuvat uudelleenkäytettävistä käyttöliittymäelementeistä ja ohjeistuksista, ja ne toimivat yksittäisenä totuuden lähteenä suunnittelijoille ja kehittäjille. Tämä tutkimus keskittyy tunnistamaan käyttäjäkokeusmalleja, joiden avulla ratkaistaan toistuvia suunnitteluongelmia ja yhdenmukaistetaan käyttäjäkokeusta eri palveluissa ja käyttöalustoilla.

Työ toteutettiin yhteistyössä anturi- ja mittausratkaisuihin erikoistuneen yrityksen kanssa, ja se vastaa organisaation tavoitteeseen parantaa digitaalisten tuotteidensa yhdenmukaisuutta integroimalla uudelleenkäytettäviä käyttäjäkokeusmalleja suunnittelujärjestelmän osaksi. Työ aloitetaan tarkastelemalla suunnittelutiimin oletettavia käyttäjäkokeusmalleja. Näitä malleja pyritään tunnistamaan valituille ohjelmitoille tehdyistä ylhäältä alas etenevistä suunnitteluanalyysistä. Tietoa täydennetään hierarkkisella tehtäväanalyysillä ja tehtävämaailman ontologialla, joiden avulla malleihin liittyvät alimallit voidaan määrittää käyttäjien mikrotason toiminnasta. Näiden havaintojen perusteella rakennetaan käyttäjäkokeusmallien tunnistamiseen sopiva työnkulku.

Mallien dokumentaation rakenteelle tehdään ehdotus vertailemalla olemassa olevia suunnittelujärjestelmiä. Näin voidaan varmistaa, että dokumentaatio kuvaa mallien toiminnalliset ja visuaaliset ominaisuudet sekä niiden ilmenemisen eri käyttöympäristöissä. Dokumentaation rakennetta arvioidaan haastattelemalla yhdeksää suunnittelijaa ja kehittäjää, ja havainnot analysoidaan temaattisesti.

Tutkimuksen tärkein tulos on uudelleenkäytettävien käyttäjäkokeusmallien dokumentointi osaksi suunnittelujärjestelmää. Ehdotettu dokumentaatio sisältää visuaalisia yleiskuvauksia, käyttöesimerkkejä, variaatioita, käyttäjän kosketuspisteiden kuvauksia sekä ohjeita mallien soveltamiseen. Työssä korostuu myös menetelmän kehittäminen käyttäjäkokeusmallien tunnistamiseen vakiintuneiden analyysimenetelmien tuottaman aineiston pohjalta.

Avainsanat Suunnittelumallit, käyttäjäkokeusmallit, design-systeemi, suunnittelujärjestelmä, käyttäjäkokeus, käyttöliittymäsuunnittelu

Preface

As I write this final page of my academic journey, I find myself reflecting not just on the thesis, but on the path that led me here. This work, centered on identifying UX design patterns in interactive software, has given rise to academic curiosity, collaboration, and personal growth.

The idea for this thesis has been quietly present in the background of many discussions in the design field. UX design patterns have long garnered interest in the field, but have rarely been explored in depth. I feel honored to have had the opportunity to study a topic that has long deserved more clarity and attention. This work is my small contribution to a conversation that is still unfolding.

I owe my deepest thanks to everyone who supported me throughout this process, especially Professor Marko Nieminen, Instructor Mr Sauli Laitinen, and the design team colleagues I had the privilege of working closely with. I am sincerely grateful to Vaisala for providing me with this opportunity.

While this thesis marks the end of my formal studies, it also opens the door to a new chapter, one filled with possibility. Whether I continue in UI/UX design or venture into a field yet unknown, I carry forward the mindset this work has instilled: to observe with curiosity, to design with care, and stay creative in whatever lies ahead.

Otaniemi, 27 April 2026

Aada Köpsi

Contents

Abstract	3
Abstract (in Finnish)	4
Preface	5
Contents	6
1 Introduction	9
1.1 Motivation	10
1.1.1 Benefits of Unified Design for User Experience	10
1.1.2 Benefits of Unified Documentation for Design	10
1.2 Problem Statement	11
1.2.1 Research Objectives	11
1.2.2 Research Questions	12
2 Conceptual Background	13
2.1 Human-Centered Design	13
2.2 Human Factors Engineering	14
2.3 Design for Experience	15
2.3.1 Definition of User Experience (UX)	15
2.3.2 UX Design	16
2.3.3 UX Goals	17
2.4 Customer Journey Mapping	17
2.4.1 UX Consistency Across Touchpoints	18
2.4.2 Integration Across Digital Environments	19
2.5 Design Pattern Language	20
2.5.1 Interplay of Design Patterns – Pattern Library	21
2.5.2 Hierarchical Structuring of Design Patterns	21
2.5.3 UX Patterns	24
2.5.4 User Interface Patterns	25
2.6 Design Systems in HCI	26
2.6.1 Atomic Design Framework	27
2.6.2 Benefits of Design Systems	30
3 Related Work on Design Systems and UX Patterns	32
3.1 Current State of Design Systems	32
3.1.1 Overview of Current Design Systems	32
3.1.2 Definitions of Design System	33
3.2 Task-Based Design	34
3.2.1 Hierarchical Task Analysis	35
3.2.2 Task World Ontology	38
3.3 UX Patterns in Practice	38
3.3.1 Documentation of UX Patterns	38

3.3.2	Practical Examples of Existing UX Patterns	40
3.4	Ownership and Governance of Design Patterns	41
3.5	Architecture and Modularity	43
4	Methods and Data	44
4.1	Research Approach	44
4.1.1	Proposed Definition of Design System in Vaisala	44
4.1.2	Proposed Definition of UX Patterns	45
4.1.3	Method for Creating UX Pattern Identification Process	46
4.1.4	Method for Designing Documentation Framework	46
4.2	Vaisala Research Context	48
4.2.1	Vaisala Digital Products	48
4.2.2	Selecting Assumed UX Patterns	49
4.3	Proposed UX Pattern Documentation Framework	50
4.4	Empirical Study for Application and Evaluation	54
4.4.1	Participants	55
4.4.2	Interview Procedure	57
4.5	Thematic Analysis	58
4.5.1	Coding Focus and Analytical Decisions	58
4.5.2	Phase 1: Familiarization With the Data	59
4.5.3	Phase 2: Generating Initial Codes	59
4.5.4	Phases 3–5: Developing, Reviewing, and Defining Themes	59
5	Results	61
5.1	Characteristics of a UX pattern	61
5.2	UX Pattern Identification Framework	62
5.2.1	Phase 1: Pattern Identification Across Systems	62
5.2.2	Phase 2: Pattern Abstraction and Formalization	67
5.3	Validating Initial Documentation Framework Proposal	68
5.3.1	Familiarity with Design Pattern Terminology	68
5.3.2	Thoughts on the Main User Group	69
5.3.3	General Feedback on the Documentation Proposal	70
5.3.4	Future Prospects and Improvements	74
5.3.5	Maintenance and Sustainability of the Documentation	75
6	Discussion	77
6.1	Revisiting the Research Questions	77
6.1.1	Research Question 1	77
6.1.2	Research Question 2	77
6.1.3	Research Question 3	78
6.1.4	Research Question 4	79
6.2	UX Pattern Documentation: Consistency, Reuse, Governance	79
6.2.1	Consistency	80
6.2.2	Reuse	81
6.2.3	Governance	82

6.3	Refined Proposal for the Documentation Framework	83
6.4	Challenges and Limitations	84
6.4.1	Lacking and Narrow Data	84
6.4.2	Analysis Methods	84
6.4.3	Empirical Evaluation	85
7	Conclusion	87
	References	89
A	Semi-Structured Interview Guide	100

Abbreviations

AI	Artificial Intelligence
B2B	Business-to-business
CJM	Customer Journey Map
DS	Design System
DSR	Design Science Research
HCD	Human-Centered Design
HFE	Human Factors Engineering
HTA	Hierarchical Task Analysis
ID	Interaction Design
UI	User Interface
UID	User Interface Design
UX	User Experience
PO	Product Owner
RQ	Research Question
SD	Service Design
TA	Thematic Analysis
TWO	Task World Ontology

1 Introduction

Ensuring a shared visual language and consistent interaction patterns across all interfaces is indispensable for producing User Experiences (UXs) that feel unified and are easy to anticipate (Ozok & Salvendy 2000). According to van Welie et al. (2001), consistency reduces cognitive load, fosters trust, and improves efficiency by ensuring that interactions happen as expected. However, as organizations scale their ecosystems across multiple platforms and teams, achieving this uniformity becomes increasingly complex (Werle 2021).

According to Yew et al. (2020), Kholmatova (2017), Design Systems (DSs) provide an organized framework that promotes interface consistency by standardizing reusable components, prescribing design guidelines, and supplying supporting documentation. While DSs accelerate development and unify design language, they are not exhaustive. Gaps often occur in undocumented scenarios, leaving designers to rely on personal judgment and make decisions about the ideal ways to combine design elements for the best UX for the particular issues (Werle 2021), introducing variability and inefficiencies. To address these challenges, Werle (2021) notes that design patterns have gained attention as recurring solutions to common design problems. They complement DSs by reducing duplicate efforts and improving workflows (Lamine & Cheng 2022). Despite their potential, existing research on discovering and documenting design patterns remains fragmented and theoretical. While existing research explores the theoretical qualities and hierarchies of patterns (van Welie & van der Veer 2003), a concrete connection to their practical application for achieving both visual and functional consistency is lacking.

Although DSs and design patterns have gained attention, their integration and systematic documentation lack clarity. Current practices differ significantly across organizations, and literature offers limited guidance on how patterns emerge or how they should be documented. This thesis addresses this gap by proposing a structured framework for identifying and documenting UX patterns, bridging theory and practice.

The study focuses on UX patterns within digital services, emphasizing their manifestation and role in enhancing consistency across platforms. It excludes unrelated design methodologies and does not aim to create a complete taxonomy of patterns. Instead, the perspective is practical: supporting designers and developers through a scalable identification and documentation approach.

The primary aim is to advance the understanding of UX pattern emergence and provide a framework for their systematic identification and documentation. By combining insights from academic literature and empirical data, this research seeks to unify terminology, improve design workflows, and strengthen the integration of UX patterns into DSs. Ultimately, the results aim to enhance UX quality while aligning with organizational requirements and brand philosophy.

In this thesis, Section 2 provides a conceptual background of the fields of Human-Centered Design (HCD) and design patterns. The current state-of-the-art in DSs and UX patterns is summarized in Section 3 through observations of the documentation provided by various technology companies. Section 4 discusses the methodology used to devise the proposed processes for identifying and documenting UX patterns, as

well as validate the documentation proposal through an empirical study. The results of this thesis are outlined in Section 5, including the proposed processes and the results of the empirical study. Section 6 discusses the results and considers related future prospects and challenges. Finally, the study is concluded in section 7.

1.1 Motivation

1.1.1 Benefits of Unified Design for User Experience

Visual and functional consistency is a cornerstone of high-quality UX. It ensures that interactions feel predictable and coherent, whether within a single service, across multiple platforms, or throughout an entire ecosystem of digital products. Consistency is formed through harmonized digital touchpoints, layouts, navigation flows, and interaction patterns, that allow users to move seamlessly between applications without relearning behaviors or adapting to new conventions (Werle 2021).

As noted by Kholmatova (2017), when consistency breaks down, users face increased cognitive load as they interpret varying design patterns and behaviors. This slows task completion, reduces efficiency, and undermines trust. Weidig et al. (2024) conclude that fragmented experiences across services can make an ecosystem feel disjointed, ultimately diminishing brand loyalty and perceived product quality.

Achieving a high level of uniformity becomes increasingly challenging as organizations scale. A growing number of designers, diverse teams, and expanding service portfolios amplify the risk of variability (Werle 2021). While DSs provide a strong foundation for visual consistency through reusable components and guidelines, according to Lamine & Cheng (2022), they primarily address User Interface (UI) styling and atomic elements. Therefore, they often fall short in governing interaction logic and holistic user flows (Lamine & Cheng 2022).

To bridge these gaps, UX patterns, recurring solutions to common design challenges, complement DSs by addressing behavioral and structural aspects of UX (Blackwell & Fincher 2010). Stoeva (2025) declares that patterns reduce duplicate efforts, accelerate development, and improve workflows. However, there is no standardized process for discovering and documenting these patterns. Werle (2021) outlines that current practices vary widely, leaving designers to rely on personal judgment, which can lead to inconsistencies. The ambiguity in current practice highlights the importance of establishing a systematic framework capable of embedding UX patterns within design processes, thereby supporting consistency and scalability across digital products

1.1.2 Benefits of Unified Documentation for Design

Unified documentation is essential for achieving consistency and efficiency in digital product ecosystems. When design decisions, reusable components, and interaction patterns are documented in a structured and accessible manner, teams can align on a shared understanding of best practices (Suarez et al. 2019). This alignment

reduces ambiguity, accelerates development, and minimizes the risk of fragmented user experiences (Stoeva 2025).

A centralized documentation framework ensures that both designers and developers operate from a single source of truth (Kholmatova 2017). For designers, it provides clarity on visual and interaction principles; for developers, it translates these principles into actionable guidelines for implementing UI as code (Lamine & Cheng 2022). Well-documented UX patterns help developers understand not only what to build but also why certain design decisions exist, reducing misinterpretations and improving implementation quality. According to Churchill (2019), this shared reference fosters collaboration across multidisciplinary teams, enabling smoother handoffs and reducing reliance on individual judgment during the concept creation and prototyping phases.

Without unified documentation, inconsistencies often arise from subjective interpretations of design principles. Designers may approach similar problems differently (Zhang & Wakkary 2014), and developers may implement components in ways that diverge from the intended behaviors by the designers (Lamine & Cheng 2022). Kholmatova (2017) highlights that these inconsistencies increase the cognitive load for users and diminish brand coherence across platforms. In contrast, a well-maintained documentation system mitigates these risks by offering standardized solutions for recurring design challenges (Lamine & Cheng 2022).

Finally, unified documentation is not limited to visual elements. Lamine & Cheng (2022) conclude that the documentation of a DS often extends to design patterns which complement DSs by addressing behavioral aspects of the UX. Incorporating UX patterns into the documentation ensures that both the aesthetic and functional dimensions of the design are captured. This integration strengthens the meaning and impact of DSs, transforming them from static component libraries into dynamic frameworks that guide holistic, user-centered design and development practices (Werle 2021). Together, DSs and design patterns, such as UX patterns, create a comprehensive foundation for predictable, high-quality (UXs) across diverse digital touchpoints, as found by Muthaffar et al. (2024) in the domain of online shopping.

1.2 Problem Statement

1.2.1 Research Objectives

This study builds upon conceptual UX patterns proposed by Vaisala’s design team, which serve as foundational references for further exploration. These patterns, hypothesized based on a brainstorming session by the design team, provide a starting point for developing a method for identifying UX patterns and understanding their impact on consistency and clarity across digital services. They also offer insights into the practicality of documentation in the daily work of designers and developers.

The primary objective of this research is to create a structured approach for discovering and documenting UX patterns that can be integrated into existing DSs. To achieve this, the study first aims to develop a method for discovering UX patterns from a collection of designs based on a top-down analysis of user interactions and supported actions. This approach relies on conceptual diagrams and the methodology of pattern

language to formalize findings in a way that supports both theoretical understanding and practical application.

The next objective is to establish a documentation framework that captures both the functional and visual aspects of UX patterns. This framework is intended to provide clarity for designers and developers, ensuring that patterns are not only well-defined but also actionable in real-world workflows. The proposed methods are then applied to Vaisala's digital services, resulting in a documented library of UX patterns that includes examples of their usage within actual service contexts. This practical application demonstrates the feasibility and relevance of the framework.

Finally, the study seeks to validate the effectiveness of the documentation through an empirical evaluation of designers', developers and DS developers satisfaction. This validation ensures that the documented patterns meet practical needs, improve collaboration, and enhance design consistency. By addressing these objectives, the research aims to strengthen the integration of UX patterns into DSs, thereby promoting consistency, improving workflows, and elevating the overall quality of digital products.

1.2.2 Research Questions

This study aims to answer the following Research Questions (RQs):

- **RQ 1:** What are the qualities or specific characteristics required for a UX pattern to exist?
- **RQ 2:** How can UX patterns be identified and abstracted from diverse software contexts?
- **RQ 3:** How can UX patterns be documented to support reuse within Vaisala's DS?
- **RQ 4:** What structure and elements of UX pattern documentation are perceived as most useful?
- **RQ 5:** In what ways do participants perceive UX pattern documentation as supporting the consistency, reuse, and governance of the DS?

The thesis addresses each of the RQs at different stages of the work. RQ 1 is addressed in Section 5.1. Section 5.2 presents the UX pattern identification framework, thereby answering RQ 2. Section 4.3 addresses RQ 3 by outlining the proposed documentation structure for UX patterns, which is evaluated through the empirical study. The refined structure for documentation is proposed in Section 6.3. The answer to RQ 4 is reported through the results of the empirical study in Section 5.3. RQ 5 is considered in Section 6.2.

2 Conceptual Background

2.1 Human-Centered Design

Don Norman is seen as the "Father of User Experience" ([IxDF - Interaction Design Foundation 2019](#)) and a pioneer of HCD as he was one of the first to emphasize the importance of solving the right problem in design. [Norman \(2013\)](#) extensively applied this ideology in his framework that comprises two primary phases: discovering the issue and resolving it in a way that fulfills the user needs. The strength of this HCD framework lies in its iterative nature where the design is refined continuously until a desired solution is achieved. Norman outlines four core activities in this framework: observation, idea generation, prototyping, and testing ([Norman 2013](#)).

These four core activities were a great influence and formed the backbone of design thinking practices. In the early 1990s, the design thinking framework was popularized by IDEO ([Brown 2008](#)). [Bender-Salazar \(2023\)](#) concludes that IDEO's work was greatly influenced by the pioneers in the field [Cross et al. \(1992\)](#), [Lawson \(2005\)](#), [Rowe \(1987\)](#) and [Archer \(1979\)](#). The framework has later been revised and supplemented by [d.school \(2010\)](#) to form a 5-phased framework of empathize, define, ideate, prototype and test.

[Norman \(2013\)](#) demonstrates that the observation phase involves understanding the core issue by emphasizing users' perspectives, contexts, and behaviors. [Hornbæk et al. \(2025\)](#) advice that at this point it is more important to understand users and other associated stakeholders than to immediately seek solutions. Furthermore, [Hornbæk et al. \(2025\)](#) suggest it should be prioritized to uncover latent needs, interests, and challenges of the users that form the foundation of design requirements. Triangulation, the combining of multiple research methods, such as observations, interviews and usability studies, are often applied to augment the data, validate and increase the reliability of findings.

During the idea generation phase, personas, use-case scenarios and key design requirements are often developed based on the gathered data ([Cooper et al. 2014](#)). In the early stages of the HCD process, it is essential to generate a wide range of ideas without prior judgment or fixation on a limited set ([Hornbæk et al. 2025](#), [Osborn 1953](#)). Otherwise, the design might attempt to solve the wrong problem ([Norman 2013](#)). Therefore, [Osborn \(1953\)](#) highlights that the primary objective of brainstorming is to remain open-minded and prioritize quantity over quality in idea generation. Once all ideas have been discovered, the optimal design can be selected.

The prototyping phase involves creating mockups that vary in fidelity ([Norman 2013](#)). Low-fidelity prototypes, for example sketches or paper models, are quick and inexpensive. On the contrary, high-fidelity prototypes, such as digital mockups created in tools like Figma or Adobe XD, or Wizard-of-Oz simulations, are more detailed and expensive. According to [Norman \(2013\)](#), the primary purpose of prototyping is to validate that the design addresses the correct problem. [Snyder \(2003\)](#) observes prototypes enable collecting early feedback from potential stakeholders, facilitating iterative refinement to better align with user expectations and needs. At this stage, it is relatively effortless to apply new changes and refine the design compared to later

stages (Norman 2013).

In testing, a group of the targeted end users are selected based on the developed personas (Cooper et al. 2014). Participants are asked to interact with the prototype, often performing specific tasks while thinking aloud their observations and feelings. Often these responses, such as reactions, thoughts, and suggestions, are recorded through audio, video, or written notes (Norman 2013). The main goal of testing is to confirm that the design effectively addresses the identified problem. If issues are discovered, the insights gained are used to refine the prototype, which is then re-tested iteratively until the design meets the established requirements (Norman 2013).

Norman & Draper (1986), Wynn & Eckert (2017) introduce the iterative development process is a key to success in HCD, as it is often difficult to arrive at a perfect solution on the first attempt. Moreover, throughout these four stages, misunderstandings or new issues may emerge which highlights the importance of continuous evaluation and adaptation within the design process (Wynn & Eckert 2017).

2.2 Human Factors Engineering

Human Factors Engineering (HFE) provides a foundational framework for understanding how people perceive, process, and interact with digital systems. Grounded in psychology, ergonomics, and cognitive science, HFE examines the capabilities, limitations, and behavioral tendencies that shape effective interaction design (Hornbæk et al. 2025, Deuschel 2018). By clarifying these human-centered determinants, HFE offers essential principles that guide the creation of interfaces aligned with real user needs and constraints (Deuschel 2018).

Within this human-centered foundation, design patterns become a practical means of operationalizing HFE insights. Emerging from recurrent usability and interaction challenges observed in real-world contexts, design patterns capture and formalize proven solutions. In this sense, they can be understood as systematic artifacts of HCD, translating underlying HFE principles into repeatable, context-sensitive guidelines for interface development (van Welie et al. 2001).

When designing interactive systems, Deuschel (2018) advice it is beneficial to consider central human-factors, such as human capabilities and limitations, particularly the way the human visual and cognitive systems function. Perception is one of the most vital human senses, enabling individuals to acquire and organize information from their surroundings (Hornbæk et al. 2025). By examining human perceptual processes, designers can better understand how users allocate attention across interface elements (Deuschel 2018). Although Fitts' Law primarily models motor performance (Hornbæk et al. 2025), it also supports perceptually informed design by encouraging the placement and sizing of interactive targets in ways that make actions visually clear and easy to execute (Bertelsen 1999).

Cognition is one of the core focus of HFE, especially important in UI design, because it encompasses the mental processes, such as thinking, understanding, memory, and reasoning, through which systems are interpreted and engaged with (Hornbæk et al. 2025). Deuschel (2018) articulates that limited cognitive resources are central to this allocation, as they influences how effectively users can process information,

navigate and perform tasks within an interface. [Hornbæk et al. \(2025\)](#) emphasize the severe limitations of human cognition, noting that working memory generally holds only 2–4 items at a time. According to [Engle \(2002\)](#), working-memory capacity does not primarily reflect the amount of information that can be stored; rather, it reflects the effectiveness of attentional mechanisms in maintaining relevant information and suppressing irrelevant information. Therefore, individuals often supplement their mental capacity with external supports such as calendars, reminders, or notes ([Gruszka & Nęcka 2017](#), [Hornbæk et al. 2025](#)). Within digital environments, cognitive processes are vital for navigating interfaces and locating elements on a display, enabling users' to execute tasks that takes them to their desired goal ([Ledneva & Kovalev 2021](#), [Deuschel 2018](#)).

In conclusion, understanding the ways in which perception and cognition are operated upon enables designers to reduce unnecessary complexity and unfamiliarity in interactive systems, thereby smoothing the learning curve. These insights also support the study of context of use, particularly in identifying the specific activities or tasks performed by users during system interaction. This, in turn, supports the design of UXs through touchpoint analysis ([Touloum et al. 2018](#)) and task analysis ([IxDF - Interaction Design Foundation 2023](#), [van Welie 2001](#)), both of which are essential for developing effective and user-centered design patterns. [Touloum et al. \(2018\)](#) advise that examining touchpoints ensures that relevant ones are not overlooked and that they remain aligned with an understanding of user behavior to facilitate the desired UX. Task analysis further deepens this understanding by clarifying users' goals and the sequential steps they execute according to a specific plan of action ([van Welie 2001](#), [IxDF - Interaction Design Foundation 2023](#)).

2.3 Design for Experience

2.3.1 Definition of User Experience (UX)

[ISO \(2019\)](#) defines UX as the "user's perceptions and responses that result from the use and/or anticipated use of a system, product or service". The perception captures the manner in which the user's interactions occur and usability and usefulness are perceived ([Hassenzahl 2008](#), [Touloum et al. 2018](#)). Furthermore, the responses are the combination of the emotions experienced and the behavioral and social responses ([ISO 2019](#), [Touloum et al. 2018](#)).

As experience is shaped through subjective and individually perceived engagement with a service ([Hassenzahl 2008](#), [Stein & Ramaseshan 2016](#)), it is rather difficult to design a precise and uniform experience for all users. As [Stein & Ramaseshan \(2016\)](#) articulate, experience can be moderately influenced through designed UIs and by crafting meaningful purposes. For this reason, emphasis should shift from attempting to create a fixed experience to shaping conditions and stimuli that enable users to form their own experiences ([Stein & Ramaseshan 2016](#)). By controlling these underlying factors, designers might positively impact customer satisfaction, engagement, and brand loyalty more effortlessly. [Kaasinen et al. \(2015\)](#) suggest designers should aim to "design for an experience rather than design an experience", allowing the experience

to emerge naturally through thoughtful interaction design.

2.3.2 UX Design

Many companies producing digital services, especially those in the Business-to-Business (B2B) domain, aim to offer a cohesive service experience composed of multiple individual systems. While UX design typically focuses on optimizing a single system or service component (Roto et al. 2016), Service Design (SD) takes a broader perspective, encompassing the entire customer journey across these interconnected systems (Roto et al. 2016, Touloum et al. 2018). Roto et al. (2016) outlines SD as follows: "For example, when you are ill and need a prescription drug, you book an appointment from a doctor, see the doctor, get a prescription, visit a pharmacy to buy the drug, and use the drug.". Therefore, SD introduces "a holistic, co-creative and user-centered approach" to understand the user needs and behavior for greater UX (Touloum et al. 2018).

Building on this example, services are experienced through a series of touchpoints, each representing a specific moment in which users interact with the service (Touloum et al. 2018), furthermore the way they are supported by the design (Weidig et al. 2024). These can include actions such discovering the products, browsing through the store, comparing the products, examining the product more detailed, adding to shopping cart or a wishlist (van Welie & van der Veer 2003). Together, these touchpoints form an interconnected whole that shapes the quality and coherence of the overall UX (Touloum et al. 2018), further reaching beyond the sum of its components (Alvarez et al. 2020).

Designing such coherent experiences requires the complementary perspectives of UX design and SD. While UX design focuses on the usability and interaction quality of individual touchpoints, SD considers their orchestration across the entire service journey (Touloum et al. 2018). When combined, these perspectives enable more consistent, user-centered, and meaningful experiences, which is central to designing for experience (Touloum et al. 2018, Roto et al. 2016).

The central idea of UX design is the overall experience that consists of insights into interpretation, response and engagement with a product or system (ISO 2019, Hassenzahl 2008). This experience is not static; as Roto et al. (2016) summarize, it evolves across different contexts, devices, and moments of interaction. Understanding this dynamic nature is essential for defining UX and setting meaningful UX goals that align with user needs and expectations.

To ensure a seamless and consistent experience across various digital services, it is essential to study these evolving customer journeys and design for multi-touchpoint experiences (Roto et al. 2016). This approach helps create a harmonious flow through different activity points and interfaces. According to Roto et al. (2016), multi-touchpoint experience design contains elements of UX, SD, and Omnichannel design. Verhoef et al. (2015) explain omnichannel encompass broader set of channels (stores, social media, online websites and mobile channels) compared to multi-channel. Furthermore, the concrete and strict boundaries between channels become to converge, thus, omnichannel management aims to manage the various touchpoints

while optimizing the seamless performance and UX through various channels ([Verhoef et al. 2015](#)).

Omnichannel design refers to the integrated orchestration of channels and touchpoints so that users experience a seamless, continuous, and consistent journey, while organizations align content, interactions, data, and operations across the customer life-cycle ([Verhoef et al. 2015](#), [Gerea et al. 2021](#), [Gahler et al. 2022](#)). Unlike multichannel approaches, omnichannel design optimizes both the experience across channels and channel performance through cross-channel integration ([Verhoef et al. 2015](#), [Gerea et al. 2021](#), [Gahler et al. 2022](#)). Building on this foundation, brand identity provides the unifying cues and experience goals that ensure consistency, authenticity, and meaning across touchpoints, which strengthens users' perceptions and behaviors in omnichannel contexts ([Massi et al. 2023](#), [Frasquet-Deltoro et al. 2021](#), [Jin et al. 2025](#)).

2.3.3 UX Goals

UX goals articulate the intended experiential qualities that a system aims to provide ([Väättäjä et al. 2015](#)), thereby guiding design decisions toward meaningful interactions within a given context ([Kaasinen et al. 2015](#), [Väättäjä et al. 2015](#)). Rather than focusing solely on task completion or interface efficiency, UX goals emphasize how users perceive, interpret, and engage with a system throughout their interaction ([Kaasinen et al. 2015](#)).

In UX design, these goals are shaped by three interconnected considerations: alignment with brand identity and long-term vision, sensitivity to users' emotional and situational needs, and grounding in established knowledge of human behavior and interaction ([Kaasinen et al. 2015](#), [Väättäjä et al. 2015](#), [Beauregard & Corriveau 2007](#)). [Beauregard & Corriveau \(2007\)](#) stress that UX goals can be used as an effective means of communication between different stakeholders. By articulating shared experiential intentions, UX goals support the creation of coherent and user-centered experiences without prescribing specific solutions, instead guiding development in an informed and consistent direction ([Beauregard & Corriveau 2007](#), [Väättäjä et al. 2015](#)). As such, UX goals function as a high-level framing device rather than a fixed set of design rules, informing design while allowing flexibility in implementation ([Kaasinen et al. 2015](#), [Beauregard & Corriveau 2007](#)).

2.4 Customer Journey Mapping

Some academic studies, such as [Heuchert \(2019\)](#), [Alvarez et al. \(2020\)](#), [Weidig et al. \(2024\)](#), [Stein & Ramaseshan \(2016\)](#) approach the identification of interaction and experience touchpoints through creating Customer Journey Maps (CJMs). [Weidig et al. \(2024\)](#), [Alvarez et al. \(2020\)](#) note that these CJMs, often referred to as "emotion timelines", present a foundational and comprehensive view of UX, focusing on macro-level interactions across different channels. CJMs serve as a bridge between UX goals and interactions that occur throughout a service ([Alvarez et al. 2020](#)). Fundamentally, they represent the sequence of activities or interactions users perform while engaging with a service, organized in chronological order ([Alvarez et al. 2020](#)).

From CJMs, touchpoints emerge as the main activity points where the user interacts with the system (Touloum et al. 2018, Kalbach 2016, Heuchert 2019). Stein & Ramaseshan (2016) define a touchpoint as the point of contact where UX emerges, emphasizing their role in shaping UX. A series of touchpoints forms a particular customer journey, and as Lundin & Kindström (2024) support, they are inevitably embedded within that journey. This interconnection highlights the importance of studying and managing touchpoints carefully to ensure seamless UX. This becomes particularly important when the service comprises multiple individual software systems.

B2B customer journeys tend to have more paths due to diverse stakeholders, a high number of touchpoints, and extended engagement timelines (Lundin & Kindström 2024). In digital platforms, these touchpoints may include purchasing a product or service, searching for information, or using tools to monitor product performance, such as the state and data of sensors. Furthermore, users may engage in multiple macro or micro-interactions within the system, which calls for a multi-touchpoint experience design approach. These interactions may also span across different systems that collectively form the service, highlighting the need for a unified design strategy.

One approach to identify touchpoints within CJMs is to analyze the various tasks users perform within the provided digital platform (Touloum et al. 2018). As emphasized by van Welie (2001), a task-based approach might improve the design process by reducing reliance on trial-and-error techniques. This analytical approach closely aligns with touchpoint identification, as it provides actionable insights into user interactions and informs more targeted design decisions. van Welie (2001) describes task analysis as the systematic breakdown of user-performed tasks within an interactive system into discrete steps. By deeply understanding how users carry out these tasks, designers can create systems that are not only intuitive and usable but also effectively support user goals and workflows.

2.4.1 UX Consistency Across Touchpoints

In modern digital service ecosystems, companies frequently deliver experiences through a constellation of interconnected software systems. These systems may serve distinct functions, such as purchasing, IT support, or product usage remotely, but together they affect the overall customer journey. Lundin & Kindström (2024) explain that the management of customer touchpoints can be approached through three dimensions: managing resources, actions, and environments. This framework enables the management of both individual touchpoints and sequences of interactions once they are identified.

Resource management involves integrating and assessing various resources across different touchpoints (Lundin & Kindström 2024). They observe that interactions occur within specific environments, such as in software platforms, and that designers define and constrain the ways users interact within them. While Nielsen (2024) emphasizes the importance of user freedom in system design, Lundin & Kindström (2024) point out that the increasing number of digital environments introduces both flexibility and complexity in managing touchpoints and crafting cohesive customer

journeys. They recommend examining the interconnections among these environments to deepen the understanding of their influence on UX and design coherence. Empirical findings from [Lundin & Kindström \(2024\)](#) suggest that managing touchpoint resources should support the integration of digital assets so that resources spread throughout the customer journey become unified and coherent. For example, enabling users to monitor and collect data within a single platform could streamline their future service interactions. Providing a single source of truth could reduce data duplication and allow a consistent representation of information. This might be particularly beneficial in applications involving multiple sensor readings, where centralized access enhances situational awareness, especially in complex B2B environments with interrelated factors. Such centralized access also minimizes the need to switch between systems or rely on others for information, thereby promoting user empowerment and autonomy.

In B2B contexts, end users often represent multiple roles, such as buyers, technicians, or managers, operating at different organization levels. The second dimension of touchpoint management focuses on enabling individual actions within clearly defined boundaries ([Lundin & Kindström 2024](#)). Their study underscores the importance of supporting parallel and sequential workflows across roles. For instance, a technician might seamlessly continue a task initiated by a manager, ensuring an uninterrupted workflow. At this stage, identifying the roles involved in the customer journey becomes essential for designing role-specific interactions ([Patrício et al. 2011](#)).

2.4.2 Integration Across Digital Environments

In digitalized service ecosystems, customer actions and content frequently intersect across multiple environments. As [Lundin & Kindström \(2024\)](#) observe, B2B customers often navigate a variety of digital platforms, each containing macro- and micro-level user flows. These individual flows collectively form the complete CJM. The results indicate that digitally connected environments must be aligned in order to facilitate smooth movement across systems and better accommodate users' objectives ([Lundin & Kindström 2024](#)). Such integration creates opportunities to centralize key features ([Lundin & Kindström 2024](#)), generalize design patterns, and connect essential functionalities across platforms ([Gahler et al. 2022](#), [Gerea et al. 2021](#)). Ultimately, [Verhoef et al. \(2015\)](#), [Gerea et al. \(2021\)](#) describe the integration facilitates a unified flow across software systems, resulting in a more coherent and seamless UX. [Bitner et al. \(2008\)](#), [Kalbach \(2016\)](#) advice by mapping the journey at both macro and micro levels within these environments, designers can identify key touchpoints and recurring patterns. These insights inform the creation of design solutions that are both visually and functionally coherent, regardless of the underlying platform ([Gahler et al. 2022](#)). This approach supports the development of a pattern language that bridges individual flows and ensures a seamless, human-centered experience across the entire service ([van Welie 2001](#), [Alexander et al. 1977](#)).

2.5 Design Pattern Language

[Erickson \(2000\)](#) defines design pattern language as shared communication tool that facilitates collaboration among stakeholders with differing roles and expertise. Ultimately, the study of design pattern languages provides a structured means to articulate recurring design problems and solutions, enabling designers to better understand the design space, prioritize relevant patterns, and identify opportunities to reuse proven solutions across similar user flows ([Alexander et al. 1977](#), [van Welie 2001](#), [van Welie & van der Veer 2003](#)).

To summarize, design patterns focus on representing recurring solutions to familiar design challenges while reducing entropy within UIs ([Werle 2021](#), [Alexander et al. 1977](#), [Wu et al. 2016](#)). They are structured from smaller UI components, to form more complex patterns ([Werle 2021](#)). These patterns are often associated with the tasks users perform and they can address principles, such as 10 heuristics by [Nielsen \(2024\)](#) ([van Welie 2001](#)) or ergonomic principles ([Folmer 2015](#)). As they are associated with the executed tasks and the context of use, it can be seen design patterns aim to depict series of events on a timeline.

The development of design patterns was fundamentally influenced by the architectural concepts introduced by Christopher Alexander ([Alexander et al. 1977](#)), which were first applied to urban planning and later adapted to software design ([Gamma et al. 1994](#), [Borchers 2000b](#)). Alexander's definition of a pattern provides the foundation for design patterns: "Each pattern is a three-part rule, which expresses a relation between a certain context, a problem, and a solution" ([Alexander 1979](#)). This methodology was further extended to software development in 1994 through Object-Oriented Software Design [van Welie & van der Veer \(2003\)](#). Currently, the definition of a design pattern is yet to be standardized, and still evolving. A study by [da Rosa & Silveira \(2022\)](#) demonstrate that the term "design pattern" is still used as a synonym for "patterns", and often they are used interchangeably.

The discrepancy between guidelines and patterns can sometimes lead to confusion, particularly when identifying design patterns. [Van Welie et al. \(2001\)](#) explain that guidelines may be perceived as too simplistic, abstract, or vague when applied to a concrete design case, as the underlying problem may remain unresolved even when a guideline is followed. In many instances, guidelines lack explicit contextual grounding, may contradict one another, and provide limited guidance on "when, how, and why a solution should be applied" ([van Welie et al. 2001](#)). As a result, they often fail to clearly articulate both the nature of the design issue and the rationale for resolving it. For example, a guideline such as "Left-align labels in dialog windows" does not specify the underlying usability problem it addresses or explain why this recommendation is appropriate in a given context [van Welie et al. \(2001\)](#).

In contrast, the solution of the proposed design patterns should provide more rationale and be concrete without leaving room for interpretation ([van Welie et al. 2001](#), [Tidwell 2010](#)). They can integrate several guidelines and are intended to benefit both the end users and the designers. According to [Werle \(2021\)](#), design patterns serve as a ground truth that unifies the best practices with principles of an effective design, and most importantly provides a reusable library of solutions that others can easily

apply.

2.5.1 Interplay of Design Patterns – Pattern Library

Design pattern libraries, in contrast, are collections of documented design patterns that provide concrete instances and examples through which this shared language can be expressed and applied (Lamine & Cheng 2022, Werle 2021). Together these individual patterns compose a pattern library, a collection or repository of reusable design solutions as functional components (Lamine & Cheng 2022). According to Werle (2021), pattern libraries render the design language tangible and exposing the system that governs the final interface.

Van Welie & van der Veer (2003) approach pattern language through the idea of User Interaction Design (UID), which focuses on both UI components and high-level UX goals. This top-down approach aims to outline the relationships and connections from business goals all the way to the low-level actions. In addition, it describes the way interactions are established with the knowledge of "wishes, technical requirements and contexts" (van Welie & van der Veer 2003). Borchers (2000b) supports this ideology of design patterns forming a hierarchy of higher- and lower-level patterns.

Van Welie & van der Veer (2003) describe the network of patterns as being structured through relationships of aggregation, specialization or association. These relationships aim to organize UI patterns, such as those of a list builder or wizard, within a broader UX pattern framework to facilitate the intended UX. This highlights the close relation between UI and UX patterns and potential confusion between them; UI patterns enable the implementation mechanisms to support the UX goals.

Van Welie & van der Veer (2003) summarize that patterns can aggregate other patterns through a "has-a" relationship. For instance, a shopping cart pattern may utilize a list builder to display the choice options and a wizard to guide users through the purchasing process. Specialization reflects an "is-a" relationship, where one pattern is a more specific version of another, such as an advanced search pattern that builds upon the logic of simple search (van Welie & van der Veer 2003). Association, on the other hand, includes a "related-to" relationship, for example when a shopping experience allows items to be purchased directly from a product comparison or via a shopping cart pattern (van Welie & van der Veer 2003).

2.5.2 Hierarchical Structuring of Design Patterns

To study design patterns more closely, van Welie & van der Veer (2003) propose a hierarchical approach for organizing design patterns into five levels (from top to bottom): business goals, posture, experience, task and action patterns as illustrated in Figure 1. This top-down approach allows designers to identify user activities, goals, and contexts level by level, ensuring the design addresses the correct issue (Norman 2013).

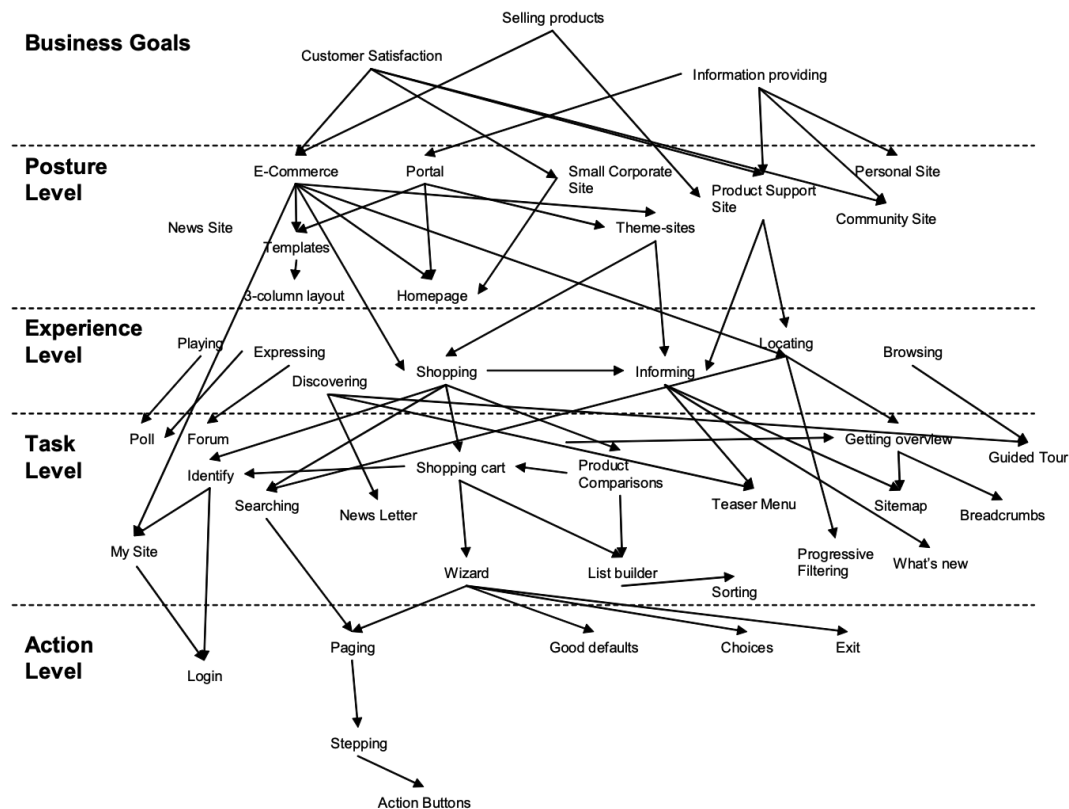


Figure 1: Partial pattern language for web design, focusing on shopping (van Welie & van der Veer 2003).

Posture Type Patterns

As van Welie & van der Veer (2003) describe, posture represents the second highest level of abstraction, it can be understood as the overall purpose, character or "stance" of a site or application, for example an e-commerce or news website. It is closely tied to a system's reasons for existence and the type of experience it aims to deliver (Cooper et al. 2014). To illustrate this concept, van Welie & van der Veer (2003) offer an analogy: "It is like deciding whether you are going to design a 'sports car', a 'family saloon car', a '4x4' or a 'city car'." Selecting a posture for the website assists in informing concept-level design choices, for instance, a layout and navigation, through related lower-level patterns.

Because each posture reflects different user expectations and modes of attention (Cooper et al. 2014), it becomes a structuring force within a pattern language (van Welie & van der Veer 2003): patterns must align with the posture's demands for cognitive load, visual density, responsiveness, and interaction flow. In other words, van Welie & van der Veer (2003) conclude that different postures represents different types of patterns, not merely in terms of their application but in the way they are conceived and structured.

A practical illustration of this relationship is provided by Cooper et al. (2014), who shows how different interaction postures shape modes of engagement and interaction

structure. These postures give rise to distinct interaction patterns aligned with specific interaction flows (Tidwell 2010, Seffah 2010). For example, Porsche's e-commerce platforms, in-car infotainment systems, and service-booking interfaces embody Porsche's own collection of interaction postures, resulting in interaction patterns optimized for their respective contexts (Cooper et al. 2014). However, such patterns do not necessarily generalize across domains: platforms such as Salesforce's CRM or Adobe's creative tools pursue different UX goals which guide their interaction posture and, consequently, the interaction patterns that emerge (Norman 2013, Hassenzahl 2008, Cooper et al. 2014, Seffah 2010).

Experience Patterns

Experience patterns describe activities and are typically expressed as verbs, such as "shopping", "playing", and "browsing" van Welie & van der Veer (2003). They can be understood as high-level user goals that users recognize during their initial interaction with a system. For example, in automotive design, experiences such as "sporty driving behavior" or a "luxury feeling" represent the primary outcomes users seek to achieve (van Welie & van der Veer 2003).

In addition, experience patterns account for the paths users take to reach these goals, their ability to perceive and process information, and their satisfaction with the resulting outcomes. Accordingly, van Welie & van der Veer (2003) argue that experience patterns guide the selection, combination, and application of lower-level design patterns, enabling designers to construct interfaces that support a coherent and intended UX.

Systems often support a primary experience alongside multiple secondary ones. For example, in e-commerce, the primary experience may be "shopping" while secondary experiences include "community-building" or "information gathering" (van Welie & van der Veer 2003). Designers must balance these experiences, often combining elements from secondary experiences to facilitate certain experiences for the users.

Task Patterns

Task patterns represent concrete and well-formulated solutions for small yet specific user activities that contribute to a connected higher-level experience. This aligns with earlier work in Human-Computer Interaction (HCI), where task patterns have been described as capturing the tasks users perform and the ways in which these tasks can be supported (Mahemoff & Johnston 1998). This notion is further elaborated by van Welie & van der Veer (2003), who illustrate the concept through an example in which a shopping cart pattern, situated within the broader e-commerce experience, is linked to task-level instances such as a wizard or a list builder.

According to van Welie & van der Veer (2003), tasks are defined as sets of interactions with one or more objects that are undertaken to resolve a problem. Additionally, Mahemoff & Johnston (1998) suggest that task pattern languages can be interpreted as sequences of tasks that collectively shape the overall UX. For instance,

a "View Document" pattern may encompass tasks such as opening a document and browsing its contents (Mahemoff & Johnston 1998).

Furthermore, Borchers (2000b) emphasize that pattern languages explicitly connect higher-level context patterns to lower-level refinements through structured references. As posture and experience patterns establish the foundational framework, task patterns serve as a mediating layer that links higher-level elements to the lowest-level components of the design (van Welie & van der Veer 2003).

Task patterns have also been characterized as small, modular units, a property highlighted by Mahemoff & Johnston (1998). This observation supports the claim by van Welie & van der Veer (2003) that task patterns are largely independent of specific domains. Consequently, task patterns can be regarded as highly reusable across different contexts and readily adaptable to multiple experiences (van Welie & van der Veer 2003, Mahemoff & Johnston 1998). Although the underlying task, for example, "Open document", may remain stable, the specific steps and interactions through which the task is supported may vary depending on the contextual setting (Mahemoff & Johnston 1998).

Action Patterns

Action patterns can be understood as the lowest-level reusable elements, or "building blocks", within the hierarchical pattern language described by van Welie & van der Veer (2003) as a multi-level tree structure. These patterns represent specific user actions, such as submitting or clearing a form (van Welie & van der Veer 2003). On their own, action patterns carry limited meaning; instead, their significance emerges through association with higher-level tasks, such as completing an order or progressing to the next step in a workflow (van Welie & van der Veer 2003). In this way, action patterns derive their meaning from the task contexts in which they are embedded.

Action patterns capture micro-level interactions and may therefore be regarded as the smallest reusable units within pattern-based design (Borchers 2000b, Tidwell 2010). Making explicit the relationship between such atomic actions and higher-level task flows supports designers in developing a more structured understanding of interaction sequences and their contribution to the overall UX (Norman 2013, Mahemoff & Johnston 1998, van Welie & van der Veer 2003).

2.5.3 UX Patterns

In the HCI literature, UI and UX patterns are frequently discussed under the broader umbrella of (interaction) design patterns (van Welie & Tr  tteberg 2000, Borchers 2000b, Seffah 2015). The term *patterns* is often used as shorthand for these pattern constructs (Tidwell 2010, van Welie & van der Veer 2003). Naming conventions vary across authors and collections, with overlapping labels such as interaction patterns, UI or HCI design patterns, usability patterns, and UX patterns (Seffah 2015, Folmer 2015).

Despite UI and UX being closely related and mutually reinforcing, they are distinct concepts (Garrett 2010). Garrett (2010) clearly separates the interface-level design

from the higher experience-level concern while emphasizing that they work closely together. The definition of UX patterns remains somewhat ambiguous and continues to evolve incrementally, as consistent terminology and formalization have yet to be fully established (Seffah 2015, Bottoni et al. 2011) which highlights the inherent complexity of the field (Bottoni et al. 2011).

Broadly, from the definitions of UX and experience itself, UX patterns can be drawn to be associated with behavioral aspects, user goals, and emotional responses (Norman 2013, Hassenzahl 2008, Borchers 2000b, van Welie et al. 2001). Through UX design, these patterns seek to identify novel ways to facilitate interaction, satisfy user needs, and deliver seamless, meaningful experiences across both macro- and micro-level interactions (ISO 2019, McCarthy & Wright 2004). Compared to technical considerations, UX patterns typically exhibit a broader and more conceptual orientation (Blackwell & Fincher 2010).

Alexander et al. (1977) emphasizes the significance of the human experience in his work on architectural patterns, "Light on Two Sides of Every Room", illustrating that patterns are not isolated solutions but part of a network of patterns that generates synergy, as further discussed by van Welie & van der Veer (2003). Consequently, UX patterns, both among themselves and together with UI patterns, are often deeply intertwined (van Welie & van der Veer 2003, Borchers 2000b). For example, UI patterns frequently enable the existence and fundamental characteristics of UX patterns (Garrett 2010, van Welie & Tr tteberg 2000, van Welie & van der Veer 2003, Norman 2013).

The significance of UX design patterns stems from their ability to translate abstract qualities into actionable and reusable design strategies, supporting systematic reasoning and reuse in design practice (Blackwell & Fincher 2010, van Welie et al. 2001, Seffah 2015). In doing so, UX patterns contribute to consistency, usability, and meaningful engagement across diverse contexts (Hassenzahl 2008, Norman 2013). Beyond promoting visual coherence, UX patterns also guide the design process by foregrounding user agency and goal-directed interaction (Alexander et al. 1977, van Welie & van der Veer 2003, Blackwell & Fincher 2010), thereby shaping how users act, decide, and progress through an experience (Garrett 2010).

2.5.4 User Interface Patterns

UI design refers to an iterative process through which design decisions are made and refined in order to achieve a functional and effective interactive solution (Roth 2017). Within this process, UI patterns emerge as recurring solutions to common problems in user interface design, streamlining both design and development by enabling designers to reuse proven interface solutions rather than repeatedly reinventing them (Tidwell 2010, van Welie et al. 2001, Dearden & Finlay 2006). Building on this, Toxboe (2020), Dearden & Finlay (2006) present a catalog of standardized UI patterns applicable across different contexts, providing a shared foundation for fostering consistency across digital platforms.

UI patterns primarily focus on interface-level elements and visual structure, operating as reusable building blocks within UID (van Welie & Tr tteberg 2000,

Tidwell 2010). Typically realized as discrete interface components, such as wizards, calendar pickers, navigation tabs, and notifications, UI patterns provide the concrete affordances through which users interact with a system (Toxboe 2020). Therefore, they can possibly be compared to functional patterns introduced by Kholmatova (2017) that are concrete UI elements, such as buttons or menus that enable specific tasks. In this role, UI patterns enhance the perceivability and usability of interface elements and contribute to visual coherence, thereby supporting higher-level UX patterns that may encompass multiple UI patterns to shape familiarity, trust, and user engagement across the overall experience (Tidwell 2010, Roth 2017, Borchers 2000b).

Familiarity is often linked to users' mental models that are shaped and updated by the perceptions and interactions with a system (Norman & Draper 1986, Norman 2013, Johnson-Laird 2010). According to Hornbæk et al. (2025), mental models facilitate users' ability to reason about and anticipate system behavior, thereby reducing effort when engaging with digital interfaces. This is illustrated by a common convention in e-commerce interfaces, in which the recognition of a shopping cart icon is associated with the expectation that selected items can be accessed (Norman 2013, Nielsen 2024). This indicates that users bring expectations from their past experiences (Hornbæk et al. 2025). Thus, when applied correctly, UI patterns can reduce cognitive load and improve user satisfaction by leveraging users' prior experience and established conventions (Sweller 1988, Norman 2013). By increasing predictability and supporting recognition over recall, such UI patterns can contribute to more intuitive and user-friendly interfaces (Nielsen 2024, Hassenzahl 2008, ISO 2019)

2.6 Design Systems in HCI

DSs have emerged as a transformative approach to scaling and reusing design across digital products (Nguyen n.d., Suarez et al. 2019, Derleth 2018) which can be seen as a growing number of open source DSs on the internet. Their adoption has gradually increased alongside technological shifts and the increased complexity of digital ecosystems (Nguyen n.d.). Despite a research interest, (Werle 2021) observes that the definition of DSs remains somewhat divergent across academic and practitioner literature. Broadly, DSs aim to establish a consistent visual language by guiding the reuse and structured composition of UI components (Suarez et al. 2019, Beres 2025, Kholmatova 2017). They also foster better mutual collaboration between multidisciplinary teams, especially between designers and developers. For designers, DSs inform the optimal arrangement of UI elements to achieve desired UX outcomes, while developers benefit from implementation references and standardized solutions (Suarez et al. 2019, Lamine & Cheng 2022, Beres 2025). Additionally, DSs function as an instrument to expressing brand identity, vision, and presence through visuals, interactions, and the overall feeling of a system (Werle 2021).

Design patterns, and especially the pattern libraries they form, are often referred alongside DSs due to their shared goal of promoting consistency and unified design. However, their major distinction derives from their scope and focus. Design patterns are primarily concept-driven (Werle 2021), whereas DSs address both the visual design language and the implementation of reusable UI components, embedding brand

values directly into UX (Beres 2025, Suarez et al. 2019). Design patterns can be considered a subtopic of DSs (Kholmatova 2017). As Kholmatova (2017) explains, DSs aim to embed patterns within a broader framework of collaboration, governance and evolution. To summarize, DSs operate as a comprehensive toolbox, while pattern libraries provide structured references and guidelines that support the use of the tools.

2.6.1 Atomic Design Framework

The rise of mobile technology revolutionized design, making responsive design one of the main functional requirements (Ragab Khalaf 2019, Werle 2021). In relation to this, Frost (2016) introduced an extensive five-layer framework known as the atomic design principle to UI elements. Drawing inspiration from chemistry, Frost (2016) illustrated how atoms as the smallest components combine to form molecules, which in turn build even larger structures, such as organisms, templates and pages. Derleth (2018) emphasize that as one moves from atoms to templates, the degree of abstraction gradually decreases, while the effort required for implementation and management increases.

Atomic methodology has gained universal adoption in UID as it offers a systematic and deliberate way to develop reusable style guides. Frost (2016) highlights that the framework is not linear but rather a mental model to illustrate the UID as a unified system composed of smaller, interrelated components. Furthermore, by accelerating design workflows, atomic design allows designers to discover and facilitate the recurring elements, which fosters consistency and scalability across digital products.

Atoms

Atoms, as the smallest building blocks, operate as the cornerstone elements in every UI. These often include HTML elements that cannot be further divided without losing their purpose or usability; examples provided by Frost (2016) include "form labels, inputs and buttons". These stand-alone components can include more abstract elements, such as states, sizes and font weights, that guide its use and functionality within the broader DS (Frost 2013).

Atoms offer the raw structure and foundational behavior of a DS. However, on their own, atoms are not especially useful, but assembling them like Lego blocks will accomplish greater and more functional elements to support the UI's goals (Frost 2013). The primary objective of a DS is to establish a consistent visual language by providing a shared set of foundational elements, ensuring that designers operate from a common reference point across projects (Suarez et al. 2019, Werle 2021).

Frost (2016), Shah (2023) observe are most commonly updated during foundational design changes, such as re-branding, accessibility improvements, design refinements or the incorporation of new technologies. Updates to atoms, such as changes in color schemes, typography and spacing, can be easily applied across the system due to the deeply intertwined architecture. In addition, by modifying the behavior or appearance of a button, such as the hover or pressed states will automatically cascade to all larger

elements in the system that use it. This fosters scalability and reinforces consistency by reducing maintenance effort.

Molecules

When one or more atoms are combined, they form a compound molecule that operates as a functional cluster. Molecules represent small clusters of elements that possess own properties and functionalities that allows users to execute a specific task (Frost 2016, Seffah 2010). For example, a form label, a search field, and a button can be combined together to establish a search form molecule (Frost 2016). These molecules serve as the structural core of the DS, offering more flexible and functional behavior than individual atoms. Each atom corresponds to a certain functionality; the input atom handles user input, while the button atom manages form submission. Together, they form a compact, reusable molecule that can be seamlessly integrated into broader contexts where search functionality is needed.

Frost (2016) emphasizes the principle of "do one thing and do it well", which is related to the simplicity and clarity of design patterns. It is more desirable to construct simple molecules as they enhance scalability and reusability of the interfaces.

Organisms

Organisms are more complex UI components that are composed of smaller elements such as molecules, atoms and possibly other organisms that can be similar or different types. They represent groups of related elements that are less abstract and more concrete as they demonstrate tangible functionality within the interface (Frost 2013). Organisms form cohesive and functional units, which are used in full-page layouts; common examples define UI areas like the header, footer, cards or navigation elements. As Frost (2016) mentions, a header is a typical organism that consists of smaller elements, each one having its own property and functionality.

The foundational goal of organisms is to promote reusability across multiple pages or views, enabling cohesive visual appearance and saving development time (Frost 2016). Another benefit is related to scalability as designs can establish larger templates efficiently from smaller units, reusing functional components rather than starting from scratch (Frost 2013, Lamine & Cheng 2022, Derleth 2018).

Templates

Templates are high-level design structures that organize low-level elements, often organisms, into a defined layout in actual action (Frost 2013). For instance, a homepage template that positions a header organism at the top demonstrates how lower-level components are composed into a coherent page structure, supporting designers' understanding of the interface as a whole. Templates slowly increase in fidelity, from low-level wireframes to finalized designs, which inevitably leads to incremental development typical for HCD (Frost 2013).

As UI development is inevitably incremental and iterative, the construction of templates derives from initially sketching the structure of a page content rather than fixating on polished outcomes ([Sangiorgi et al. 2012](#), [Larman & Basili 2003](#)). [Sangiorgi et al. \(2012\)](#) emphasizes that early sketching can lead a smooth, cost-efficient progression from early prototypes to final user interfaces. [Frost \(2016\)](#) emphasizes the importance of defining image dimensions and character limits for headings and text fields at this stage. These type of constrains help maintain visual rhythm and guide a user's attention effectively while possibly avoiding clutter ([Sangiorgi et al. 2025](#), [He et al. 2024](#)).

[Boulton \(2012\)](#) interprets that templates assist in establishing posture by reinforcing the underlying content structure and structural hierarchy of the interface, which allows different content types to be articulated consistently within the UI. This process clarifies the behavior of constituent interface components, enabling templates to serve as a shared point of reference between designers and developers ([Frost 2016](#)). Although templates occupy a higher level within the design hierarchy than atoms or molecules, they are inherently dynamic, evolving in response to brand evolution, technological change, and emerging user requirements ([Frost 2016](#), [Larman & Basili 2003](#), [Raneburger et al. 2014](#)).

Pages

Pages are concrete realizations of templates, demonstrating the final UI with actual, representative content ([Frost 2016](#)). They combine various elements into a complete experience, and the behavior of components used together is revealed. [Frost \(2016\)](#) expands the earlier homepage example: On a page level, the homepage template is filled with sample text, images and media to show the true functions and interactions within the design.

Pages often represent the final structure, making them the most tangible layer of atomic design. [Frost \(2016\)](#) stresses the importance of pages as they validate the strength of the DS. At this point, designers and developers can perceive the durability of design patterns as the real content and composition is applied ([Frost 2016](#), [Krug 2014](#)). Pages also expose the edge cases, such as content overflow, localization issues or accessibility gaps, that are important to be identified before final deployment ([Frost 2016](#), [Nielsen 2024](#), [ISO 2019](#), [He et al. 2024](#)). If visual appeal or functionality falls short, revisions can be made to molecules, organisms, and templates to better accommodate the content requirements ([Frost 2016](#)).

In addition, [Frost \(2016\)](#) narrate that pages serve as a sandbox for testing variations within templates, an essential step in building resilient and adaptable DSs. Different user scenarios can be demonstrated with the same underlying template by arranging and applying new elements in various order. For instance, two users having different amount of items in their shopping cart ([Frost 2016](#)). By constructing and arranging the elements, molecules, organisms and templates, designers can introduce variety into the interfaces ([Frost 2016](#)), ultimately fostering more flexible and resilient DSs.

2.6.2 Benefits of Design Systems

The growing complexity of digital interfaces has accelerated the adoption of DSs. These systems have emerged as a powerful tool for managing scalability, ensuring consistency and quality, as well as establishing a common language between designers and developers (Lamine & Cheng 2022, Idean 2019). Lamine & Cheng (2022) introduce Google’s perspective on DSs as they highlight that these systems drive for minimizing effort through supporting learning, fostering collaboration, and aligning processes with desired outcomes. Similarly, Frost (2016) has acknowledged the power of DSs. According to Frost (2016), Werle (2021), some of the most influential benefits include consistency, shared vocabulary, education, speed and testing.

Consistency

As UIs grow more complex, both architecturally and in terms of usability, maintaining consistency becomes increasingly important. DSs ensure that every component of the UI follows the same visual guidelines, hence, reducing design drift and eliminating drastic variations (Frost 2016). They define clear rules for the application of elements, also taking into account some of the possible corner cases. Beyond visual consistency, DSs also enforce behavioral consistency across digital platforms. Reusable components and standardized styles ensure that users experience a coherent interface regardless of the use context (Frost 2016). Frost (2016) emphasizes that consistency builds trust, reduces cognitive load, and strengthens brand identity.

Shared Vocabulary

Consistency, combined with the notion of a shared design vocabulary, fosters collaboration by promoting unified design practices across various digital platforms. This shared understanding reduces miscommunication and supports more efficient decision-making, especially in cross-functional teams (Frost 2016, Werle 2021, Carey et al. 2025). Beyond internal coordination, consistent application of design rules contributes to users’ perception of a single, continuous service, enabling smoother transitions between digital products. A shared vocabulary allows designers to provide structured and actionable peer feedback to facilitate collaborative design, as common rules and terminology frame critique and discussion (Dalsgaard et al. 2014, Krakowski et al. 2023). Similarly, DSs streamline developers’ work by establishing mutual understanding across disciplines, leading to faster development, validation and more effective cross-disciplinary feedback loops (Lamine & Cheng 2022, Carey et al. 2025).

Education

Education fosters mutual understanding among various stakeholders (McCarthy et al. 2020, Brubaker et al. 2022). Lee (1997) and Werle (2021) claim DSs (as shared documentation artifacts) can support this by making design rationale and logic explicit, helping stakeholders outside design better understand decisions. Rather than presenting interfaces as arbitrary, they demonstrate the way individual decisions are curated

([Mirabito & Goucher-Lambert 2022](#), [Lee 1997](#)) and collectively contribute to the overall UX.

In practice, this educational benefit is often realized through concrete artifacts such as pattern libraries, which function as living textbooks for the DS. By outlining both specifications and the underlying design philosophy, pattern libraries document not only the visual realization of components, but also the underlying rationale informing their design ([Frost 2016](#)). As a result, these resources facilitate onboarding by enabling new team members to quickly understand the behavior, constraints, and interactions of UI elements.

A pattern library acts as a living textbook for the DS because it outlines the specifications and design philosophy ([Frost 2016](#)). This resource facilitates onboarding by assisting new team members to quickly comprehend the behavior and interaction of UI elements. Pattern libraries document recurring problems and reusable solutions (patterns/components), whereas broader documentation artifacts (e.g., design rationale and shared system documentation) additionally communicate the reasoning and intended use that situates these elements within a wider design context ([Waller & Delin 2010](#)). They typically provide more hands-on guidance by illustrating the application of components, patterns, and principles across varying scenarios and product contexts, supporting both learning and practical implementation across disciplines ([Lee 1997](#)).

Speed

With predefined components and design rules, teams avoid reinventing the wheel for every new screen or feature ([Frost 2016](#)). The DS aims to offer all the up-to-date elements and their variations which can be directly applied to new designs ([Lamine & Cheng 2022](#), [Idean 2019](#), [Werle 2021](#)). This allows designers to focus more on solving user, usability or accessibility concerns rather than making pixel-perfect layouts ([Lamine & Cheng 2022](#), [Frost 2016](#)). [Idean \(2019\)](#) emphasize as a result, design and prototyping processes are accelerated, allowing multiple units and teams to work in parallel while maintaining alignment.

Testing

DSs promote modularity through atomic design principles according to which UI components and patterns are built as isolated, reusable units. [Frost \(2016\)](#) stresses that modularity allows more efficient and targeted testing as individual components can be validated independently of the rest of the UI. Moreover, component-based UI architectures allow developers to isolate, identify, and fix issues at the component level, minimizing unintended effects on neighboring UI components or overall page layout ([Werle 2021](#), [Thomas 2025](#), [Mounika 2021](#)).

3 Related Work on Design Systems and UX Patterns

3.1 Current State of Design Systems

The methodology behind DSs has gradually evolved and matured from industry practice ([Lamine & Cheng 2022](#)), gaining widespread adoption among major technology companies such as Microsoft, Apple, and IBM ([Werle 2021](#)). As a result, an increasing number of DSs are now publicly accessible, enabling organizations to benchmark their own systems, learn from one another, and continuously refine their approaches. The openness of DSs encourages collaborative knowledge sharing, enabling UX best practices to emerge and stabilize through repeated use of shared patterns and structural conventions, which in turn accelerate UX design ([Lamine & Cheng 2022](#), [Feng et al. 2023](#)). In parallel, academic research has contributed to a deeper understanding of DSs, contributing to the standardization of terminology and clarifying their role in digital product development.

According to [Lamine & Cheng \(2022\)](#), constructing a DS entails four key activities: auditing the current visual design, defining a coherent design language, assembling a UI pattern library, and generating thorough documentation. This conceptual framework is reflected in several widely recognized DS, including IBM’s Carbon DS ([Carbon Design System n.d.](#)), the Salesforce Lightning DS ([Lightning Design System 2 n.d.](#)), and Shopify’s Polaris ([Polaris React n.d.](#)). While differing in scope and domain, these systems consistently organize their DS guidance around comparable design artifacts, foundational principles, reusable components or patterns, and supporting documentation, closely corresponding to the stages described by [Lamine & Cheng \(2022\)](#).

3.1.1 Overview of Current Design Systems

At the foundation of each DS lies a set of clearly defined goals, such as accessibility, personal values, and a shared vision. Each system reflects its unique focus: [Carbon Design System \(n.d.\)](#) emphasizes accessibility, [Polaris React \(n.d.\)](#) is tailored for e-commerce, and [Lightning Design System 2 \(n.d.\)](#) is built to streamline the mutual exchange of information between companies and customers. These systems also promote distinct benefits for various roles, including designers, developers, and other stakeholders.

DSs typically articulate their purpose through extensive documentation that supports both design and implementation work. Core resources commonly include design guidelines, UX writing standards, implementation instructions, component libraries, and reusable UI patterns. In particular, UI components are often documented in depth using multiple complementary formats. Examples include embedded Figma components or UI kits to illustrate visual structure, Storybook previews (or links to GitHub repositories) to demonstrate states and interactions, and framework-specific code examples, most often in React, to support technical implementation. These resources enable designers and developers to explore visual structure, behavior, and

technical implementation in parallel. In contrast, design patterns are frequently presented in a more abstract manner, relying primarily on static images and textual descriptions of do's and don'ts, with limited interactive or code-level representations.

Across contemporary DSs, UI components are generally documented using a shared set of practices. Systems such as IBM's *Carbon Design System* (n.d.), Salesforce's *Lightning Design System 2* (n.d.), Shopify's *Polaris React* (n.d.), and the *Kela Design System* (n.d.) consistently present components through structured documentation that combines visual references and implementation guidance. Components are typically introduced either as isolated elements or within composed UI contexts, followed by usage recommendations, behavioral specifications, and accessibility considerations.

Within this broadly shared documentation approach, certain systems distinguish themselves through a more advanced use of interactive and contextual visualization. In particular, *Carbon Design System* (n.d.) and *Kela Design System* (n.d.) stand out by offering live, interactive demos that explicitly demonstrate component behavior, state transitions, and usage scenarios, enabling deeper exploration beyond static representations. While all examined systems follow comparable structural conventions for component documentation, Carbon and Kela exemplify a more thorough and experiential presentation of UI components.

Together, these insights illustrate the current state of DSs as practice-driven frameworks that show increasing alignment in maintaining and documenting design knowledge. This alignment emerges through the adoption of shared best practices, enabling them to remain open, adaptable, and continuously updated. In this sense, DSs operate as living documentation that supports consistency and efficiency while reflecting an organization's evolving design philosophy and strategic priorities (Ayres 2025, Lamine & Cheng 2022, Idean 2019).

3.1.2 Definitions of Design System

Due to the novelty of the term DS, no single, universally accepted definition has yet emerged (Kholmatova 2017). As Werle (2021) notes, this has led practitioners to adopt terminology that best fits their disciplinary context and team practices. In particular, software-engineering perspectives often emphasize the component-based and programmable aspects of DSs (Lamine & Cheng 2022), while Kholmatova (2017) observes that HCI and UX-oriented perspectives focus on shared languages, principles, and collaborative design practices. This disciplinary breadth contributes to the diversity in how DSs are defined and discussed (Werle 2021).

Werle (2021) has assembled and categorized various definitions of DSs, each reflecting varying perspectives. Among these, three definitions were the most influential within the collection (Werle 2021):

"A design system is a set of interconnected patterns and shared practices coherently organized to serve the purpose of a digital product. Patterns are the repeating elements that we combine to create an interface: things like user flows, interactions, buttons, text fields, icons, colors, typography, microcopy. Practices are how we choose to create, capture, share and use those patterns, particularly when working in a team."

— [Kholmatova \(2017\)](#)

"Design systems lay out the core elements and components of visual and interactive design for the devices we use every day—cellphones, tablets, laptops, desktops, and car dashboards, to name just a few. They offer guidance for layout, shape/form, color palettes, buttons, forms, tables, typography, user task flows, and can offer advice for content, branding, tone, and personality. Design systems enable designers and developers to create a consistent and cohesive product or suite of products."

— [Churchill \(2019\)](#)

"A series of documented elements, components, and regions that include both design and front-end guidelines. The documentation contains live code examples, allowing cross-functional teams to easily reuse styles and components in several instances across an application. A design system also includes underlying design principles, rules, and guidelines that help a team build one or multiple products "

— [Vesselov & Davis \(2019\)](#)

These three distinctive definitions propose slightly different perspectives on DSs. [Kholmatova \(2017\)](#) projects a DS as a structured ecosystem of reusable UI elements, offering best practices as patterns. Additionally, this definition emphasizes close collaboration within multidisciplinary teams through a mutual definition and documentation of these patterns. [Churchill \(2019\)](#), on the other hand, approaches DSs from the perspective of visual consistency across everyday devices, and further notes that they often embody a predefined UX vision through tone, personality, and branding. Meanwhile, [Vesselov & Davis \(2019\)](#) present DSs as a tool for reusing design and front-end components as well as principles across applications, with a primary focus on enabling cross-functional collaboration.

3.2 Task-Based Design

As the complexity of interactive systems increases, [van Welie \(2001\)](#) proposes the Designing for Users and Tasks from Concepts to Handles (DUTCH) design process methodology, demonstrated in Figure 2, which is led by a thorough understanding of user tasks. The DUTCH process consists of two models: Task Models 1 and 2. Task Model 1 represents the "current task model" ([van Welie 2001](#)) which facilitates analysis of the current situation. This analysis serves as a foundation for evaluating new systems and informing the design process. Task Model 2, similarly structured as Model 1, reflects the "future task model" ([van Welie 2001](#)) by outlining expectations for task performance within the envisioned environment.

Building on these principles, [van Welie \(2001\)](#) proposes several task analysis methods associated with Task Model 1, as they describe the present state. Among these, Hierarchical Task Analysis (HTA) and Task World Ontology (TWO) are particularly

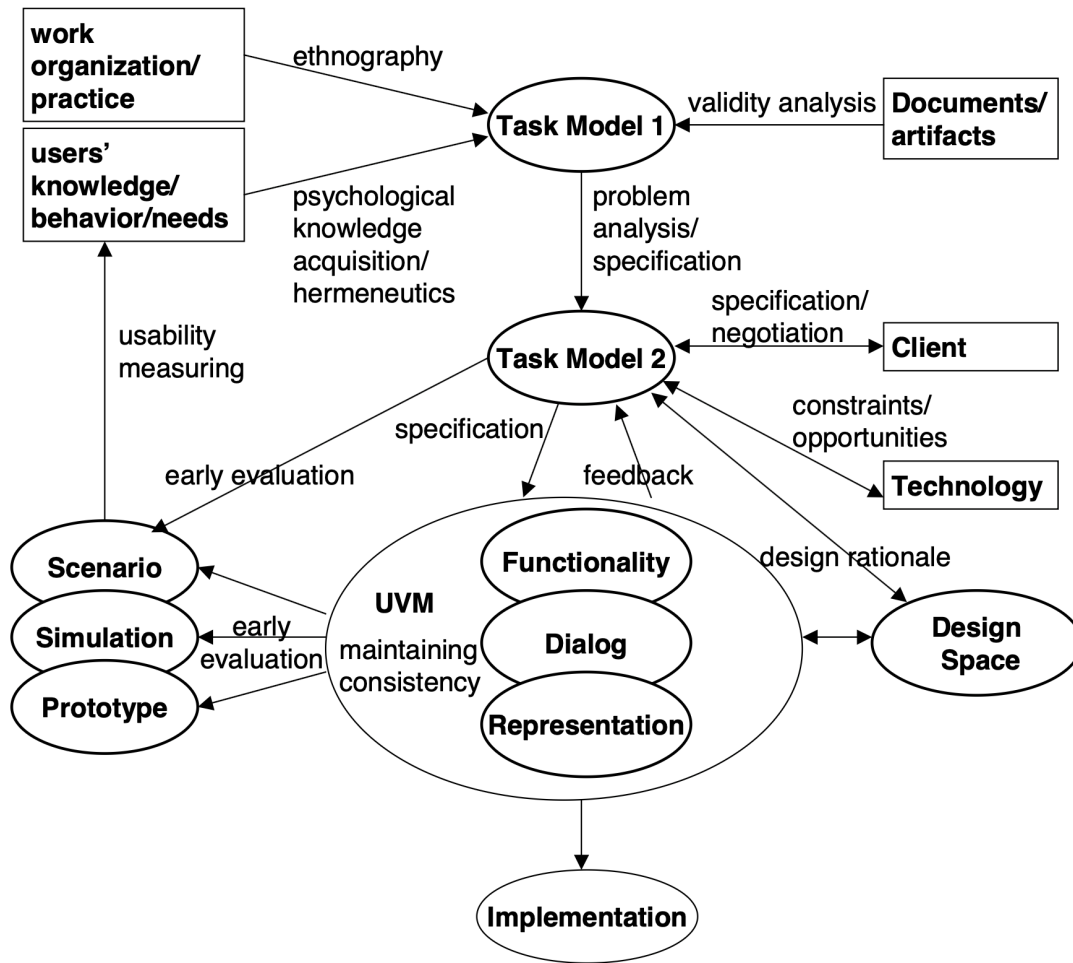


Figure 2: DUTCH design process (van Welie 2001).

promising for validating UX patterns. HTA mainly addresses micro-level interactions but can also be applied to model broader goals. In contrast, TWO emphasizes more macro-level aspects, focusing on the conceptual and contextual framework of tasks and the interconnected relationships of various actors and actions.

3.2.1 Hierarchical Task Analysis

HTA, originally introduced by Annett and Duncan (Annett & Duncan 1967), has influenced numerous subsequent methods built upon its principles (van Welie 2001). According to French et al. (2019), it applies the groundwork concepts of task analysis which provide a conceptual perspective "road map" for examining user interactions in detail. Task analysis offers insights for identifying tasks and hierarchically categorizing them into groups, while also specifying the conditions and sequences under which tasks and subtasks are performed (Hornbæk et al. 2025), enabling these observations to be translated into accurate and effective workflows. Moreover, this method facilitates close collaboration among various stakeholders by visualizing task flows thoroughly without focusing excessively on design or implementation details.

Task analysis also encompasses different scenarios that might occur during task execution to achieve one specific goal. To clarify, it specifically focuses only on one user performing all tasks to achieve a single goal ([IxDF - Interaction Design Foundation 2017](#)). Moreover, [IxDF - Interaction Design Foundation \(2017\)](#) emphasizes it does not model multiple actors or distributed responsibilities. [Hornbæk et al. \(2025\)](#) provide an example in which task analysis is used to demonstrate the sequence of tasks carried out by a nurse during interaction with a medical application.

HTA often considers potential failures or errors which makes it an effective tool for identifying risks and their root causes. Through use case identification, specific tasks and associated sub-tasks are recognized, which can clarify the steps users carry to successfully achieve their goals. Tasks represent higher-level objectives defined by the user, while sub-tasks consist of individual steps that support the completion of these broader tasks.

HTA mainly focuses on micro-level interactions and is heavily task-centered; it consists of the step-by-step decomposition of operations and plans ([van Welie 2001](#)). Operations describe the actions and decisions users take to accomplish specific goals, whereas plans specify the conditions under which each operation is performed ([van Welie 2001](#)). [French et al. \(2019\)](#) propose a three-step framework: 1) identify use case(s), 2) gather information on task execution, including steps, interactions and other associations discovered through observation or walkthrough, and 3) construct the HTA diagram, beginning with the main task and decomposing it into smaller sub-tasks. Additional details often distinguish different workflows, such as successful versus unsuccessful actions. For example, [Hornbæk et al. \(2025\)](#) note that plans typically include step-by-step instructions ("do this, then that"), conditional rules ("if this happens, then do that"), parallel tasks that allow simultaneous steps, and stop rules indicating when a task can be interrupted.

As an illustrative example, consider account onboarding or login as the primary goal for accessing digital software, shown in Figure 3. Each task is numbered sequentially, reflecting its execution order. Plans specify the sequence required for successful authentication and outline alternative scenarios, such as registration for new users versus login for returning users. At the sub-task level (second tier of the hierarchy), plans guide conditions and sequences for completing steps, such as password recovery.

[French et al. \(2019\)](#) highlight the importance of HTA in the design process, as it helps streamline workflows by effectively taking into account potential painpoints and risks in UI design. HTA is valuable for describing user flows in complex systems by visually representing task and sub-task relationships through a numbering system and illustrating their contribution to the overall goal. In addition, HTA can provide alternative paths for completing the same task. Ultimately, this approach enables designers to detect problematic interactions and mitigate usability risks effectively. The resulting insights inform design decisions, preserve scarce resources and prevent costly redesigns that fail to address the underlying issues.

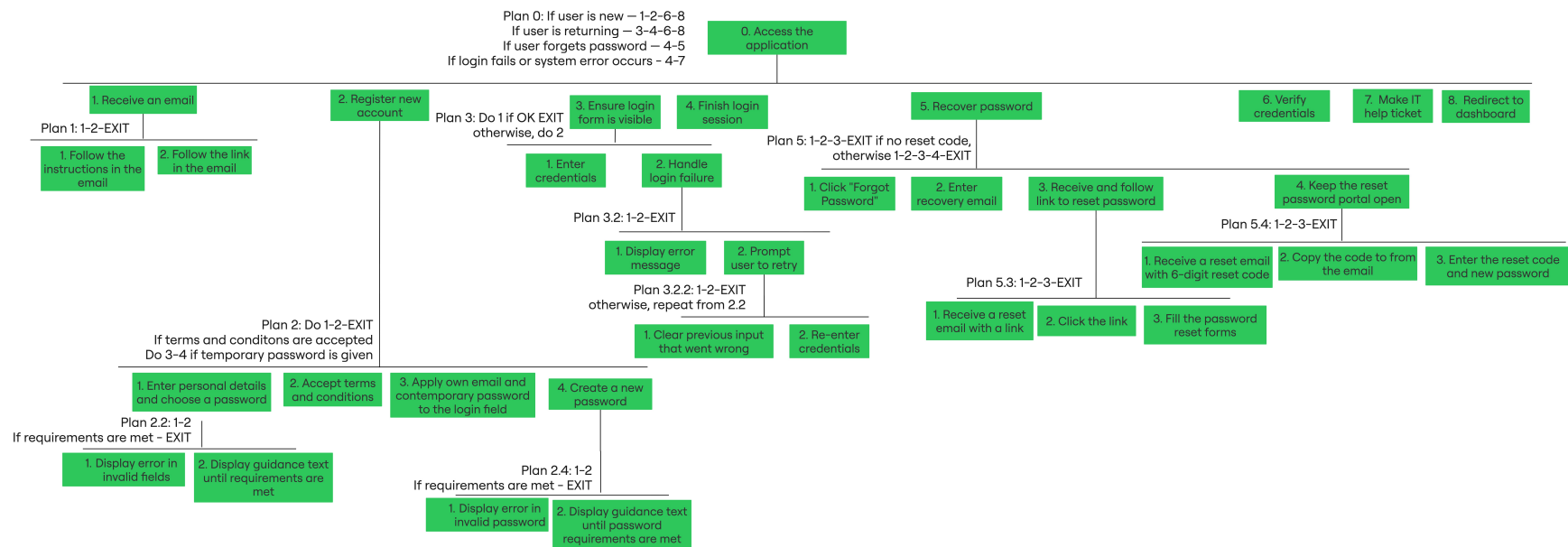


Figure 3: HTA analysis for self-account onboarding, applied from [van Welie \(2001\)](#).

3.2.2 Task World Ontology

TWO underscores the interrelation between users and their natural operation environment, while also taking into account groups of multiple users. As observed by [van Welie et al. \(1998\)](#), a main limitation of classic task models stems from their inability to capture the dynamic nature of environments, thereby neglecting the multi-user perspective.

Modern software systems typically involve multiple users with diverse roles, and many tasks are executed in parallel. TWO attempts to model these multi-user environments ([van Welie et al. 1998](#)). It provides a conceptual framework that extends beyond HTA by emphasizing relationships among essential elements such as tasks, goals, roles, objects, agents and events ([van Welie 2001](#)). The core concepts of the TWO framework can be further detailed through associations such as "uses", "triggers", "plays", "is performed by", "has", "influences", "responsible for" and "is used by" [van Welie & van der Veer \(2003\)](#). In addition, the elements itself can have individual states, such as "start_condition", "stop_condition", "status" and "role". The main elements are illustrated in Table 1.

Specifying these factors is important, as they significantly inform design decisions. Tasks summarize the desired state or goal associated with the various roles of agents while roles represent attributes of the engaged actors. Tasks define which roles are responsible for certain actions, whereas actors embody individual persons. However, system complexity may shift responsibilities, allowing tasks to be delegated or reassigned due to mandates or operational constraints ([van Welie et al. 1998](#)).

For example, in a simple environment a role might be a projectionist responsible for playing the movie. In practice, the movie projectionist could have delegated this task to someone from the snack bar ([van Welie et al. 1998](#)). In such cases, it is often more useful to indicate that a task was done by a role, like administrator or viewer, than specifying the individual actor; especially when the identity of the actor is irrelevant. Events represent uncontrollable occurrences that happen in the environment, such as power outages. While their importance varies, documenting role reactions to unexpected situations can provide valuable insights ([van Welie et al. 1998](#)).

Another example is shown in Figure 4, which encompasses the authentication task while accounting for various roles that function as sub-roles of the agents (users). The figure illustrates separate goal-oriented tasks, role-based interactions and multiple entry points for registration, thereby increasing system transparency.

3.3 UX Patterns in Practice

3.3.1 Documentation of UX Patterns

In his work on architectural patterns, [Alexander et al. \(1977\)](#) revolutionized the discovery and the documentation of patterns. He proposed that patterns encompass a three-part approach; to identify the context, the problem and the solution. In academic research, multiple documentation formats have emerged; some more detailed, others more general. However, a universally standardized structure for documenting design

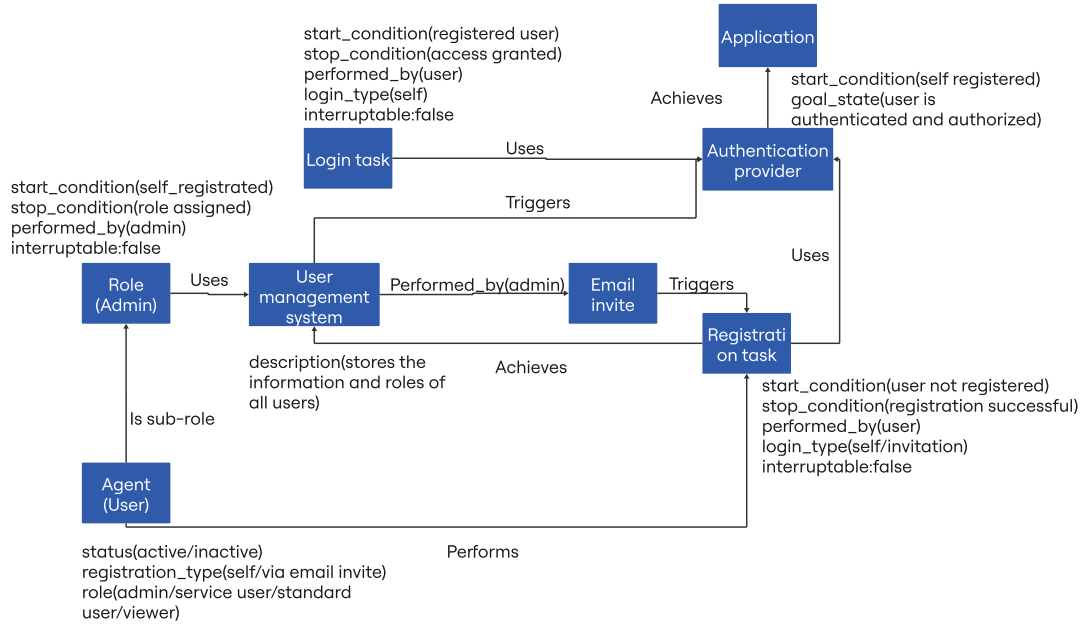


Figure 4: TWO analysis for authentication, applied from [van Welie \(2001\)](#).

Table 1: Core concepts of the TWO ([van Welie & van der Veer 2003](#)).

Concept	Description
Task	An action carried out by agents to achieve a specific goal. Complex tasks can be broken down into smaller sub-tasks.
Goal	A desired outcome in the environment or system of multiple tasks. Goals can also be divided into sub-goals.
Role	A set of responsibilities or tasks assigned to agents. Roles can be hierarchical.
Object	Physical or conceptual entities used or affected by tasks (e.g., documents, tools).
Agent	An entity (usually a person or system) that performs tasks.
Event	A change in the task world that can trigger tasks.

patterns has to yet to be established.

As illustrated in Table 2, the formats used for documenting UX patterns in academic literature vary significantly. [Van Welie & van der Veer \(2003\)](#) has been influenced by [Alexander et al. \(1977\)](#) approach by introducing a three-step framework: *problem*, *use when* and *solution*. Expanding on Alexander's work, van Welie proposed several frameworks including a more detailed version comprising *problem*, *usability principle*, *context*, *forces*, *solution*, *rationale*, *examples*, *known uses* and *related patterns* ([van Welie & Tr tteberg 2000](#), [van Welie et al. 2001](#)). Similarly, [Tidwell \(1999\)](#) adopted a format of *name*, *problem*, *context*, *examples* and *forces* ([van Welie et al. 2001](#)). Alternatively, [Nilsson \(2009\)](#) structured documentation around *name*, *use when*, *how* and *why*. Finally, [Axelsson \(2022\)](#) presented a comprehensive format consisting of

name, description, purpose, applicability, participants, solutions, effects and examples.

Table 2: Comparison of framework structures for documenting design patterns.

Framework	Structure
van Welie & van der Veer (2003)	problem, use when and solution
Tidwell (1999)	name, problem, context, examples and forces
Nilsson (2009)	name, use when, how and why
van Welie & Trætteberg (2000) , van Welie et al. (2001)	problem, usability principle, context, forces, solution, rationale, examples, known uses and related patterns
Axelsson (2022)	name, description, purpose, applicability, participants, solutions, effects and examples

3.3.2 Practical Examples of Existing UX Patterns

As companies have gradually adopted the concept of design patterns, the formats used to document them remain fluid and varied. While academic literature, consisting of works such as those by [Alexander et al. \(1977\)](#) and [van Welie & van der Veer \(2003\)](#), provides theoretical definitions and structures for design pattern language, industry DSs offer practical implementations that reflect real-world applications. Documented patterns from companies, such as Salesforce and GE Healthcare, demonstrate the way design teams operationalize UX principles, often aligning with usability heuristics and ID theories.

A notable challenge lies in the ambiguity between UI and UX patterns definitions. Many companies use the generic term "pattern" without further categorizing them as either UI or UX patterns. Based on these observations, it could be concluded that such division of patterns is challenging. However, when comparing these findings with the categorization provided by [Toxboe \(2020\)](#), it seems that most patterns in various DSs, specifically those from [Porsche Design System \(n.d.\)](#), [Spectrum - Adobe's Design System \(n.d.\)](#) and [Polaris React \(n.d.\)](#), but also others, appear more UI-oriented. Examples include patterns covering chat interfaces, timelines, cards, and filters.

The benchmarking analysis for [Lightning Design System 2 \(n.d.\)](#), [Ethos Design System \(n.d.\)](#), [Carbon Design System \(n.d.\)](#) and [Pajamas Design System \(n.d.\)](#) indicates that the terminology used across the examined DS does not strictly classify patterns as either UI or UX. Since UX inherently encompasses UI elements, companies may avoid explicit categorization to prevent oversimplification of complex interactions or unintended hierarchy between the terms. Several DSs therefore leave the UI/UX distinction unspecified, as enforcing a strict separation may obscure the intended purpose of a pattern, hinder shared understanding across disciplines, and complicate communication between teams working at different levels of the product. Common

examples include login/logout flows, password change and reset mechanisms, and navigation structures, which were frequently observed across DSs.

In conclusion, UX patterns remain an evolving concept, with many yet to be discovered. Currently, no standardized method exists for identifying or documenting them as a part of an existing DS. Therefore, this thesis aims to explore approaches for discovering UX patterns as well as contribute to advancing their documentation and the scientific knowledge in this domain.

3.4 Ownership and Governance of Design Patterns

Clear divisions of ownership and governance for design patterns are often not consistently established (Feng et al. 2023). According to Dearden & Finlay (2006), patterns are frequently developed within relatively small groups and are often grounded in the best practices and tacit understanding of designers. Ownership of design patterns is therefore recommended to be shared within a collaborative, co-working environment, as such shared stewardship supports more effective implementation and deeper collective understanding.

Regarding governance, reflections on DS development teams reveal that companies commonly adopt either a centralized model, where a single team maintains the system, or a federated model in which contributors are distributed across multiple teams (Suarez et al. 2019). Close collaboration among diverse domain experts is strongly encouraged, as emphasized by Kholmatova (2017) and Suarez et al. (2019), because such collaboration provides greater depth, precision, and efficiency during the development process. Building on experiences at Salesforce, Suarez et al. (2019) also introduce a hybrid approach that combines centralized and federated elements, consisting of a core team supplemented by members drawn from additional teams.

Governance should therefore not be conceptualized as the responsibility of a single team, but rather as a workflow in which contributions are passed between groups (Suarez et al. 2019). Designers may begin the process, after which other members can further refine particular aspects. In general, development can be understood as agile, as pattern modules may be created in parallel (Dearden & Finlay 2006). Through structured brainstorming sessions, broader scale, dimensional considerations, and relevant influencing factors can be identified. This aligns with insights from Borchers (2000b), who emphasizes the importance of cultivating a shared design language. Furthermore, the establishment of clear goals is necessary to ensure that teams progress in a coherent direction. Insights provided by da Rosa & Silveira (2022) additionally indicate that an average of 5.5 participants is generally preferable, based on comparisons across studies examining design pattern creation.

With regard to additional participants, specialists such as designers, engineers, HCI experts, and software developers are frequently involved (da Rosa & Silveira 2022). Further inspiration may be drawn from Suarez et al. (2019), who presents a recommended set of team roles for DS development. These various experts, along with descriptions of their responsibilities, are presented in Table 3.

As noted by Borchers (2000b), patterns are fundamentally oriented toward supporting usability, and for this reason it is essential that end-users, those for whom the

interface is ultimately designed, are included in the development process. Insights from this study indicate that it is particularly important for designers, UX writers (or content strategists), and front-end developers to be involved in the initial stages. Through close collaboration with additional experts, design patterns can be more effectively anchored in their real-world application contexts, allowing their behaviors and significance to be articulated in a manner that captures both positive and negative consequences (Borchers 2000b).

Table 3: Roles involved in building and sustaining a DS and their responsibilities, adapted from Suarez et al. (2019).

Role	Rephrased tasks
Designers	Define and refine the system’s visual language, including core UI elements and interaction patterns.
Front-end developers	Implement scalable, modular front-end solutions using clean, efficient, and maintainable code.
Accessibility experts	Assess and ensure compliance with accessibility requirements, such as WCAG and relevant internal guidelines.
Content strategists	Develop and maintain consistent voice, tone, and content guidelines across the system.
Researchers	Investigate user and customer needs through empirical studies and translate findings into actionable insights.
Performance experts	Optimize the system’s performance to ensure responsive behavior across devices and contexts of use.
Product managers	Align the design system with product strategy and customer needs, and prioritize development accordingly.
Leadership (VPs, directors)	Champion the design system vision, secure resources, and align initiatives across the organization.

If the development of design patterns is viewed as a four-phased framework consisting of planning, discovery, validation, and documentation, designers can be positioned to lead the visual and UX-oriented aspects of validation as well as the documentation of the resulting patterns (Borchers 2000b). UX writers can support this work by refining the communicative dimension of the patterns and ensuring that the tone of voice is coherent and precise. Developers can be responsible for translating design patterns into programmable code snippets that reflect the intended interaction logic. Through the combined contributions of these teams, coherent UI solutions and documentation can be produced, accompanied by broad examples that encompass the relevant product families (Borchers 2000b).

3.5 Architecture and Modularity

Section 2 discusses the modality of design patterns, and [Alexander et al. \(1977\)](#) emphasize that patterns exist within a network rather than as isolated solutions. This networked perspective reflects the architectural nature of pattern languages, in which each pattern contributes to a larger system of interdependent elements. These relationships ensure that patterns move beyond solving local problems and instead integrate into a coherent design ecosystem. [Van Welie & van der Veer \(2003\)](#) reinforce this view by highlighting the hierarchical structure and interconnectedness of pattern modules, noting however that such interdependence significantly slows their evolution process. Nevertheless, it is stressed by [Kholmatova \(2017\)](#) that patterns should be evolved with the products to stay relevant.

This architectural perspective extends to system-level design. Patterns operate across multiple scales, ranging from micro-interactions to macro-level workflows, and alignment across these layers is required to maintain consistency and usability. This multi-level structuring reflects Alexander's original conception of architectural design patterns as foundational building blocks for complex systems, later adapted to digital UID.

The underlying goal of DSs is modularity, which provides scalability and flexibility. As noted by [Kholmatova \(2017\)](#), components must be interchangeable and assembled in various configurations to support diverse UX. Modularity enables reduced duplication, promotes consistency, and accelerates development through the reuse of established solutions. In a similar manner, [Frost \(2016\)](#) advocates for Atomic Design, in which UI components are organized into a hierarchy of atoms, molecules, organisms, templates, and pages. This approach emphasizes progressive composition across scales, ensuring that small, well-defined units can be combined into more complex structures while preserving clarity, adaptability, and systematic coherence.

4 Methods and Data

4.1 Research Approach

This research follows a Design Science Research (DSR) approach in a single-company, case-like industrial context and incorporates elements of action research to develop and validate a framework for discovering and documenting UX patterns. The DSR approach offers a systematic, iterative structure that connects theory and practice, enabling the integration of UX patterns into existing DS. Following the guidelines proposed by [Hevner et al. \(2004\)](#), the approach is organized into the three phases of analysis, demonstration, and evaluation. Each phase is designed to bridge theoretical rigor with practical applicability within Vaisala's digital service environment. In the analysis phase, the emergence of a set of manually proposed assumed UX patterns in several established academic models is explored to uncover a generalizable process for identifying them. Also a framework for documenting them is created in this stage based on industry benchmarking, research recommendations, and company priorities. The resulting documentation framework is applied to Vaisala's digital products in the demonstration phase. Finally, the documentation is validated by designers in the evaluation phase.

The resulting design artifact comprises:

- A UX vocabulary to foster mutual understanding among designers and developers (Sections [2.5](#), [2.6](#) and [3.1](#)).
- Insights from academic models, such as top-down interaction analysis and task-based methods, translated into a UX pattern identification process (Sections [2.5.2](#) and [3.2](#)).
- Deliverables in the form of textual documentation, visualization flow charts, and Figma prototypes to clarify and support the implementation of identified UX patterns in real-world settings (Section [4.1.4](#)).

4.1.1 Proposed Definition of Design System in Vaisala

Due to the multidisciplinary and relative novelty of DSs, existing definitions emphasize partly different aspects, ranging from pattern ecosystems and collaborative practices to visual consistency and reusable coded components ([Kholmatova 2017](#), [Churchill 2019](#), [Vesselov & Davis 2019](#), [Werle 2021](#)). Drawing on these complementary perspectives and the synthesis presented in Section [3.1.2](#), the following definition of a DS is adopted for the Vaisala context in this thesis:

A design system is a centralized collection of guidelines, rules, and reusable UI elements that are aligned with a brand, ensuring consistent user interfaces and user experience across applications.

A DS for Vaisala is envisioned as a strategic framework that fosters a unified approach to digital product development across all teams and services. Its purpose is to ensure visual

and experiential consistency by providing clear guidelines, principles, and reusable UI/UX components that support predictable and scalable development. Grounded in the Atomic Design methodology ([Frost 2013](#)), the system structures components into atoms, molecules, and organisms, reinforcing modularity and systematic reuse throughout the design and development process.

To further extend the atomic methodology at the organism level, the DS is supported by the proposed UX patterns artifact, which is expected to enrich this layer by promoting recurring interaction behaviors and offering reusable solutions to common UX challenges. Combined with supporting documentation and clearly defined criteria, this artifact helps reduce ambiguity, enables coherent decision-making, and ensures consistent implementation across products.

More than a static toolkit, the DS functions as a living, constantly evolving foundation that adapts to organizational goals and operational realities. Its governance model supports correct and compliant use of the system, enabling teams to work cohesively while maintaining alignment with shared design logic. By establishing a common vocabulary and shared practices, the DS strengthens cross-functional collaboration and builds a unified understanding across distributed teams. Ultimately, it aims to improve efficiency, enhance UX quality, reduce duplicated effort, and support continuous improvement throughout the product life cycle.

4.1.2 Proposed Definition of UX Patterns

Although UX patterns have gained attention, their definition and impact on UI/UX design remain somewhat ambiguous. The proposed UX pattern artifact is envisioned as reusable documentation that captures recurring UX problems and articulates solutions that enable desired interactions within a broader interaction flow ([Alexander et al. 1977](#), [Seffah 2015](#), [van Welie & van der Veer 2003](#), [Lamine & Cheng 2022](#)). Each pattern documents not only what the solution is but also why and when it is applicable by identifying the contextual conditions in which the problem occurs ([Seffah 2010](#), [Alexander et al. 1977](#)). To promote clarity and shared understanding across teams, the patterns are expressed through structured textual descriptions complemented by visual examples that illustrate how interactions unfold over time.

Emphasizing user paths and temporal progression ([Borchers 2000a](#), [van Welie & van der Veer 2003](#)), the artifact focuses on the touchpoints between the user and the system, representing patterns as timelines or flows that may span multiple screens and vary from short sequences to extended interaction journeys. Although implementation contexts differ, the interaction logic underpinning each pattern remains stable and is realized through UI components, such as atoms, molecules and organisms ([Frost 2016](#)), that connect the intended UX flow to concrete interface structures ([Lamine & Cheng 2022](#)). As a whole, the artifact supports scalable reuse, reduces ambiguity, and strengthens cross-functional alignment by providing a coherent means of reasoning about interaction challenges and their solutions.

The general definition of design patterns, and by extension UX patterns, used in this thesis has been synthesized from the existing literature discussed above. Accordingly, the definition of a UX pattern adopted in this thesis is as follows:

UX patterns are reusable, context-dependent solutions that address recurring UX problems by articulating concepts that enable desired interactions or experiences within an interaction flow. In addition to the solution, they describe the rationale for when and why it should be applied by explicitly characterizing the context in which the problem occurs through visually enhanced practical examples. They emphasize how user experiences unfold over time by focusing on interaction paths and touchpoints rather than isolated interface elements.

4.1.3 Method for Creating UX Pattern Identification Process

The analysis phase begins by identifying assumptions about existing UX patterns within Vaisala's portfolio of digital products. These assumptions are gathered through interviews and brainstorming sessions with designers. Additional evidence is collected by reviewing historical design artifacts, such as Figma projects, stakeholder maps, and concept illustrations. This process produces an initial list of candidate patterns that are hypothesized to recur across multiple services.

Using these assumptions, CJMs are created for selected products, encompassing both traditional web interfaces and embedded UIs on sensor devices. Each product is then also modeled using the top-down tree-structure approach proposed by [van Welie & van der Veer \(2003\)](#), which decomposes high-level user goals into experiences, tasks, and actions. Complementary methods by [van Welie \(2001\)](#), such as HTA and TWO, are applied to uncover micro-level interactions and contextual factors influencing pattern behavior.

These CJM, top-down, and task-based analyses are hypothesized to reveal how the assumed high-level patterns manifest as sub-patterns within individual products and enable cross-comparison across multiple products. By identifying recurring structures and flows, the study aims to formulate a systematic process for recognizing UX patterns and determining their generalizability, ideally without any cues of assumed patterns as input.

4.1.4 Method for Designing Documentation Framework

Documentation and application practices vary substantially across digital products, as discussed in Section 3.3.1. This thesis aims to address the need for scalable, reusable UX knowledge by creating a framework for the structured documentation of UX patterns, aware of their characteristics and the existing best practices in documentation.

The documentation framework design process includes benchmarking existing academic and industry practices to create a schema that captures the essential elements and dimensions for documentation. To identify articles for academic benchmarking of UX pattern documentation, a systematic literature review approach is applied. Recent studies from reputable journals and conferences are prioritized. However, some thesis work or other smaller individual research studies is also included.

The databases used in this study included SpringerLink, ScienceDirect, ACM Digital Library, IEEE Xplore, and some open platforms such as ResearchGate and

arXiv. The papers referred were often closely related to HCI, ID, software engineering, and design science. The keywords consisted of combinations of "interaction pattern", "patterns", "UX patterns", "design patterns", and "pattern language", combined with AND/OR logic. In addition, some studies are found from the references of other publications.

The selection of guidelines for UX pattern documentation was based on both empirical grounding and author credibility. Preference was given to sources supported by empirical research or widely recognized in the academic and professional design communities, ensuring that the documentation framework was both theoretically robust and practically applicable. Authors with multiple publications on UX patterns, such as Martijn van Welie, were prioritized due to their sustained contributions to the field. Additionally, works by forerunners like Tidwell (2010), Alexander et al. (1977) were included because of their foundational role in shaping design pattern theory and influencing contemporary documentation practices.

For visualizing UX patterns, industry best practices were adopted to ensure effective communication with the intended end-users of the documentation. Benchmarking was performed for the companies *Spectrum - Adobe's Design System* (n.d.), *Lightning Design System 2* (n.d.), *Carbon Design System* (n.d.), *Kela Design System* (n.d.), and *Ethos Design System* (n.d.) to gather a collection of elements for the documentation structure. These DSs are most often referenced by the designers when tackling difficult design tasks. Furthermore, these were the only ones found that included design patterns within their DSs.

The styles in these DSs were compared before making the documentation choices in this thesis. The primary aim was to provide the same information in visual and textual form to enable both quick scrolling and more detailed reading. Furthermore, features for skipping sections and locating meaningful points were considered highly important. By combining the insights from industry leading companies, it was deemed helpful to enrich the documentation with visual representations, including annotated UI designs (*Ethos Design System* n.d.), Figma layouts (*Carbon Design System* n.d.), and descriptions of the UX flow as diagrams (*Ethos Design System* n.d.) where applicable. Furthermore, copiable code snippets (*Kela Design System* n.d.) may be incorporated to support developers in implementing patterns consistently. This dual focus ensures that the framework serves both designers and developers, bridging the gap between conceptual design and practical implementation. To ensure ease of navigation, the vertical tabs from (*Lightning Design System 2* n.d.) and the table of contents menu, visible at all times, from (*Kela Design System* n.d.) were referenced. From these ideas the goal was to form and develop the most optimal suggestion of the design pattern documentation for Vaisala's context while considering the capabilities of the current DS.

Once all individual elements for both high-level patterns and their sub-types had been gathered and formed, they were combined and implemented as part of Vaisala's existing DS. The empirical research stage focused on validating the impact and goodness of the documentation by reviewing the opinions and feedback given by the interviewed designers and developers. The empirical methodology is described in Section 4.4.

4.2 Vaisala Research Context

The UX pattern identification process, one of the design artifacts of this thesis, is devised from analyses performed within Vaisala's context of digital products. Additionally, the study includes a practical illustration of how the proposed documentation design framework operates within the same empirical setting. The material collected for this study varied across products, as design problems had been approached differently. Different products included diverse types of design information at varying levels of detail, typically documented in tools such as Miro and Figma. Examples included early brainstorming notes, use case scenarios, and concept development artifacts in both textual and visual forms to facilitate stakeholder communication. However, in most cases, early design concepts had not been thoroughly documented; instead, the process had often commenced directly by creating visualizations such as Figma layouts and prototypes. This section first presents the digital products included in the scope of this study and then describes the selection of assumed UX patterns within these products.

4.2.1 Vaisala Digital Products

The digital products analyzed in this thesis encompass web applications such as Jade Smart Cloud, Insight, Avimet and Iris Focus, as well as embedded UIs in the physical products Indigo 80 and the Indigo 500 series. Their features and functionalities are summarized in Table 4.

Applications were selected based on the design team's prior beliefs on UX pattern existence, aiming to maximize the potential for their discovery. Diversity of the set of applications was a guiding factor in the selection, ensuring they represent heterogeneous user groups and tasks. Diversity of the set of applications used for the framework development process is important in enabling the generalizability of the resulting framework under realistic constraints (e.g., data operations, device, task and technical factors). Inclusion was limited to products where prior indications of relevant patterns were observed, so that candidate patterns could be verified rather than inferred from isolated instances. Preference was given to mature, widely used products with sustained development histories, enabling stable comparisons and facilitating both the discovery of similarities in functionality and the detection of meaningful differences. Collectively, the set supports pattern discovery and cross-application validation by exposing comparable problems solved through convergent design solutions.

Web User Interfaces

Jade Smart Cloud is a cloud-based monitoring and analytics platform that enables real-time and historical monitoring of environmental conditions, such as humidity and temperature, using Vaisala measurement devices. It is widely used in various environments, for instance in laboratories, within the food industry, in logistic centers and in museums.

Insight is a tool used for configuring and calibrating Vaisala sensors, such as Indigo series transmitters, in laboratory or field environments. It also enables access to diagnostics and analytics retrieved from sensors.

Table 4: Overview of Vaisala products.

Product Name	Specifications	Functionalities	Operating Environment
Jade Smart Cloud	Cloud-based monitoring platform	Real-time and historical data monitoring; humidity and temperature tracking	Laboratories, food industry, logistics centers, museums
Insight	Software for configuring and calibrating Vaisala sensors	Device setup, diagnostics, firmware updates	Lab and field environments
Avimet	Aviation weather observation system; integrates with runway sensors	Real-time weather data for aviation; visibility, wind, runway condition monitoring	Airports, air traffic control centers
Iris Focus	Software for LIDAR/radar systems	Visualization and predictive analysis of extreme weather conditions like storms	Meteorological institutes, airports, research facilities
Indigo 80, Illustration 5	Handheld indicator; supports smart probes; portable	Spot-checking, diagnostics, data logging, calibration	Field inspections, calibration labs
Indigo 500 Series, Illustration 6	Modular transmitters; supports continuous monitoring	Standalone or cloud-connected monitoring; plug-and-play support	Cleanrooms, manufacturing, environmental monitoring

Avimet is widely adopted in aviation to gather weather observations by combining for example visibility, wind, rain and thunder sensor data from runways. The Iris Focus system provides early prediction of extreme weather and storms, especially for meteorological institutes and airports to gain a comprehensive understanding of weather hazards. It relies on data from weather radars, wind lidars, and lightning sensors to gather information about weather conditions.

Embedded User Interfaces

Indigo 80 is a handheld user terminal device whereas the Indigo 500 series consists of modular transmitters for continuous monitoring. Indigo 80 can be used, for instance, in field inspections and calibration labs for portable diagnostics and spot-checking. Furthermore, it supports data logging and calibration from connected probes. Products from the 500 series can be used independently or integrated with Jade Smart Cloud for remote monitoring. They are especially suitable for cleanrooms, manufacturing and environmental monitoring.

4.2.2 Selecting Assumed UX Patterns

As stated in Section 4.1.3, this thesis utilizes a set of conceptual UX patterns that guided the exploratory analysis. The design team initially defined a collection of assumed UX patterns, which were further categorized by functionality and prioritized



Figure 5: Vaisala Indigo 80 handheld indicator ([Vaisala n.d.b](#)).

to select three of the most influential ones for detailed study. The selected patterns were:

- Unified account access flow (registration and login)
- Editing methods for selected elements or parameters in forms
- Card dashboard view customization

These UX patterns were chosen due to their diversity and prevalence across digital products. Diversity, distinctiveness from one another, and recurrence in several UX flows were used as selection criteria for the patterns to ensure that the developed framework could identify additional patterns in the future. These qualities were considered strong indicators for the existence of true underlying UX patterns.

An important question raised during the selection process considered the fundamental characteristics that qualify something as a UX pattern, essentially the features by which a pattern is recognized as such. This question influenced the inclusion of RQ 1.

4.3 Proposed UX Pattern Documentation Framework

The proposed UX pattern documentation framework is grounded in the objectives and goals derived from academic literature, as described in Section 3.3.1 and further



Figure 6: Vaisala Indigo 500 series transmitters (Vaisala n.d.a).

elaborated in Section 4.1.4. Section 4.1.4 presents benchmarking insights against established industry practices, informed the initial structure and ensured alignment with contemporary DS approaches. Together, these sources shaped a framework designed to support consistent, comprehensible, and reusable documentation across the organization's DS.

By providing predefined components that capture both conceptual logic and practical implementation, the framework aims to unify how patterns are described and interpreted. Each pattern, including its potential sub-patterns, incorporates the key dimensions identified in the literature: the problem it addresses, the context in which it applies, the forces influencing the design decision, the proposed solution, and illustrative examples. High-level patterns are intended to convey overarching concepts, whereas sub-patterns offer more detailed, implementation-oriented guidance.

To strengthen interpretability and reduce ambiguity, the framework integrates both textual and visual representations. These include annotated screenshots or Figma references, organism-level UI code snippet structures that clarify component relationships, and UX flow diagrams that capture dynamic behavior across multiple states. Collectively, these materials provide a multi-layered representation of each pattern, making the documentation accessible and informative for both designers and developers.

To further support navigation efficiency and ease of use, features such as a table of contents or a floating sidebar should be implemented to facilitate quick browsing

(*Kela Design System* n.d., *Ethos Design System* n.d.). Additionally, *Lightning Design System 2* (n.d.) decomposes complex patterns into sub-patterns to prevent cognitive overload, presenting a general overview before detailed sub-pattern contexts. Tabbed navigation can effectively support this structure. Clear heading levels (e.g., H1 for page titles and H2 for sections) should be applied to improve accessibility (*World Wide Web Consortium (W3C)* n.d.), and single-column layouts with appropriate text width contribute to better readability.

Above all, the documentation must adopt a consistent language and structural format aligned with the existing DS to ensure coherence and familiarity. Maintaining uniform terminology, tone, and presentation style enables contributors to work within a shared system of meaning. Finally, the framework incorporates a mechanism for iterative refinement, allowing continuous improvement as new patterns emerge or existing ones evolve.

By following the method presented in Section 4.1.4 a documentation proposal was structured as follows, serving as a starting point for answering RQ 3:

1. Introduction
2. Annotations
3. Presenting the variations, corner cases and associated UI organisms
4. Describing the UX flow between various user touchpoints

Before the introduction, an optional notification box may be included to indicate that the pattern is under development and to provide brief guidance on its appropriate use.

The introduction section aims to articulate what the pattern is about and set the UX pattern to its context. The five-dimensional table from Section 5.2.2 can be included as a table directly in the documentation or broken down into summarized text snippets for a structured narrative. This table can be useful for making the underlying problem–solution relationships explicit and ensuring consistent documentation of contextual factors.

The annotation section as illustrated in Figure 7 can include screenshots of the central UI organism or Figma component representations to illustrate the structural relationships among components, design decisions, and micro-interactions embedded within each pattern. All the components in the UI element must be annotated with their names, reducing ambiguity and promoting a shared understanding of the pattern’s structure and behavior.

Representing patterns as UI organisms as demonstrated in Figure 8 emerged as a critical part of the documentation process. These organism-level representations move the focus beyond isolated components in the DS by situating them within the broader layout demonstration of the pattern. Instructions are included detailing the suitable use of each organism-level variation of the pattern, allowing the user of the documentation to decide definitively whether the pattern is applicable to a use-case at hand. This step clarifies how components are arranged within the element layout and how each pattern applies across different screen states and corner cases.

Login annotation



1. Background (only applicable on desktop)
2. Application title (includes the name of the product)
3. Login forms body (one example)
4. Input field (one example)
5. Button to log in
6. Button for "Forgot your password?"
7. Footer

Figure 7: Annotation section of the documentation for the "Login" pattern.

Finally, constructing UX flows enables the documentation of the patterns' dynamic behavior, an aspect that static images alone cannot capture. In practice, this is achieved by including timelines of the patterns' touchpoints, the UI organisms corresponding to the touchpoints, and a video summarizing the entire user journey. Figure 9 represents an overview of the two user journeys in Login, and Figure 10 represents an example of the touchpoints of the password reset flow.

The documentation proposal is studied to assess how effectively the documentation template conveys essential information in a coherent structure and to examine which aspects of the documentation the participants most rely on when encountering it for

Login forms

Login forms allow the user to enter the credentials, such as email addresses or username, and password. It should provide clear labeling on top of the input field, possibly a placeholder or help text or for guidance, and real-time validation to ensure accuracy. In addition to the input fields, "Forgot your password" -option is displayed for the user to ensure easy access.

Login forms: style 1

The image shows a login form UI mockup. It consists of a white rectangular container with a thin grey border. Inside the container, the word "Email" is written in bold at the top left. Below it is a text input field with the placeholder text "Enter your email". Below the email field, the word "Password" is written in bold. Below it is a text input field with the placeholder text "Enter new password" and a small eye icon on the right side. Below the password field is a wide button with the text "Log in". At the bottom left of the container is a link that says "Forgot your password?". Below the container, there is a row of controls: the text "VIEW CODE", the text "Desktop" followed by a toggle switch that is currently off, the text "Mobile" followed by a small icon of a mobile device.

Figure 8: An example of a UI organism.

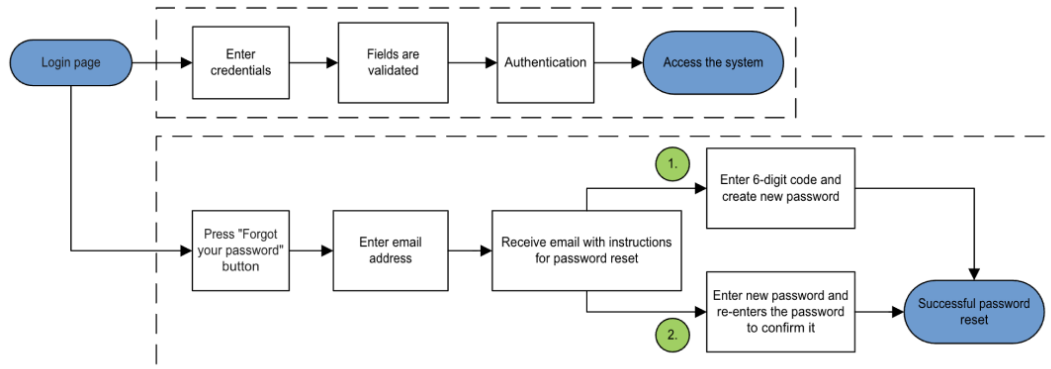
the first time. The impact and effectiveness of the proposed structure for UX pattern documentation is further validated in the empirical research.

4.4 Empirical Study for Application and Evaluation

For the demonstration phase of the DSR approach, the proposed documentation framework is applied to the selected UX patterns used within Vaisala's digital products. In the subsequent evaluation phase, the framework is examined through an empirical study with designers and developers. The purpose of this evaluation is to assess the clarity, completeness, and applicability of the proposed structure of the documentation for UX patterns, and to determine how effectively the documentation supports real-world design and development work.

To address these goals, the study adopts a qualitative research approach by including a short task-based walkthrough as part of a semi-structured interview. This approach enables an exploration of the participants' experiences, perceptions, and challenges when using the documentation, and it aligns with RQ 4, which is concerned with the

Login flow



Login flow can be divided into two phases:

1) Regular login to the system with existing credentials that are verified before the access is granted

- The relevant UX touchpoint is the login form that was introduced in **Section Components: Login forms**.
- There are two styles, and data center can be added [now left out].

2) Password reset flow to reset/update credentials:

- One flow that requires **6-digit reset code**.
- Second flow **without the reset code**.

Figure 9: An example of the "Login" flow graph.

preferences of the participants. Qualitative inquiry is particularly suited for uncovering nuanced insights regarding documentation understandability, decision-making, and practical applicability in multidisciplinary teams.

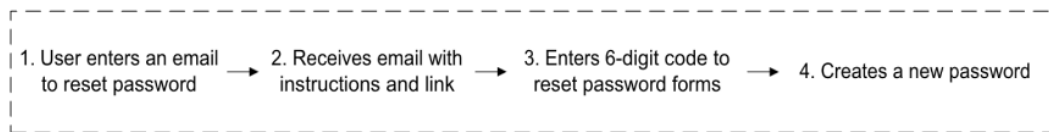
Due to time constraints, the empirical evaluation was conducted using an interactive documentation prototype implemented in Figma, representing a proposed addition to the existing DS. This prototype served as a realistic approximation of the intended final implementation and allowed participants to engage with the documentation in a way that mimicked real-world usage.

4.4.1 Participants

Participants were selected using purposeful sampling, focusing on individuals with substantial experience in design, development, or DS work. Inclusion criteria required participants to have demonstrated competence in their respective fields, familiarity with the existing DS documentation, and a potential need for UX pattern documentation in their daily work.

To identify suitable participants, the most active ongoing projects within the organization were examined, and their most engaged contributors were invited to

Password reset: 6-digit reset code



1. User begins the flow by clicking **"Forgot your password?"** button in the login screen. This new view asks the user to provide their email to which the reset email is sent.
2. The email contains only **a 6-digit code** that must be entered to the password reset forms. As the email does not contain link to new forms, **the original website must be kept open and not refreshed.**
3. and 4. Once correct 6-digit code is entered, the user can provide a new password. By sending the forms, password is updated.

Password reset main touchpoints

1. Requesting the password reset

VAISALA
Name of the product

Forgot your password?

Enter the email address where the password reset code will be sent.

Email

Send code

[← Back to login](#)

[Terms of use](#) | [Privacy policy](#)
© Vaisala - All rights reserved

2. Email with reset code

Vaisala [Name of the product] <[product name]-noreply@vaisala.com>
To: firstname.lastname@vaisala.com

12356 is your Vaisala [Name of the product] password reset code.

Figure 10: An example of the touchpoint representation through a graph and UI organisms.

participate. This ensured that participants had recent, hands-on experience with the tools, workflows, and challenges associated with DS usage and documentation. Additionally, because embedded UI work was included in the scope of the new documentation, the most active contributor from this area was also recruited. All of the selected participants were highly involved in project-level design decisions, making them well positioned to evaluate the clarity, usefulness, and applicability of UX patterns.

As UX patterns are closely tied to the organization’s broader DS methodology, it was essential to include key DS developers as participants as well. Their involvement provided a strategic, long-term perspective on the manner in which the documentation should be crafted, updated, and maintained, and ensured alignment with existing DS principles and practices.

Nine participants were recruited for the study, representing three distinct role categories: designers, developers, and DS developers. Three participants were included in each category to ensure balanced representation across these perspectives. Due to the scope of the thesis, time constraints, and the limited availability of suitable participants, the study focused exclusively on individuals who were already actively contributing to design-system-related work. This approach ensured that the insights gathered were both relevant and grounded in current practice, while also reflecting the practical constraints of conducting empirical research within a restricted time-frame.

All participants took part voluntarily and provided informed consent (GDPR) prior to participating in the interviews. No sensitive personal data was collected, and all responses were anonymized to protect participant confidentiality. The characteristics of the participants are summarized in Table 5.

Table 5: Interview participants by job title and role category.

ID	Job title	Role category
P1	Software Engineer	DS developer
P2	Software Engineer	Developer
P3	Senior UX Designer	Designer
P4	Full-Stack Developer	Developer
P5	Senior Software Engineer	DS developer
P6	Senior Frontend Developer	Developer
P7	Senior UX Designer	Designer
P8	Principal UX Designer	DS developer
P9	UX Designer	Designer

4.4.2 Interview Procedure

Semi-structured interviews were conducted both via Microsoft Teams and face-to-face depending on participant availability. Each session was recorded and automatically transcribed by an AI tool. Interviews lasted approximately 50 minutes and followed

a four-part structure consisting of: 1) introduction, 2) warm-up questions, 3) main questions and prototype-based tasks, and 4) a wrap-up phase. The interview questions are outlined in Appendix A.

The warm-up questions focused on the participants' day-to-day responsibilities and their prior experience with DSs. The main section included questions about their familiarity with design patterns as well as an evaluation of the developed UX pattern documentation artifact. This evaluation was supported by two short real-world use-case scenarios designed to probe the usefulness, clarity, and potential shortcomings of the proposed UX pattern documentation. However, if the interviewee was naturally forthcoming and discussed these aspects without prompting, the use-case scenarios were omitted, as the relevant insights often emerged organically during the conversation.

While the question set remained largely identical across interviews to ensure consistency, role-specific adaptations were included. For example, in interviews conducted with DS developers, questions related to the use of documentation were replaced with items targeting documentation maintenance, update workflows, and the development side of the DS.

During the final wrap-up phase, participants were invited to raise additional thoughts or topics they felt were relevant but had not been addressed.

4.5 Thematic Analysis

Thematic Analysis (TA) served as the analytic approach for extracting and classifying the insights that arose from the interview data. As noted by [Ahmed et al. \(2025\)](#), TA offers a structured yet flexible approach that supports the systematic exploration of qualitative material and the identification of patterned meanings that can be organized into overarching themes.

The analysis was structured around the six-phase framework ("familiarization", "initial coding", "theme search", "theme review", "definition/naming", and "reporting") by [Braun & Clarke \(2006\)](#). To suit the scope of the project, the coding structure was consolidated into two hierarchical levels: parent codes denoting overarching themes and child codes capturing specific manifestations, influences, or contextual factors. This two-level scheme was found sufficient given the available timeframe and resources, while still making the relationships between parent and child codes explicit.

The analysis approach was primarily inductive and data-driven, as the coding was used to surface patterns that could inform and validate the research questions. In particular, the analysis sought to uncover themes related to the benefits, needs, and impacts of UX pattern documentation.

4.5.1 Coding Focus and Analytical Decisions

Coding focused on capturing interview segments related to participants' prior experiences and use of the current DS, their perceptions and decision-making processes regarding the terminology of design patterns, as well as their wishes and needs for UX pattern documentation. In addition, comments reflecting participants' views on

the proposed UX pattern documentation and suggestions for potential next steps were included in the coding.

The coding remained primarily semantic, emphasizing the explicit meanings expressed by participants, while also allowing space for latent interpretations when underlying assumptions or recurring rationales appeared across the dataset. This approach ensured that the analysis remained closely aligned with the research questions while still allowing unexpected insights to emerge from the interviews.

4.5.2 Phase 1: Familiarization With the Data

The interview transcripts were generated by an AI assistant that participated in each session. These transcripts were then exported to ATLAS.ti for further examination and manual coding. Before any coding was initiated, all transcripts were read through in full to gain a comprehensive understanding of the dataset and to begin noting preliminary observations. Because the automated transcription occasionally produced errors, unclear segments, or incomplete sentences, these instances were cross-checked against the original video recordings. Missing information was added where possible, and mistranscribed portions were corrected to ensure accuracy before coding began. This familiarization phase also involved noting early impressions, recurring ideas, and potential areas of interest, which later helped guide the initial coding process while maintaining a close and reliable connection to the raw data.

4.5.3 Phase 2: Generating Initial Codes

Initial codes were generated using a line-by-line approach to ensure a consistent level of granularity and to enable the identification of similarities and differences across participants' responses. Because the semi-structured interviews followed a partially shared structure, this method supported systematic comparison while still allowing space for unique insights to surface. Throughout the coding process, codes were iteratively refined, merged, and split where necessary as new relationships between segments became apparent. Each code was reviewed for clarity and relevance, and descriptive labels were assigned to more accurately reflect the underlying meaning of the coded content.

As part of this iterative refinement, earlier codes were revisited to maintain coherence across the dataset and to avoid overlap between concepts. Memos were created during coding to document emerging interpretations, questions, or connections between codes, supporting transparency in analytic decision-making. This process ensured that the coding system evolved alongside the growing familiarity with the data and helped establish a solid foundation for developing preliminary themes in the subsequent analysis phase.

4.5.4 Phases 3–5: Developing, Reviewing, and Defining Themes

Codes were initially grouped according to participants' roles and responsibilities, their use or development of the DS, their wishes and needs for UX pattern documentation,

and their reflections on the documentation proposal. Within these broader categories, insights were further organized into evaluative dimensions, such as positive experiences, negative experiences, suggested improvements, and ideas for future development, to distinguish what participants considered immediately relevant from what they viewed as beneficial in the longer term.

Themes were developed by identifying conceptual connections across these groups of codes. A theme was defined as a shared, higher-level idea or abstraction that captured an underlying pattern with meaningful implications for UX pattern documentation or DS practice. As part of consolidation, themes with small coverage (1–3 codes) were merged or reassigned to thematically adjacent parents, and the previous parent labels were discarded. This decision supported the grouping confirmed in the third review cycle: at first, all segments were coded and provisionally matched to a suitable group where possible. Connections were then established and further reviewed in subsequent rounds.

Emergent patterns were validated across participants and stakeholder groups. Although participants tended to focus on aspects most directly linked to their own work, needs, and responsibilities, many also articulated perspectives on adjacent teams or roles. This cross-role overlap strengthened the robustness of the themes, ensuring they reflected shared organizational experiences rather than isolated individual viewpoints.

5 Results

Building on the research approach and methods outlined in Section 4, this section reports the results produced by the UX pattern identification and documentation work. Section 4 describes how the study adopts a DSR approach in Vaisala's digital service context, details the methods used in the analysis phase, as well as describes the design of the documentation framework and its empirical evaluation using thematic analysis.

Section 5 now follows this structure in presenting the results. Section 5.1 summarizes the characteristics that qualify a UX pattern, distilled from the analytical work. Section 5.2 presents the resulting UX pattern identification framework that combines the analytical methods into a repeatable two-phase process. Section 5.3 then reports the findings from the demonstration and evaluation phases, describing how the initial documentation framework was perceived by designers, developers, and DS developers, and how their feedback informs a refined proposal.

5.1 Characteristics of a UX pattern

This section addresses RQ 1, which concerns the qualities and specific characteristics required for a UX pattern to exist. The findings from the top-down analyses indicate that the existence of a UX pattern can be identified through two indicators: vertical height and recurrence across applications.

Within a single application, a candidate pattern appears as a flow that spans multiple levels of the top-down hierarchy, extending from high-level goals to tasks and concrete user actions. A flow with such height and a broad span at the task level suggests that the underlying solution has a central role within the UX.

This structural prominence aligns with the literature-based definition of UX patterns used in this thesis, which emphasizes reusable, context-dependent solutions that express the unfolding of interactions over time within an interaction flow (Section 4.1.2). According to that definition, a UX pattern is not determined by visual elements alone, but by the interaction logic that connects user goals to interface behaviors. A vertically tall flow mirrors this definition by demonstrating how a solution extends from the intended experience to actionable UI-level steps.

The second indicator is recurrence across different applications. In cases where the top-down analyses of multiple products contain flows with similar height across hierarchical levels and similar internal structure, recurrence suggests that the same underlying interaction logic is being reused. Such repetition indicates that the flow represents a generalizable problem-solution structure, consistent with theoretical definitions of UX patterns as solutions addressing recurring UX problems within varying contexts.

Together, these findings indicate that a UX pattern identified through top-down analysis can be expressed as:

Within several top-down analyses, a UX pattern is defined as the largest common connected subset of top-down analysis elements shared between applications.

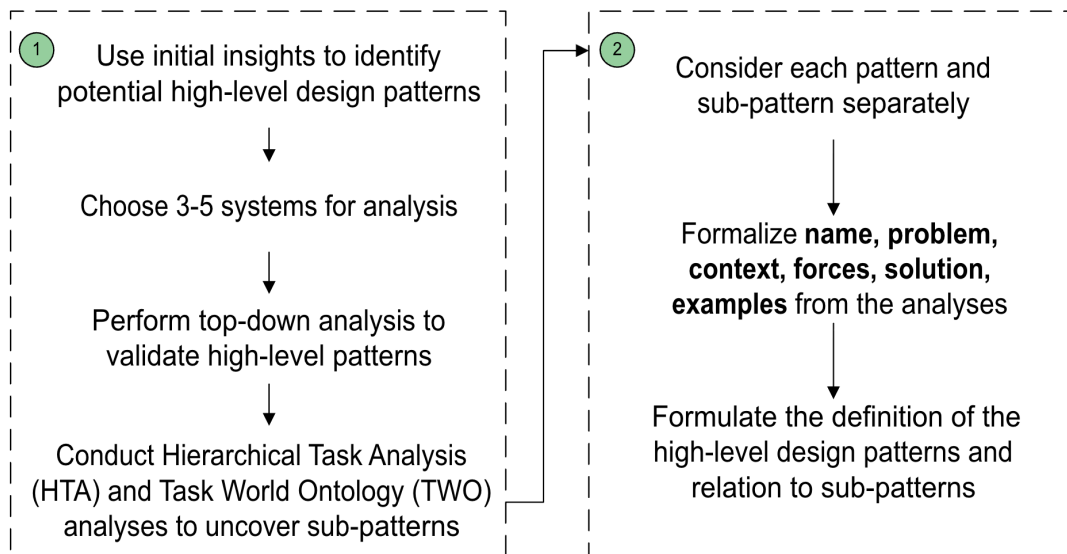


Figure 11: Framework for UX pattern discovery and documentation.

This analytical definition aligns with the literature-based definition in Section 4.1.2: both emphasize that UX patterns represent interaction flows rather than isolated interface elements, and that a valid pattern must appear across multiple contexts while having similar structure across the hierarchical levels.

5.2 UX Pattern Identification Framework

From the perspective of RQ 2, in which the abstraction and documentation of UX patterns across diverse software contexts are examined to support reuse, the combination of top-down analysis, HTA, and TWO is central. This section describes the manner in which these analyses are used for identifying UX patterns, while the proposed pattern-documentation structure is presented in Section 4.3. Each analysis method contributes a distinct type of information that is necessary both for discovering patterns and for completing the documentation structure in a consistent manner.

The framework illustrated in Figure 11 outlines a two-phase process for discovering, abstracting, and documenting UX patterns across varied software contexts. The framework consists of 1) an identification phase in Section 5.2.1, in which candidate patterns and sub-patterns are discovered through systematic analysis, and 2) an abstraction and formalization phase in Section 5.2.2, in which the resulting findings are distilled into reusable pattern descriptions.

5.2.1 Phase 1: Pattern Identification Across Systems

The identification phase starts from initial insights derived from earlier research, domain expertise, or observed user needs. These insights are used to propose candidate high-level UX patterns that are then validated and refined through analysis of a set of digital systems. The purpose of this phase is to move from vague, experience-based

hunches toward empirically grounded patterns that recur across diverse contexts.

In this study, CJMs were initially used to gain an overview of phases and touchpoints, but they proved insufficient for capturing the granularity and variability needed for robust pattern discovery. While CJMs offered high-level narratives, they did not adequately represent concurrent paths, detailed user interactions, use-case-specific constraints, or micro-interactions that differentiate one pattern subtype from another. For example, in the "Account Access" case in cross-application workflows (Figure 12), CJMs did not reveal the ways in which tasks decompose and recompose as users transition between physical interfaces and digital screens. In addition, it could not capture the small differences regarding the task execution, or the ways in which constraints such as screen size, input modality, and latency alter action sequencing and error recovery. These limitations motivated the adoption of task-based methods.

Across cases, this combined approach yielded actionable design knowledge:

1. Top-down diagrams identified *where* patterns and sub-patterns could be found within complex activities,
2. HTA specified *what* users do and *how* interaction sequences differ across pattern variants, and
3. TWO explained *why* those sequences and variants emerge given the surrounding task world.

Utilizing these complementary insights, the UX pattern identification framework supports abstraction and documentation of UX patterns that are empirically grounded, theoretically justified, and comparable across diverse software contexts.

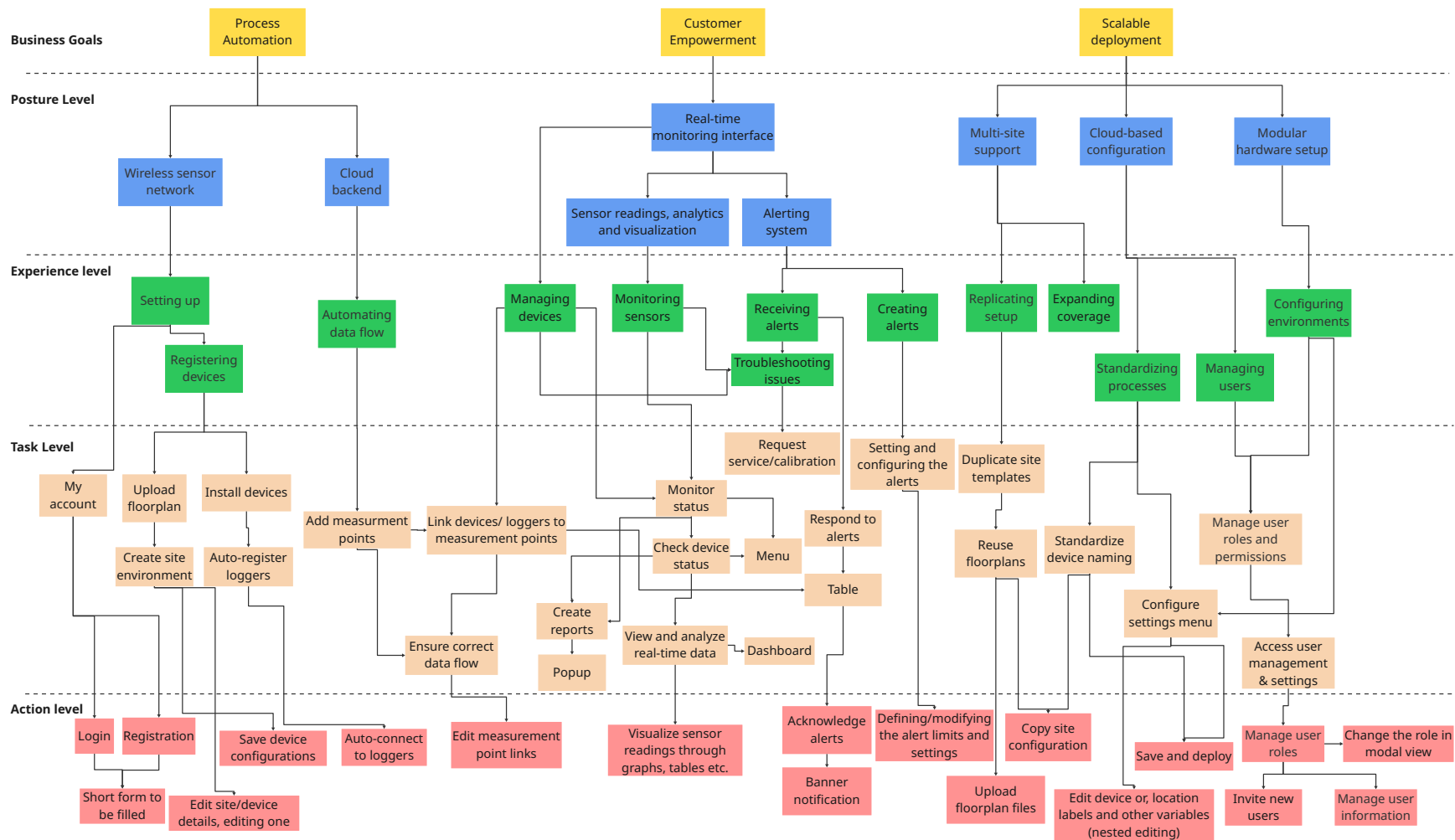


Figure 12: Vaisala Jade Smart Cloud top-down analysis, applied from (van Welie & van der Veer 2003).

Top-down Tree Analysis

Top-down analysis provides high-level insight into the activity space in which potential patterns emerge, highlighting their frequency, importance, and surrounding conditions (van Welie & van der Veer 2003). In this study, the analysis informed the selection of touchpoints and user journeys for closer examination by revealing where critical path decisions occurred and which regions of the activity space were most likely to contain reusable patterns.

This approach exposed structural details that traditional customer-journey maps tend to obscure, such as the prerequisites for a feature to occur, its dependencies on surrounding elements, and its effects on subsequent decisions and actions. These flows were then compared across services to evaluate their generalizability and determine whether they represented reusable patterns addressing shared design or interaction challenges.

For UX pattern discovery, top-down diagrams essentially indicate the areas in which the attention should be directed. They highlight the activity segments in which patterns consistently emerge, the contextual factors on which those patterns depend, and the points at which critical decisions or transitions occur. In Figure 12, for example, the top-down analysis highlights the "Account Access" pattern, showing that it spans from the experience-level stage of "Setting up" through to the task-level stages of "Login" and "Registration". By making these continuities visible, the diagrams help clarify the structural reach of such patterns, supporting their identification and subsequent abstraction into reusable forms.

Hierarchical Task Analysis

HTA provides the detailed descriptions of user actions and decision points needed to distinguish sub-patterns and specify their boundaries. By decomposing activities such as "Account Access" into alternative flows, first-time registration, returning login, and credential recovery, it clarified which steps are essential or optional, and where meaningful variation emerges.

After top-down analysis identified candidate regions, HTA was used to break activities into tasks, sub-tasks, and concrete actions. This micro-level representation exposed the alternative paths users may take and highlighted the points at which differences generate distinct subtypes within a pattern family. These insights directly informed the *Problem*, *Solution*, and *Examples* fields in Section 5.2.2.

To examine cross-application consistency for identical user goals, a unified HTA was constructed for each UX pattern subset (e.g., Account Access, inline editing with explicit save, inline editing with implicit save). HTA's goal-subgoal structure enabled a system-agnostic representation in which core steps, entry conditions, decision points, and error-recovery loops could be compared systematically. Application-specific differences that did not alter the underlying task topology were treated as variants and documented under the relevant sub-goals to maintain traceability.

Although the patterns appeared across multiple systems, available documentation was uneven. Most differences were minor, while the primary task structures remained

consistent. In cases where not all applications provided sufficiently detailed evidence, the combined HTA was based on those systems that documented the required touch-points clearly. Under these conditions of shared goals, limited variation, and incomplete evidence, a unified HTA provided the most coherent and reliable representation.

Task-World Ontology

TWO contributed the task-world rationale that links system constraints to user goals, satisfaction, and broader UX outcomes. By modeling actors, artifacts, tasks, and constraints, it informed the *Context* and *Constraints / Forces* fields in Section 5.2.2 and complemented HTA by clarifying the interaction structures that arise within particular environments.

A single consolidated TWO (Figure 4) was constructed to analyze authentication across all included applications. This ontology captures domain-level invariants, agents, roles, tasks, resources, and their relations, rather than application-specific interface details. Stable relations (e.g., uses, triggers, achieves, performed_by) and core attributes (e.g., start_condition, stop_condition, login_type, interruptable) were formalized to highlight structural commonalities.

The shared TWO serves as a semantic backbone for the composite HTA (Figure 3). The TWO currently combines insights from two HTA diagrams: the admin-led HTA representation concerning the user invite, and the user-initiated completion of the registration process. Each HTA sub-goal is anchored to an ontological entity, ensuring consistent entry conditions, exit criteria, and role constraints across products. This structure also prevents duplication: tasks such as "Login" or entities such as the "Authentication provider" appear once in the ontology but are referenced by multiple HTA branches. In cases where product documentation was uneven, the ontology enabled a coherent, unified representation by aligning HTA fragments to a single semantic frame.

For UX-pattern discovery, TWO clarifies the relationship between interaction sequences and their underlying constraints and intentions. This linkage enables recurrent patterns and context-dependent variants to be identified. As illustrated in Figure 4, nodes such as "Login" task, "Registration" task, "Email invite", "User management system", and "Authentication provider" encode preconditions, postconditions, triggers, and resource dependencies that correspond directly to task-model elements.

The "Registration" task carries attributes describing its initiation and termination conditions, the responsible agent, and its interruptibility, which are inherited by the corresponding HTA sub-goal. Likewise, the "Email invite" provides the semantic connection between administrative action and the initiation of registration, which is reflected in the task decomposition. Through these correspondences, recurrent sequences and context-sensitive variations are systematically identified within a unified semantic structure.

5.2.2 Phase 2: Pattern Abstraction and Formalization

Taken together, the top-down, HTA and TWO analyses provide a multi-resolution, evidence-based account of UX patterns. The abstraction and formalization phase translates these graph-form findings into a consistent textual format. The goal of this phase is to create pattern descriptions that can be compared across a wide range of software contexts and applied in new design situations. While creating these descriptions, the designer is forced to evaluate the validity of the patterns and prepares the information required for its documentation. To execute this phase, each UX pattern is formalized using the following structure (synthesized from the insights in Section 3.3.1), which is directly informed by the analyses:

- **Name:** A concise but clear and descriptive label that combines the context and purpose of the pattern to make it easily identifiable and memorable (e.g. "Account Access" or "Inline Editing with Auto-save") (Borchers 2000b). Insights from the top-down analysis help determine which goals and contexts should be reflected in the name.
- **Problem:** A definition of the challenge or issue the pattern addresses, clarifying why it needs to exist. An example of the this field for a nested editing sub-pattern might be: "Complex edits require full screen and context as well as full focus, mandating a new nested window." The information required for this field can often be synthesized from the HTA and TWO analyses.
- **Context:** A description of the relevant situation (actors, objects, tasks, and environment) in which the pattern applies, including preconditions and typical usage scenarios (van Welie & Trøttemberg 2000). This field is primarily informed by the top-down analysis (overall activity and environment) and TWO (actors, artifacts, and resources).
- **Constraints / Forces:** Key trade-offs, limitations, or influencing factors (such as technical limits, security requirements, business rules, or user expectations) that shape how the solution can be instantiated (Borchers 2000b). TWO is central here, as it explicitly models constraints and resource dependencies.
- **Solution:** A recommended, proven design approach that captures the essential interaction structure while remaining flexible enough to support adaptation across platforms and applications (Borchers 2000b). HTA provides the detailed task flows that underpin the solution, while top-down analysis ensures that it remains aligned with overall goals and experiences.
- **Examples:** Real-world or mock examples of the pattern in use, represented through organisms and UX flows. These examples illustrate how the abstract solution can be realized in different interfaces and contexts, and they are derived from the concrete cases analyzed with HTA and TWO.

In addition to completing the information in the fields, definitions of main patterns are synthesized based on the corresponding sub-patterns in this phase.

5.3 Validating Initial Documentation Framework Proposal

To verify whether the proposed documentation framework is grounded in real user needs, an empirical study was conducted, as described in Section 4.4. The study examined how designers, developers, and DS developers engage with design pattern documentation, the challenges they encounter, and the types of content and presentation formats they perceive as most valuable. The findings provided a foundational understanding of the needs, expectations, and goals of the documentation's primary users, answering RQ 4. In addition, interviews with DS developers offered deeper insights into the development and maintenance processes associated with DS documentation.

5.3.1 Familiarity with Design Pattern Terminology

Participants were asked about their familiarity with design pattern terminology, including distinctions made between UI and UX patterns. Their experiences with, and preferences regarding, this division were also explored.

Designers

Overall, the designers demonstrated the strongest understanding of the terminology and of the differences perceived between UI and UX patterns. Participants 7 and 9 expressed a preference for using the more specific terms "UI patterns" and "UX patterns" rather than more general labels such as "patterns" or "design patterns." The need for clearly defined terminology was emphasized so that consistent usage could be supported. However, they also noted that other stakeholders may easily become confused by these distinctions, which could introduce risks of miscommunication. For this reason, the use of the broader term "design patterns" was regarded as a safer and more widely comprehensible alternative in certain contexts.

Developers

Among developers, understanding of design pattern terminology was more varied. P2 associated design patterns with a process, P4 primarily with cohesive visual layouts within UI components, and P6 with the manner in which interfaces should function and appear, essentially a pattern to be followed. Due to limited familiarity with the underlying meaning of the term "design patterns", P2 was not able to distinguish between UI and UX patterns. P4 possessed a partial, high-level sense of their difference, whereas P6 understood UX mainly as the way users interact with the application.

In relation to communication between stakeholders, P4 indicated that the use of more detailed terminology could create challenges, since individuals frequently rely on basic, informal labels when referencing interface elements. Some situations (not specified in detail) were acknowledged as cases where the distinction might nevertheless hold relevance. P6 did not consider the division between UI and UX patterns to be a major concern.

Design System Developers

DS developers demonstrated a more foundational understanding of design patterns and their categorization. P1 approached the concept from a visual perspective, understanding design patterns as descriptions of how components operate or as indications of acceptable ways in which they may be combined. P5 characterized design patterns as recurring formulas within user flows that can be captured and documented. P8 adopted a more UX-oriented perspective, viewing design patterns as entities more complex than individual components, often involving interaction and therefore considered particularly important to document.

Regarding the distinction between UI and UX patterns, uncertainty was expressed by P1 about a clear definition of UX patterns, as the term had not been frequently encountered. Nevertheless, UX patterns were intuitively interpreted as more abstract models that incorporate considerations such as accessibility, rather than solely concrete examples of UI components. P5 suggested that UX patterns might also encompass user personas and decision-making aspects. In communication with other stakeholders, P1 noted that dividing patterns into UI and UX categories might not be necessary and could even complicate understanding, since the UI is inherently a part of the broader UX. Similarly, P5 preferred referring directly to the relevant context (e.g., alarms, login) rather than explicitly labeling a pattern as either UI or UX. P8 expressed uncertainty about how terminology would evolve over time, noting that practical use of the patterns would ultimately shape how such terms are adopted in communication.

5.3.2 Thoughts on the Main User Group

In the practical examples and elements presented in the DSs of *Ethos Design System* (n.d.), *Kela Design System* (n.d.), and *Lightning Design System 2* (n.d.), designers and developers emerge as the primary user groups for DS documentation. In the empirical study conducted for this thesis, participants were therefore asked to reflect on which stakeholders might review and potentially benefit from the proposed documentation.

Visual examples, interactive prototypes, and curated collections of available UI organism views or components are generally preferred by designers, as these formats align closely with their daily workflows. Developers, by contrast, rely predominantly on existing component documentation, Storybook entries, design tokens, and other technical libraries. Since their collaboration with designers is already extensive, it may be more challenging for UX pattern documentation to be incorporated naturally into developers' everyday practices.

Participants also indicated that additional roles, such as POs, accessibility or documentation specialists, software testers, and other professionals, may interact with the documentation. For these stakeholders, UX pattern documentation could serve to support the maintenance of a cohesive UX and visual consistency and could assist in identifying potential new features or extensions by enabling the benchmarking of existing pattern examples.

5.3.3 General Feedback on the Documentation Proposal

A prototype of the account access UX pattern, covering both login and registration, was created in Figma. The structure was based on the initial proposed documentation model in 4.3. Participants evaluated the prototype to assess the extent to which the framework supported their prior expectations, understanding, and practical workflows. This validation phase centered on the clarity, usefulness, and appropriateness of the structure, as well as its alignment with real-world tasks and decision-making processes.

Initial Expression

All participants were initially surprised by the amount of text displayed and by the overall density of information on a single page. Their attention was drawn strongly to the notification message indicating that the UX pattern was still under development, and this element was perceived as overly prominent. Six participants found it difficult to grasp the underlying purpose of the page, as questions emerged in their minds about the reason for being on the page, the content that should be read, and the purpose the material served. Understanding improved gradually as they proceeded through the page and navigated to other tabs. The prototype was ultimately interpreted as a form of documentation, description, or informational resource.

Structure of the Documentation

Across all participants, strong emphasis was placed on the need for visual elements, either through UI examples or through UX flow representations. It was widely agreed that each page should begin with a visual reference rather than with introductory text. Five participants suggested positioning an annotated screenshot or a UX flow diagram at the top of the page so that readers could quickly confirm that they had reached the correct pattern. As P7 stated, "First, I would like to see a picture so that I know whether I am in the right place or whether I am looking for this particular pattern." P1 explained, "It should quickly become clear what we are looking at here. Then you know that these are samples from 'Login', and these are its components." P8 echoed this need for instant clarity: "If I look at this for a second, I should quickly understand what it's about."

Beyond the initial overview, the proposed structure of the remaining content was generally seen as clear and logical. Participants appreciated that individual UI organisms were introduced first, followed by more detailed descriptions of UX flows that address the main tasks and touchpoints within the UI during interaction. Nevertheless, concerns were raised regarding the volume of information and number of examples presented on a single page. P2 suggested that, in some cases, the material could be distributed across multiple pages to ensure greater manageability and ease of navigation. P7 specifically noted that error cases might be more effectively presented on separate pages, with links provided in the main documentation to direct users to the corresponding content.

Introduction

Overall, the introduction was perceived as slightly too long and text-heavy. The notification message box drew more attention than intended; all participants felt it could remain, but in a more concise and less visually dominant form.

P5 highlighted the five-dimensional table outlining the problem, context, forces, solution, and examples, noting that it provided valuable insight into the rationale behind the pattern and helped distinguish among patterns, particularly in situations where more time is available for careful reading. However, both P3 and P5 observed that the "Forces" dimension was less intuitive and might require clearer definitions or illustrative examples. Because most participants preferred to view visual examples at the beginning, the introduction did not receive extensive feedback, as it was not read in full during the validation session.

Annotations

Across all validation sessions, participants emphasized the importance of combining textual and visual content. Visual examples, especially annotated interface views, were seen as significantly accelerating comprehension. Eight participants appreciated the annotations and the way they clarified the individual elements, the relationships among them, and their placement within the UI. P9 underscored the need for clear and well-defined names for each element so that communication and shared understanding are facilitated. P8 responded positively to the visual overview, noting, "This is nice, since it's a visual overview. From this you can probably get off to a very good start." P6 suggested that responsiveness could also be represented within the annotated material.

The annotated examples were generally considered helpful, as they reflected current best practices and decisions made in ongoing projects. The quantity of examples and variations was appreciated, and P1 emphasized that these concrete examples were among the most valuable aspects of the documentation.

User Interface Organisms

Within the UI organism views, participants stressed that visual material should be well-curated and consistent across patterns so that readers are not overwhelmed. This reinforced the need for a balanced integration of modalities and a consistent proportion between visual content and accompanying text. At the same time, several participants requested more targeted and specific guidance regarding configuration options and the ways in which patterns can be adapted to different contexts. Clear dos and don'ts were also desired so that design alternatives could be compared more directly. P5 expressed concern that examples might become lost among other information and suggested that they should be visually elevated within the layout.

More detailed information about configuring the available elements was requested by P1, especially in situations where multiple options exist (e.g., alternative password reset methods). Greater clarity was desired regarding the selection process: "Which one is recommended, which one is better designed, is one of them already in use

somewhere, where do I make the decision, and how does the product make that decision?" Similar concerns were raised by P2, who asked for concrete implementation guidance: "What are the concrete steps needed to implement this, and what are the alternative steps, if there are any?" P3 further stressed the need for the documentation to explicitly articulate and explain differences between solutions, noting that current variability across applications reflects a lack of maturity and unification in the system.

The prototype provided options for viewing and hiding code, switching between mobile and desktop versions, and copying code snippets. Despite other concerns, participants responded positively to the idea of presenting both desktop and mobile variants of UI organisms, along with copy-to-clipboard functionality for each. As P4 noted, "Having both desktop and mobile, so you could look at their different versions. We don't always necessarily have this, what it looks like on mobile. This could be really good."

Addressing the key corner cases of UI elements, such as form validation errors, other error states, and password reset flows, was also well received. Participants emphasized the importance of including such cases, as they are often overlooked during the design process; documenting them was seen as a useful reminder to ensure that they are explicitly considered.

Naming Convention for User Interface Organisms

The UI organism views combined Figma UI elements such as forms, user management views, email invitation views, and password reset views. The section was initially called "Components", and participants were requested to evaluate the term and propose other, more appropriate names.

All participants associated the term "components" with individual UI elements, for example, labels, input fields, buttons, notification boxes, or icons. However, P2 and P6 viewed the existing term as acceptable, noting that "components" is commonly used in development to refer to concrete interface constructs, and thus felt familiar. P9 remained uncertain and hesitant about proposing a more suitable alternative.

P1 preferred the term patterns, while P5 considered the elements to be UI patterns or solutions, arguing that they constituted larger entities than components. P3 described them as views, a position also taken by P4, whereas P7 suggested terms such as assemblies, combinations, or groups. P8 considered pattern library an appropriate option, as it conveyed the idea of bringing multiple element types together.

After collecting the initial reflections, participants were presented with three naming options for evaluation and selection:

1. Building blocks
2. Organisms
3. Assembled solutions

The results are summarized in Table 6. Responses varied a lot of between the participants.

Table 6: Participants' preferred naming for abstraction levels.

Preferred naming	Participants
Building blocks	P4, P5, P6, P7, P9
Organisms	P3, P8
Assembled solutions	P1, P2

Most participants expressed reservations about all three naming options and framed their preferences as relative rather than absolute. P1, for instance, considered assembled solutions workable but did not regard any of the alternatives as a fully adequate fit. Three participants critiqued "building blocks" for being either too generic or too closely aligned with the existing terminology of "components": P2 described it as somewhat forced and insufficiently descriptive of the material, while P3 viewed it as overly similar to "components" and therefore preferred "organisms", drawing on its established use in earlier DS literature. P8 likewise regarded "building blocks" as "not incorrect" but overly broad, and thus viewed "organisms" as the more specific option.

In contrast, the rest viewed "building blocks" more favorably. P4 found it logically sound and suitable; P5 indicated that, despite general hesitation about all options, it could function effectively in practice; P6 maintained that "components" would be the most accurate choice from a development perspective but considered "building blocks" the most acceptable alternative for non-developers; and P9 selected it as the clearest and most descriptive for the intended audience. The term "organisms" was also explicitly rejected by some participants: P7, for example, associated it too strongly with biology and therefore favored "building blocks", which was seen as flexible enough to encompass both simple and more complex groupings.

Finally, although P2 considered "organisms" conceptually the strongest candidate, "assembled solutions" was ultimately chosen due to concerns that the term "organisms" could be misunderstood by some users.

UX flow

Regarding practical utility, participants reported that the UX flow diagrams supported confident decision-making, particularly in determining the circumstances under which a pattern should be applied. The diagrams were considered highly helpful, not only for gaining a more detailed understanding of the flow and the user's steps, but also for facilitating communication between stakeholders. UX flows aligned well with the participants' natural approach to interpreting and evaluating design patterns.

Because designs are approached from a more technical perspective by developers, P4 noted that the UX flow could be used to double-check the sequence prior to implementation or to "spar" on corner cases, enabling reflection on possible scenarios and the subsequent steps that should be presented to the user. P8 similarly observed that the UX flow clearly highlighted the principal steps and interaction points, allowing designers and developers to easily identify the situations in which a pattern is applicable. In particular, there was a preference for complex patterns to present the steps and

navigation points explicitly so that the overall logic could be understood quickly.

A video was also included alongside the UX flow to demonstrate it within a real use case. Although this was appreciated, especially by designers, participants generally felt that a clickable prototype aligned with the step descriptions would be more beneficial. The ability to examine the steps independently and at a self-selected pace was considered important. P9 noted that, in video format, the specific step they wished to inspect often passed too quickly, and P5 similarly commented that the pace of the video made it difficult to fully grasp the sequence of actions involved.

Side Navigation

Navigation and overall page structure emerged as key themes during the validation. Participants emphasized that a table-of-contents-style sidebar on the right side of the page was perceived as highly useful for enabling efficient movement between sections. This side navigation was regarded as dividing the page into clearer segments and improving the ability to scan through it at a glance. It was considered particularly valuable for bypassing longer text blocks and accessing the relevant section directly. P7 noted that, provided a table of contents with quick links is continuously available, a long page becomes acceptable because users can rapidly jump to the required specifications.

The side navigation was noticed almost immediately by participants, who attempted to interact with it even though the links were not yet functional in the prototype. As they continued navigating through other pages, the expectation that the sidebar should be present and accessible became more pronounced, indicating that it quickly formed part of their mental model for navigating the documentation.

5.3.4 Future Prospects and Improvements

Participants identified several opportunities to enhance both the visual design and the structural organization of the documentation pages. However, concerns were also expressed regarding overall clarity. As P8 articulated, "These pages are really long, so I'm not entirely sure I can grasp where we are and what is there. Could it be organized, structured even more clearly? That's always a challenge, of course, in cases where a large amount of material must be presented."

Suggestions included incorporating additional visual elements and positioning them closer to the top of the page so that quicker comprehension could be supported. Numerous comments addressed the overall page layout and style, such as the need for increased spacing, clearer section divisions and headings, and improved visual distinctiveness between UI organism views and the surrounding text. Participants also expressed a clear preference for clickable prototypes over videos.

In addition, participants called for stronger alignment between textual descriptions and visual examples. P1 recommended that each example could be accompanied by an explanatory text placed directly above it, ensuring that the described content corresponds precisely with the visual representation. A clearer distinction between alternative styles was also requested by P5, including explicit explanations for the

existence of multiple variants and guidance on the circumstances under which each variant should be applied.

Across these suggestions, participants emphasized the importance of balancing textual content with visual material. P1 proposed that more general explanations could be reduced in favor of a slightly more technical perspective with less narrative detail. Although many participants expressed a preference for shorter and more concise text, they also acknowledged that identifying a universally optimal level of detail is challenging, as some users require more extensive textual information than others.

Connection to Existing Information

Several participants expressed a desire for stronger connections between the documentation and existing implementations in production applications, particularly in cases in which the same UI organisms are already in use. It was suggested that such connections be made explicit so that real examples can be located more easily, existing solutions can be benchmarked, and contact with responsible teams can be facilitated when necessary. In practice, this could include links to code repositories, Storybook entries, or Figma files, as well as contact information for the owning team.

P3 additionally noted that, if the documentation introduces multiple styles, it would be important to link these to existing products with the style in use as well as the reasoning behind the choice. P6 further suggested that reducing page length and content density could be supported by linking to other internal sources, especially in instances where decision-making authority resides with another team, such as IT security.

Developers emphasized the importance of linking UI organisms more directly to existing DS components. As P2 commented, "If any team has already implemented a solution based on this model, could you include some references to it, even if it was done by team A, so that you could ask them about it or look at their code if necessary?" P2 also questioned the current lack of clarity by noting, "What I can't see directly here is, for example, which DS components these elements use. Is there a practical implementation available for any of these things?"

Taken together, these perspectives underscore the need for tighter integration between the proposed UX pattern documentation, real-world application examples, and the technical artifacts of the DS, enabling users to draw more confidently on existing knowledge, established implementations, and organizational expertise.

5.3.5 Maintenance and Sustainability of the Documentation

Concerns were expressed by many participants regarding the extent to which the documentation would remain up-to-date, as outdated information was seen as significantly diminishing its usefulness and credibility. P3 emphasized that the documentation should be structured so that maintenance and updates can be carried out with minimal effort, given that UX patterns may evolve either rapidly or in more incremental ways over time.

DS developers reflected on this topic from a broader perspective, considering the challenges inherent in managing multiple layers of documentation and a diverse set of media formats, including images, videos, diagrams, text, and code components. P1 noted that the maintenance burden increases with the quantity of examples provided: while updates to code examples are often propagated automatically when changes are made (except in cases involving completely new examples), a deliberate effort must still be made to keep supporting textual descriptions aligned. P8 observed that users often appreciate interactive demos, videos, and images, yet both P8 and P5 emphasized that visual assets in particular become outdated quickly.

Several practical measures were proposed to support long-term maintainability. Many participants considered it essential that each UX pattern entry include version information or a clearly visible "last updated" date. In the absence of such indicators, an impression may arise that the content is outdated, which could deter use. Additionally, P3 implicitly pointed to the need for clear ownership and defined maintenance responsibilities, lightweight update processes, and regular review cycles so that the documentation can keep pace with evolving patterns, tools, and implementation practices.

6 Discussion

6.1 Revisiting the Research Questions

This section reflects on the findings of the study in relation to the RQs 1–4 introduced in Section 1.2.2. RQ 5 is further discussed in Section 6.2. The refined proposal for the documentation framework, derived from the interview results, is presented in Section 6.3.

6.1.1 Research Question 1

RQ 1 addressed the main qualities of the UX pattern to embody. The analyses in Section 5.1 showed that UX patterns in this thesis can be identified through two analytical indicators: vertical reach within a single application and recurrence across applications. Within one system, candidate patterns appear as flows within top-down analyses that extend from high-level goals through tasks to concrete user actions, indicating structural prominence in the overall UX. Across systems, similar flows with comparable internal structure suggest that the same underlying interaction logic is being reused. This led to the analytical definition that a UX pattern is the largest common connected subset of top-down analysis elements shared between applications.

These characteristics operationalize the conceptual definition in Section 4.1.2, where UX patterns were defined as reusable, context-dependent solutions that address recurring UX problems and emphasize how experiences unfold over time through interaction paths and touchpoints. The results uncover UX patterns as compact elements at the organism level of the classification by Frost (2016) with the ability to be transferred to new contexts with small modifications. This supports the view that UX patterns are temporally extended (Borchers 2000a, van Welie & van der Veer 2003), hierarchically spanning (van Welie & van der Veer 2003), and reusable across contexts (Seffah 2010), rather than being isolated UI fragments. At the same time, the analysis highlights a dependency on the availability and quality of design artifacts: when flows or documentation are incomplete or highly divergent, it becomes harder to determine whether a recurring structure represents a true UX pattern or a local design coincidence. In practice, the existence of identifiable UX patterns is tightly linked to the maturity and coherence of the surrounding design work.

6.1.2 Research Question 2

RQ 2 considers the identification of UX patterns from diverse software contexts. Section 5.2 introduced a UX pattern identification framework that combines top-down analysis, HTA and TWO into a two-phase process: pattern identification across systems, followed by abstraction and formalization. Top-down analysis highlights regions of the activity space where potential patterns are likely to emerge (van Welie & van der Veer 2003). HTA decomposes those regions into concrete actions and decision points, making alternative paths and sub-patterns visible (Annett & Duncan 1967, French et al. 2019, Hornbæk et al. 2025). TWO captures roles, tasks, artifacts

and constraints, explaining why particular interaction sequences take the form they do (van Welie 2001).

Together, these methods allow candidate patterns and sub-patterns to be discovered and compared across heterogeneous products in a systematic, empirically grounded way. The abstraction phase then translates graph-based findings into a common textual schema (name, problem, context, constraints/forces, solution, examples) that is compatible with existing pattern literature (Table 2). and directly feeds into the documentation framework. This answers RQ 2 by demonstrating that UX patterns can be identified through a structured combination of activity-level and task-level analyses, and abstracted into reusable descriptions that preserve both interaction logic and contextual constraints. The approach, however, is inherently oriented towards patterns that are strongly tied to user tasks and flows; purely visual conventions or very local micro-interactions are less likely to be captured, which is consistent with the interaction-centered definition of UX patterns adopted in this thesis (Section 4.1.2).

A notable finding is that CJMs, despite their widespread use in service design and UX practice (Alvarez et al. 2020, Weidig et al. 2024, Kalbach 2016), proved insufficient for robust UX pattern discovery. CJMs provided valuable macro-level narratives and helped establish an initial understanding of service phases and touchpoints, yet they did not reveal the granularity or structural depth that distinguishes one pattern variant from another. Conceptually, this finding reinforces a view of UX patterns as fundamentally task-embedded and temporally structured artifacts. The evidence indicates that the distinguishing features of a UX pattern reside mainly at the task and action levels, consistent with the hierarchical structuring described by van Welie & van der Veer (2003) and the task-based design process proposed by van Welie (2001). This aligns with the definition adopted in Section 4.1.2, which defines UX patterns as solutions that emphasize how experiences unfold over time through interaction paths and touchpoints.

6.1.3 Research Question 3

The thesis addresses RQ 3 by proposing a documentation framework (Section 4.3) and by evaluating an initial instantiation of that framework (Section 5.3). The framework structures each pattern as a combination of concise textual fields and rich visual material: an introduction that situates the pattern in its context, a structured description of problem, context, forces, examples (Alexander et al. 1977, van Welie et al. 2001), as well as annotated UI organisms and their variations, UX flows that capture temporal progression, and links to relevant design and implementation assets.

This design aligns with the DS definition in Section 4.1.1, which describes a DS as a centralized collection of guidelines, rules and reusable UI elements aligned with the brand to ensure consistent UIs and UX across applications (Kholmatova 2017, Churchill 2019, Vesselov & Davis 2019, Werle 2021). UX pattern documentation extends this collection by adding a reusable layer of interaction flows and their rationale above atomic components and organisms (Frost 2016). Empirical feedback in Section 5.3 indicates that such documentation supports reuse when patterns are clearly named, quickly recognizable from their visual overview, and explicitly connected to existing Figma components, code and Storybook entries. In this way, the documentation

makes it easier for teams to see when a known solution fits a new problem and how to apply it without re-inventing it, thus integrating UX patterns into the DS as practical, product-spanning artifacts rather than purely theoretical constructs (Feng et al. 2023, Yew et al. 2020).

6.1.4 Research Question 4

The answer to RQ 4 was provided in Section 5.3, and the results showed broad agreement on several design principles. First, participants preferred visual-first entry points: an annotated screenshot or UX flow diagram at the top of the page helped them immediately decide whether they were looking at the right pattern, in line with recommendations on pattern visualization in both academic and industry sources (Tidwell 2010, *Carbon Design System* n.d., *Lightning Design System 2* n.d., *Ethos Design System* n.d.). Second, they valued layered detail, where a brief overview and key fields are available for quick scanning, with more extensive explanations and examples accessible when needed (Shneiderman 1996, Ayres 2025). Third, they emphasized the importance of explicit guidance on variations and corner cases, including recommendations on when to choose one variant over another or extend it (Feng et al. 2023, Ayres 2025) and how different options relate to constraints such as device type or security requirements (Nilsson 2009). Finally, robust navigation support, such as a sidebar table of contents and clear headings, was seen as critical for working efficiently with long documentation pages, as applied in *Kela Design System* (n.d.).

By contrast, long blocks of uninterrupted text without immediate visual anchors, and pages that attempted to show too many examples at once, were experienced as heavy and difficult to approach. These findings refine the initial framework by underscoring that structure and presentation are as important as content: to be adopted in everyday work, pattern documentation must balance completeness with readability (Alchimowicz & Nawrocki 2016) and must respect the time-pressured, task-oriented way in which designers and developers consult DS resources (Gustafsson 2021).

6.2 UX Pattern Documentation: Consistency, Reuse, Governance

This subsection synthesizes participants' accounts to clarify how UX pattern documentation functions as a mediating layer between abstract interaction logic and concrete implementation assets and, in doing so, addresses RQ 5. Viewed through this lens, the documentation aims to unify definitions across roles, align interaction sequences over time with component libraries, and embed maintenance responsibilities into everyday practice, by integrating all the relevant information and assets into a single source of truth. This interpretation is consistent with earlier views of design patterns and the DS as shared artifacts, making up a "lingua franca" that coordinates understanding across disciplines (Erickson 2000, Borchers 2000b). The subsections that follow detail how participants perceived the UX pattern documentation to support

1) consistency of interaction and visual language, 2) reuse of solutions across contexts, and 3) governance of the DS.

6.2.1 Consistency

A central contribution of UX pattern documentation lies in its ability to establish and sustain consistency across teams, products, and implementation contexts. One important way in which this occurs is the use of a common terminology. Although participants reported varying interpretations of what should be classified as a "UI pattern" versus a "UX pattern", they agreed on the view that the documentation should provide a shared naming scheme that reduces ambiguity and clarifies levels of abstraction. In this regard, terminology was treated as a practical instrument for coordinating design and development work, echoing prior observations that a shared design vocabulary supports collaboration and mutual understanding in pattern-driven environments ([Borchers 2000b](#), [Werle 2021](#), [Carey et al. 2025](#)). A UI organism name that is conceptually meaningful while also traceable to concrete DS artifacts, such as Figma components, Storybook entries, design tokens, or code modules, helps participants connect the documented concept to the available assets.

Beyond terminology, participants emphasized the importance of consistency and visual clarity within documentation entries in fostering a shared understanding across roles. Designers, developers, and other stakeholders often approach patterns from different perspectives, prioritizing either intended UX, implementation feasibility, or system fit. The findings suggest that visual-first documentation can bridge these differences by making the logic of the pattern immediately visible. In doing so, they help align mental models across roles and reduce the risk that the same pattern will be interpreted differently by different contributors. As described in [Hornbæk et al. \(2025\)](#), a mental model built from earlier experiences with a common structure aids in grasping new entries faster due to the inherent limitation of working memory, corresponding to the participants' wishes. In addition, because a visual-first entry point reduces cognitive load by presenting the pattern's structure through its natural medium ([Shneiderman 1996](#), [Sweller 1988](#)), the threshold for utilizing such documentation is expected to be low. This is also supported by the multimedia learning principle by [Mayer \(2009\)](#), which states that people learn better from the combination of words and pictures than from words alone.

Consistency was also understood more broadly as the ability to maintain a unified UX feel and UI look across products and use contexts, even when the underlying implementations or visual details may vary. Participants consistently associated patterns with the orchestration of components over time, echoing the multi-level view of experience as discussed in Section 2.5.2 ([Borchers 2000a](#), [van Welie & van der Veer 2003](#)). This understanding positions pattern documentation as a means of preserving interaction consistency across system states and products. By documenting patterns as recurring sequences of interactions, together with their touchpoints, options, and edge cases, the documentation helps ensure that products express a coherent interaction logic even when domain-specific adaptations are necessary. Therefore, consistency extends beyond rigid visual uniformity by facilitating continuity in the intended UX.

The documentation supports this continuity by clarifying what aspects of the pattern should remain fixed and what can vary. In this sense, the proposed documentation structure can be read as a concrete instantiation of the consistency and shared-practice goals that contemporary DSs pursue through pattern libraries and component catalogs. This links the architectural concerns of Section 3.5 directly to day-to-day design and development work.

6.2.2 Reuse

The findings further indicate that UX pattern documentation is expected to support reuse by helping teams decide how and when a documented solution should be reused. Despite this, participants expressed a need for clearer guidance on how variations should be executed in practice to stay consistent with the pattern. This was especially relevant in cases where participants pictured encountering a familiar UX problem structure but needed to address domain-specific constraints or platform-specific requirements. This suggests that reuse depends on explicit principles for adaptation that preserve coherence within the broader system. This reflects the earlier observations in literature stating that patterns must make their context, constraints and trade-offs visible in order to be safely reused (Alexander et al. 1977, van Welie et al. 2001).

Participants also highlighted that reuse becomes more effective when the pattern's scope and intended interaction logic are made immediately understandable through a visual-first approach. It was considered especially effective in helping users rapidly understand what the pattern is intended to achieve, how it unfolds across states, and what it covers. Such materials support rapid recognition of whether a current design or implementation challenge corresponds to an already documented pattern. This precision helps teams determine whether reuse is appropriate and reduces uncertainty that might otherwise encourage them to create a custom solution unnecessarily. When combined with layered documentation structures that allow users to enter at different levels of detail, these representations further support reuse by reducing cognitive load while preserving access to more detailed rationale, variants, and edge cases when needed. This layered approach mirrors the way existing pattern collections in both academia and industry combine high-level descriptions with concrete “known uses” and examples (van Welie et al. 2001, Carbon Design System n.d., Lightning Design System 2 n.d., Ethos Design System n.d.), and it aligns with Section 3.3.1 where differences in documentation depth were shown to affect how easily patterns can be operationalized.

A further prerequisite for reuse identified by participants was clear linkage between UX patterns and implementation assets in the DS. Developers in particular emphasized that documentation becomes substantially more reusable when patterns are connected directly to code components, design tokens, and existing visual assets. In practical terms, such linkage helps prevent unnecessary redevelopment by making visible that a relevant solution, or part of one, already exists within the system. This requirement strongly reflects key DS activities identified in the literature; defining a design language, assembling a pattern library and generating documentation (Lamine & Cheng 2022), and supports the view from Section 3.5 that modular, interchangeable components are

important for scalable reuse in complex ecosystems.

6.2.3 Governance

Participants' concerns about sustainability (Section 5.3.5) underscore that documentation promotes consistency and reuse only if it is maintained and governed in ways that users trust. Visible metadata, such as development status, and last-updated information, together with explicit version ties to components and example implementations, were regarded as essential signals that allow users to assess reliability at a glance. Clear deprecation markers were likewise seen as necessary for preventing unintended reliance on outdated guidance. DS developers described these controls as fundamental mechanisms for mitigating maintenance risk, particularly as the number of patterns, variants, and visual assets expands. These expectations are closely aligned with the governance models discussed in Section 3.4, where DSs are described as requiring explicit workflows for change management, contribution and deprecation rather than being maintained informally (Suarez et al. 2019, Lamine & Cheng 2022, Feng et al. 2023).

The findings also highlight broader implications for the governance of UX pattern documentation within DS practice. Participants emphasized that meaningful progress cannot be achieved through the responsibility of a single role or team. Instead, the results suggest that more diverse contributors should be involved throughout the creation and maintenance of patterns. When guidelines and instructions are shaped collectively, a shared understanding is strengthened, and patterns become better aligned with the everyday realities faced by different disciplines. This broader participation enables the documentation to reflect a wider range of use cases, increasing its relevance and reducing the risk that patterns become detached from actual product work. In addition, this projects earlier recommendations that pattern work should involve designers, developers, HCI specialists and other stakeholders in co-creation and review activities (Bayle et al. 1998, Kholmatova 2017, Feng et al. 2023), and it is compatible with the hybrid governance model in which a core DS team collaborates with distributed contributors across the organization (Suarez et al. 2019).

Furthermore, the empirical results imply that systematic review practices or a dedicated working group could substantially improve the long-term usefulness of the documentation. Regular evaluation cycles would help ensure that patterns remain up to date, that emerging inconsistencies are identified early, and that the documentation evolves in parallel with ongoing DS development. In this sense, the maintenance process is a strategic mechanism for supporting quality, cohesion, and long-term adoption. From the perspective of Section 3.5, such iterative refinement can be seen as part of the slow evolution process of a pattern language, where relationships between patterns are continuously harmonized to keep the overall architecture coherent (Kholmatova 2017).

These observations collectively indicate that UX pattern documentation functions most effectively when it is supported by sustained, collaborative governance. This interpretation suggests that the success of a pattern documentation depends on factors beyond the artifact content itself. It is affected by the processes and social structures

that maintain it. In summary, the governance-related findings of the empirical study are well aligned with the ownership and governance concerns (Section 3.4) raised and the architectural and modularity requirements (Section 3.5) covered in literature. They reinforce the view that for UX pattern documentation to be truly effective, it should be embedded within the agile organizational structures, review processes and development practices used in product development, to ensure the information is reliable and continuously updated (Lamine & Cheng 2022, Frost 2016, Kholmatova 2017).

6.3 Refined Proposal for the Documentation Framework

The empirical study indicates that the proposed UX pattern documentation framework is somewhat aligned with the everyday practices and expectations of a subset of its primary user group. At the same time, the evaluation revealed notable tensions. The prototype pages were perceived as lengthy and text-heavy, and the introductory notification of the pattern being under development was reported as drawing more attention than intended. Several participants described initial difficulty in understanding the overall purpose of the page, the scope of the pattern, and the areas requiring focused attention. Although comprehension increased as scrolling, tab navigation, and the discovery of visual elements occurred, the initial cognitive load remained high. These reactions imply that, although the conceptual structure of the framework resonates with users, its current presentation risks overwhelming them, particularly under time constraints.

Based on the results of the empirical validation, an updated proposal for RQ 3 regarding the documentation structure is as follows:

1. Provide descriptive yet concise names for the pattern and any related sub-patterns.
2. A version number, and if needed, the development status (e.g., "under development").
3. A visual example, such as an annotated UI organism or a UX flow diagram.
4. An introduction that ties the UX pattern to its context and problem.
5. UI organisms, variations and corner cases with detailed guidelines to foster broader adoption.
 - (a) References to existing use cases of the pattern and the corresponding visual and code assets.
 - (b) An option for users to inspect the UI organism at desktop, mobile, and (optionally) tablet sizes.
6. A UX flow that emphasizes the main activities through a graph or a clickable prototype.

Overall, the findings can be interpreted as offering a conditional validation of the framework. The content types and pattern abstractions are regarded as useful and relevant, yet they require a more modular, visually oriented, and role-sensitive form of presentation to function effectively in practice. Furthermore, emphasis must be placed on the actual needs and pain points of end-users, and the issue should be resolved in a manner that promotes greater benefit and higher utilization.

6.4 Challenges and Limitations

6.4.1 Lacking and Narrow Data

The empirical validation in this thesis reflects participants' feedback primarily on the overall structure and key elements of the proposed UX pattern documentation model. Because the pattern content itself had not yet reached full maturity, the evaluation focused on the clarity, logic, and usefulness of the documentation framework rather than on the completeness of the individual patterns. This framing is important for interpreting the results: participants were able to validate the organization of the documentation but could not fully assess the content that a more mature set of patterns would contain.

Because UI organism representations, coherent visual standards, and shared design rationales were missing or inconsistent across the products considered, it was not possible to provide actionable recommendations, prescriptive best practices, or detailed implementation instructions. As a consequence, the identified patterns were conceptually clear but practically thin. These challenges point to a broader theoretical issue: the maturity of a pattern fundamentally shapes the type of guidance it can support. When a pattern is still emerging, implemented inconsistently across products, or lacking stable visual and interaction conventions, deriving actionable and generalizable guidelines becomes inherently difficult. Immature patterns do not yet possess consolidated rules, typical variations, or shared design rationales that can be translated into robust documentation ([van Welie et al. 2001](#)).

Taken together, the findings suggest that the documentation model proposed in this thesis provides a solid structural foundation, but its practical utility will increase as more complete and consistent data becomes available. Only once patterns reach sufficient maturity, supported by unified visual styles, shared design rationales, and stable UI organism structures, can the documentation be enriched with detailed guidance that reliably supports designers, developers, and DS teams in implementation.

6.4.2 Analysis Methods

It is acknowledged that a composite HTA, combining several products, involves a trade-off in which UI-level granularity is reduced in favor of cross-product generality. However, because the software products supported the same primary goal and demonstrated substantial structural similarity with only minor parameter-level variations, consolidation into a single HTA was considered methodologically appropriate ([Martinié et al. 2011](#)). Separate HTAs would be warranted if materially different personas

or contexts were involved, if the task flows diverged in risk-relevant ways, such as mandatory verification stages or SSO-exclusive pathways, or if UI-level ergonomics constituted the primary analytical focus. In the present study, these conditions were not met.

A similar rationale applies to the consolidated ontology used in the TWO analysis. A unified ontology also entails a trade-off between UI-level detail and domain-level generality. The unified approach is appropriate when the goal, actor set and task topology are shared, and observed differences are minor.

Several limitations must be acknowledged when interpreting the findings and evaluating the proposed framework. Because the software products supported the same user goal and exhibited high structural similarity, the single composite HTA and TWO provided an effective basis for identifying sub-patterns across touchpoints. Pattern variants preserved product-specific nuances, but fine-grained UI details were necessarily reduced. Furthermore, since HTA is governed by relatively few formal rules, different analysts may produce varying hierarchical structures when analyzing the same task domain ([Stanton 2006](#)), which should be taken into account when evaluating the replicability of the analysis.

6.4.3 Empirical Evaluation

First, the empirical evaluation was conducted within a single organization and a specific DS context. Although this enabled in-depth engagement with the existing practices and constraints of that environment, the generalizability of the findings can be understood through a lens of analytical generalization rather than statistical generalization ([Yin 2017](#)). Other organizations may operate with different team structures, toolchains, or documentation cultures, which could influence both expectations regarding UX pattern documentation and the ways in which such documentation is interpreted.

Second, the prototype used for validation focused on a single pattern: account access (login and registration). This domain is important and relatively complex, yet it represents only a narrow segment of the interaction types, domains, and abstraction levels that a comprehensive pattern library would be expected to support. It therefore remains an open consideration whether the framework would scale effectively to domain-specific business flows, cross-channel experiences, or micro-interactions.

Third, the prototype itself was in an early stage of fidelity. Certain interaction elements, such as side-navigation links, were not fully functional, and the visual design of specific components, such as notification boxes, drew more attention than intended. As prototype fidelity is known to influence how design solutions are perceived and evaluated ([Snyder 2003](#)), these limitations may have shaped participants' experience of information density and navigational clarity, and such influences should be taken into account when interpreting the evaluation.

Fourth, although the participant group included designers, developers, and DS developers, the group was relatively small and composed of individuals already familiar with existing DS assets. However, research on qualitative sample sizes indicates that studies with relatively homogeneous populations and narrow aims commonly reach data saturation within nine to seventeen interviews ([Hennink & Kaiser 2022](#)),

suggesting that the sample of nine participants was within an appropriate range for the scope and population of this study. Their perspectives are therefore likely to differ from those of entirely new users or stakeholders who engage with the documentation only occasionally, such as POs, software testers, or accessibility specialists.

Finally, the study relied on qualitative interviews and observational data from validation sessions. Although these methods provide rich contextual insight ([Braun & Clarke 2006](#), [Ahmed et al. 2025](#)), they do not enable quantification of outcomes such as development speed, defect rates, or long-term consistency across products. Future work would benefit from complementing these qualitative findings with quantitative metrics collected over extended periods of real-world use.

7 Conclusion

This thesis examined how reusable UX patterns can be identified from existing software and documented as part of a DS in order to support greater consistency across digital products. From an academic perspective, the work sought to establish a preliminary framework for identifying UX patterns from stable software products. From an industrial perspective, the work responded to the commissioning organization's need to strengthen consistency across its product portfolio by integrating reusable UX patterns into the existing DS. The results indicate that UX patterns can be treated as reusable, temporally unfolding interaction solutions that connect higher-level user goals to concrete interface behavior, and that their systematic documentation can strengthen a DS by introducing a layer between atomic UI elements and broader UX intent.

First, RQ 1 asked what characteristics qualify a UX pattern. The findings showed that a candidate UX pattern appears as a flow that spans multiple hierarchical levels, from goals to tasks and actions, and becomes a valid reusable pattern when a similar connected structure recurs across products. This led to the analytical understanding of a UX pattern as the largest common connected subset of top-down analysis elements shared between applications.

Second, RQ 2 examined how UX patterns can be identified and abstracted from diverse software contexts. The thesis proposes a two-phase identification framework. In phase one, top-down analysis is conducted to uncover main patterns while sub-patterns are extracted from HTA and TWO analyses. In phase two, the uncovered patterns are formalized and prepared for documentation by defining, naming, and scoping them. The resulting framework demonstrates that UX patterns can be systematically extracted from stable software artifacts and abstracted into reusable descriptions within the organization's context.

Third, RQ 3 focused on how UX patterns should be documented in a DS context. The answer developed in this thesis is a documentation model that combines concise textual fields with visual and behavioral representations. The proposed structure comprises a clear pattern name, versioning information, a context description, UI-level specifications and visual examples with links to relevant DS assets, and UX flow representations. The results suggest that such documentation enables UX patterns to be communicated as reusable design knowledge and extends the DS beyond components and guidelines to also capture recurring interaction logic.

Fourth, RQ 4 examined which aspects of the documentation were perceived by its users to support them the most in practice. The empirical evaluation showed that a visual-first approach, a layered access to detail, and clear connections from the documentation to existing visual and technical assets were considered useful. Annotated examples, UI organisms, and UX flows were consistently valued because they allowed participants to quickly recognize the pattern, assess its relevance, and move efficiently between sections. At the same time, the evaluation showed that long, text-heavy pages without immediate visual anchors create unnecessary cognitive load. These findings refined the initial proposal by emphasizing that documentation effectiveness depends on both content and the way in which that content is presented,

structured, and maintained.

Fifth, the answer to RQ 5 clarified how UX pattern documentation contributes to the consistency, reuse, and governance of the DS. The results indicate that documented patterns can support 1) consistency by providing a shared language and a common interaction logic across teams and products; 2) reuse by helping teams recognize when an existing solution fits a new problem and how it should be adapted; and 3) governance by requiring ownership, versioning, review cycles, and links to implementation assets so that the documentation remains trustworthy over time. In this sense, the thesis shows the importance of embedding UX patterns into sustainable organizational practices.

The academic goal was achieved through the demonstration that analytical artifacts can be converted into reusable interaction knowledge. The assumed UX patterns from the team could be traced in these analytical artifacts and understood as recurring common elements. The thesis therefore makes a methodological contribution by showing how established analytical methods can be combined to support UX pattern identification.

The organizational goal was addressed in several ways. First, the work initiated discussion about UX patterns within the design team, creating a basis for shared understanding. Second, implicitly assumed patterns were surveyed and then examined against existing software products in order to determine whether they could be supported analytically. Third, established design systems were benchmarked to inform the structure and contents of pattern documentation. Finally, the work resulted in a clickable prototype of a UX pattern documentation gallery that demonstrated how such patterns could be presented for team-wide use. As a result, the organization now has a concrete starting point for introducing UX patterns into the DS in a way that supports coherence across products.

In practical terms, this work identifies several immediate opportunities for further development. These include publishing the pattern documentation as part of the organization's DS and linking the patterns directly to current Figma assets, Storybook entries, component implementations, and known production examples in order to strengthen both usability and credibility. In parallel, the existing product portfolio could be aligned more systematically with the identified UX patterns.

Building on this starting point, several directions for future work remain. Methodologically, future research should examine the generality and replicability of the identification framework in other organizational settings, with other analysts, and under more varied product conditions. In the longer term, the qualitative insights generated in this study should be complemented with quantitative measures, such as effects on development speed, defect reduction, onboarding efficiency, or UX consistency across products.

In conclusion, this thesis demonstrates that UX patterns can be systematically identified from existing software and formalized into a documentation structure that supports reuse within a DS. In the organization's environment, this work provided both an initial methodological framework for pattern discovery and a practical prototype for pattern documentation. While the resulting patterns and documentation remain at an early stage of maturity, the thesis establishes a credible and extensible foundation for future UX pattern work.

References

- Ahmed, S. K., Mohammed, R. A., Nashwan, A. J., Ibrahim, R. H., Abdalla, A. Q., M. Ameen, B. M. & Khedhir, R. M. (2025), 'Using thematic analysis in qualitative research', *Journal of Medicine, Surgery, and Public Health* **6**, 100198.
URL: <https://www.sciencedirect.com/science/article/pii/S2949916X25000222>
- Alchimowicz, B. & Nawrocki, J. R. (2016), 'The COCA quality model for user documentation', *Software Quality Journal* **24**, 205–230.
URL: <https://doi.org/10.1007/s11219>
- Alexander, C. (1979), *The Timeless Way of Building*, Vol. 1 of *Center for Environmental Structure*, Oxford University Press, New York.
- Alexander, C., Ishikawa, S., Silverstein, M., Jacobson, M., Fiksdahl-King, I. & Angel, S. (1977), *A Pattern Language: towns, buildings, construction*, Oxford University Press, New York.
- Alvarez, J., Léger, P.-M., Fredette, M., Chen, S. L., Maunier, B. & Senecal, S. (2020), 'An enriched customer journey map: How to construct and visualize a global portrait of both lived and perceived users' experiences?', *Designs* **4**, 29.
- Annett, J. & Duncan, K. (1967), 'Task analysis and training design', *Journal of Occupational Psychology* **41**, 211–221.
- Archer, B. (1979), Whatever became of design methodology?
URL: <https://api.semanticscholar.org/CorpusID:201629405>
- Axelsson, J. (2022), Systems-of-systems design patterns: A systematic literature review and synthesis, in '2022 17th Annual System of Systems Engineering Conference (SOSE)', pp. 171–176.
- Ayres, C. (2025), 'Design system 103: Design system documentation that drives adoption'. Accessed: 24 January 2026.
URL: <https://www.figma.com/blog/design-systems-103-documentation-that-drives-adoption/>
- Bayle, E., Bellamy, R., Casaday, G., Erickson, T., Fincher, S., Grinter, B., Gross, B., Lehder, D., Marmolin, H., Moore, B., Potts, C., Skousen, G. & Thomas, J. (1998), 'Putting it all together: towards a pattern language for interaction design: A chi 97 workshop', *SIGCHI Bull.* **30**(1), 17–23.
URL: <https://doi.org/10.1145/280571.280580>
- Beauregard, R. & Corriveau, P. (2007), User experience quality: A conceptual framework for goal setting and measurement, Vol. 4561, pp. 325–332.
- Bender-Salazar, R. (2023), 'Design thinking as an effective method for problem-setting and needfinding for entrepreneurial teams addressing wicked problems', *Journal of Innovation and Entrepreneurship* **12**(24).

- Beres, J. (2025), 'A comprehensive guide to design systems'. Accessed: 22 January 2026.
URL: <https://www.infragistics.com/blogs/design-systems/>
- Bertelsen, O. (1999), 'Fitts' law as a design artefact: A paradigm case of theory in software design', *Lecture Notes in Computer Science* **876**.
- Bitner, M. J., Ostrom, A. L. & Morgan, F. N. (2008), 'Service blueprinting: A practical technique for service innovation', *California Management Review* **50**(3), 66–94. (Original work published 2008).
- Blackwell, A. F. & Fincher, S. (2010), 'PUX: patterns of user experience', *Interactions* **17**(2), 27–31.
URL: <https://doi.org/10.1145/1699775.1699782>
- Borchers, J. (2000a), 'Interaction design patterns: Twelve theses', Position Paper, Workshop "Pattern Languages for Interaction Design: Building Momentum", CHI 2000.
URL: <http://www.acm.org/sigchi/chi2000/>
- Borchers, J. O. (2000b), A pattern approach to interaction design, in 'Proceedings of the 3rd Conference on Designing Interactive Systems: Processes, Practices, Methods, and Techniques', DIS '00, Association for Computing Machinery, New York, NY, USA, p. 369–378.
URL: <https://doi.org/10.1145/347642.347795>
- Bottoni, P., Guerra, E. & de Lara, J. (2011), *Formalising Interaction Patterns*, Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 257–276.
URL: https://doi.org/10.1007/978-3-642-14562-9_13
- Boulton, M. (2012), 'Structure first. content always'. Accessed: 23 January 2026.
URL: <https://markboulton.co.uk/journal/structure-first-content-always/>
- Braun, V. & Clarke, V. (2006), 'Using thematic analysis in psychology', *Qualitative Research in Psychology* **3**(2), 77–101.
URL: <https://doi.org/10.1191/1478088706qp063oa>
- Brown, T. (2008), 'Design thinking', *Harvard Business Review* **86**(6), 84–92.
- Brubaker, E., Sheppard, S., Hinds, P. & Yang, M. (2022), Objects of collaboration: Roles of objects in spanning knowledge boundaries in a design company.
- Carbon Design System (n.d.). Accessed: 21 August 2025.
URL: <https://carbondesignsystem.com/patterns/overview/>
- Carey, L. Z. H., Johnson, B. J. E. L., Kumuthini, S., Zhao, C. & Shubham, R. J. (2025), 'Developing shared vocabulary system for collaborative software engineering'.
URL: <https://arxiv.org/abs/2507.14396>

- Churchill, E. F. (2019), 'Scaling UX with design systems', *Interactions* **26**(5), 22–23.
URL: <https://doi.org/10.1145/3352681>
- Cooper, A., Reimann, R., Cronin, D. & Noessel, C. (2014), *About Face: The Essentials of Interaction Design*, 4th edn, Wiley.
- Cross, N., Dorst, K. & Roozenburg, N., eds (1992), *Research in Design Thinking*, Delft University Press, Delft, The Netherlands.
- da Rosa, D. & Silveira, M. (2022), 'May the patterns be with you: A framework for HCI patterns development', *Interaction Design and Architecture(s)* pp. 175–208.
- Dalsgaard, P., Halskov, K. & Basballe, D. A. (2014), Emergent boundary objects and boundary zones in collaborative design research projects, in 'Proceedings of the 2014 Conference on Designing Interactive Systems', DIS '14, Association for Computing Machinery, New York, NY, USA, p. 745–754.
URL: <https://doi.org/10.1145/2598510.2600878>
- Dearden, A. & Finlay, J. (2006), 'Pattern languages in HCI: a critical review.', **21**(1), 49–102.
URL: <https://shura.shu.ac.uk/22/>
- Derleth, T. (2018), Corporate component and service libraries – a concept for creating, maintaining and managing a company-specific user interface component library in the field of frontend web development.
- Deuschel, T. (2018), On the influence of human factors in adaptive user interface design, in 'Adjunct Publication of the 26th Conference on User Modeling, Adaptation and Personalization', UMAP '18, Association for Computing Machinery, New York, NY, USA, p. 187–190.
URL: <https://doi.org/10.1145/3213586.3213587>
- d.school (2010), 'Bootcamp bootleg'.
- Engle, R. W. (2002), 'Working memory capacity as executive attention', *Current Directions in Psychological Science* **11**, 19–23. Original work published 2002.
- Erickson, T. (2000), Lingua francas for design: sacred places and pattern languages, in 'Proceedings of the 3rd Conference on Designing Interactive Systems: Processes, Practices, Methods, and Techniques', DIS '00, Association for Computing Machinery, New York, NY, USA, p. 357–368.
URL: <https://doi.org/10.1145/347642.347794>
- Ethos Design System* (n.d.). Accessed: 21 August 2025.
URL: <https://ethosdesignsystem.com/patterns/administration/user-registration>
- Feng, K. J. K., Li, T. W. & Zhang, A. X. (2023), Understanding collaborative practices and tools of professional UX practitioners in software organizations, in 'Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems', CHI '23,

- Association for Computing Machinery, New York, NY, USA.
URL: <https://doi.org/10.1145/3544548.3581273>
- Folmer, E. (2015), 'Interaction design patterns'. Accessed: 22 January 2026.
URL: <https://ixdf.org/literature/book/the-glossary-of-human-computer-interaction/interaction-design-patterns>
- Frasquet-Deltoro, M., Molla-Descals, A. & Miquel-Romero, M. (2021), 'Omnichannel retailer brand experience: conceptualisation and proposal of a comprehensive scale', *Journal of Brand Management* **28**, 388–401.
- French, A., Taylor, L. K. & Lemke, M. R. (2019), Chapter 6 - task analysis, in M. B. Privitera, ed., 'Applied Human Factors in Medical Device Design', Academic Press, pp. 63–81.
URL: <https://www.sciencedirect.com/science/article/pii/B9780128161630000062>
- Frost, B. (2013), 'Atomic design'. Accessed: 1 September 2025.
URL: <https://bradfrost.com/blog/post/atomic-web-design/>
- Frost, B. (2016), *Atomic Design*, Brad Frost. Accessed: 1 September 2025.
URL: <https://atomicdesign.bradfrost.com/table-of-contents/>
- Gahler, M., Klein, J. F. & Paul, M. (2022), 'Customer experience: Conceptualization, measurement, and application in omnichannel environments', *ournal of Service Research* **26**(2), 191–211. (Original work published 2023).
- Gamma, E., Helm, R., Johnson, R. & Vlissides, J. (1994), *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley Professional.
- Garrett, J. J. (2010), *The elements of user experience*, New Riders Publishing. (2nd ed.).
- Gerea, C., Gonzalez-Lopez, F. & Herskovic, V. (2021), 'Omnichannel customer experience and management: An integrative review and research agenda', **13**(5). (This article belongs to the Special Issue Multichannel and Omnichannel Retailing: Latest Advancements and Further Challenges).
- Gruszka, A. & Nęcka, E. (2017), 'Limitations of working memory capacity: The cognitive and social consequences', *European Management Journal* **35**(6), 776–784. Managing Overflows.
URL: <https://www.sciencedirect.com/science/article/pii/S0263237317301020>
- Gustafsson, E. W. (2021), Defining and evaluating design system usability for improved developer experience, Master's thesis, KTH Royal Institute of Technology, Stockholm, Sweden.
URL: <https://www.diva-portal.org/smash/get/diva2%3A1555395/FULLTEXT01.pdf>
- Hassenzahl, M. (2008), User experience (UX): Towards an experiential perspective on product quality, Vol. 339, pp. 11–15.

- He, S., Chen, Y. & Xia, Y. e. a. (2024), 'Visual harmony: text-visual interplay in circular infographics', *Journal of Visualization* **27**, 255–271.
- Hennink, M. & Kaiser, B. N. (2022), 'Sample sizes for saturation in qualitative research: A systematic review of empirical tests', *Social Science & Medicine* **292**, 114523.
URL: <https://www.sciencedirect.com/science/article/pii/S0277953621008558>
- Heuchert, M. (2019), Conceptual modeling meets customer journey mapping: Structuring a tool for service innovation, in '2019 IEEE 21st Conference on Business Informatics (CBI)', Vol. 01, pp. 531–540.
- Hevner, A. R., March, S. T., Park, J. & Ram, S. (2004), 'Design science in information systems research', *MIS Q.* **28**(1), 75–105.
- Hornbæk, K., Kristensson, P.-O. & Oulasvirta, A. (2025), *Introduction to Human-Computer Interaction*, Oxford University Press.
- Idean (2019), *Revolutionize the way your organization scales design*, Idean Publishing.
- ISO (2019), Ergonomics of human-system interaction – part 210: Human-centred design for interactive systems., Standard 9241-11, ISO - International Organization for Standardization.
- IxDF - Interaction Design Foundation (2017), 'What is hierarchical task analysis?'. Accessed: 23 December 2025.
URL: <https://www.interaction-design.org/literature/topics/hierarchical-task-analysis>
- IxDF - Interaction Design Foundation (2019), 'Don norman'. Accessed: 11 November 2025.
URL: <https://www.interaction-design.org/literature/topics/don-norman>
- IxDF - Interaction Design Foundation (2023), 'How to improve your UX designs with task analysis'. Accessed: 21 January 2026.
URL: <https://www.interaction-design.org/literature/article/task-analysis-a-ux-designer-s-best-friend>
- Jin, Z., Suh, T. & Lee, J.-Y. (2025), 'Omnichannel identity dimensions and their differential impact on customer–brand relationships: A comparative analysis of south korean retailers', *Sustainability* **17**(9).
- Johnson-Laird, P. (2010), 'Mental models in cognitive science', *Cognitive Science* **4**, 71 – 115.
- Kaasinen, E., Roto, V., Hakulinen, J., Heimonen, T., Jokinen, J. P. P., Karvonen, H., Keskinen, T., Koskinen, H., Lu, Y., Saariluoma, P., Tokkonen, H. & Turunen, M. (2015), 'Defining user experience goals to guide the design of industrial systems', *Behaviour & Information Technology* **34**(10), 976–991.
URL: <https://doi.org/10.1080/0144929X.2015.1035335>

- Kalbach, J. (2016), *Mapping Experiences: A Complete Guide to Creating Value Through Journeys, Blueprints, and Diagrams*, O'Reilly Media.
URL: <https://books.google.fi/books?id=fK4fDAAAQBAJ>
- Kela Design System (n.d.). Accessed: 5 January 2026.
URL: <https://design.kela.fi/v16/>
- Kholmatova, A. (2017), 'Design systems: A practical guide to creating design languages for digital products', *Smashing Magazine* .
- Krakowski, A., Bishop, L., Greenwald, E. & Yun, S. (2023), The role of boundary objects in collaborative design, in 'Curator: The Museum Journal', Vol. 66, Wiley Periodicals LLC, pp. 561–568.
URL: <https://doi.org/10.1111/cura.12575>
- Krug, S. (2014), *Don't Make Me Think, Revisited A Common Sense Approach to Web Usability*, New Riders.
- Lamine, Y. & Cheng, J. (2022), 'Understanding and supporting the design systems practice', *Empirical Software Engineering* **27**.
- Larman, C. & Basili, V. (2003), 'Iterative and incremental developments. a brief history', *Computer* **36**(6), 47–56.
- Lawson, B. (2005), *How Designers Think (4th ed.)*, Elsevier Ltd.
- Ledneva, T. & Kovalev, A. (2021), 'Cognitive load measurement during navigation and information retrieval in digital text', *Procedia Computer Science* **192**, 2720–2730. Knowledge-Based and Intelligent Information & Engineering Systems: Proceedings of the 25th International Conference KES2021.
URL: <https://www.sciencedirect.com/science/article/pii/S1877050921017798>
- Lee, J. (1997), 'Design rationale systems: understanding the issues', *IEEE Expert* **12**(3), 78–85.
- Lightning Design System 2 (n.d.). Accessed: 21 August 2025.
URL: <https://www.lightningdesignsystem.com/2e1ef8501/p/76969d-get-started>
- Lundin, L. & Kindström, D. (2024), 'Managing digitalized touchpoints in B2B customer journeys', *Industrial Marketing Management* **121**, 88–99.
URL: <https://www.sciencedirect.com/science/article/pii/S0019850124001135>
- Mahemoff, M. J. & Johnston, L. J. (1998), Pattern languages for usability: An investigation of alternative approaches, in 'Proceedings of APCHI '98 (Asia-Pacific Conference on Human-Computer Interaction)'. HTML version available online.
URL: <http://mahemoff.com/paper/candidate/>

- Martinie, C., Palanque, P. & Winckler, M. (2011), Structuring and composition mechanisms to address scalability issues in task models, *in* P. Campos, N. Graham, J. Jorge, N. Nunes, P. Palanque & M. Winckler, eds, 'Human-Computer Interaction – INTERACT 2011', Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 589–609.
- Massi, M., Piancatelli, C. & Vocino, A. (2023), 'Authentic omnichannel: Providing consumers with a seamless brand experience through authenticity', *Psychology & Marketing* **40**, 1280–1298.
- Mayer, R. E. (2009), *Multimedia Learning*, 2 edn, Cambridge University Press.
- McCarthy, J. & Wright, P. (2004), 'Technology as experience', *Interactions* **11**(5), 42–43.
URL: <https://doi.org/10.1145/1015530.1015549>
- McCarthy, S., O'Raghallaigh, P., Fitzgerald, C. & Adam, F. (2020), Building bridges, burning bridges: The use of boundary objects in agile distributed ISD teams.
- Mirabito, Y. & Goucher-Lambert, K. (2022), Investigating how engineers and designers communicate design rationale.
- Mounika, K. (2021), 'The evolution of component-based architecture in front-end development', *Journal of Scientific and Engineering Research* **8**(7).
- Muthaffar, A., Vilches-Montero, S. & Bravo-Olavarria, R. (2024), 'From digital touchpoints to digital journeys: How shopping mindsets influence appraisal of omnichannel journeys', *International Journal of Information Management* **77**, 102778.
URL: <https://www.sciencedirect.com/science/article/pii/S0268401224000264>
- Nguyen, G. (n.d.), Design system: A tool for scaling product design in large technology companies, Master's thesis, Haaga-Helia. Accessed: 22 January 2026.
URL: https://www.theseus.fi/bitstream/handle/10024/156934/Nguyen_Giang.pdf?sequence=1
- Nielsen, J. (2024), '10 usability heuristics for user interface design'. Accessed: 19 August 2025.
URL: nngroup.com/articles/ten-usability-heuristics
- Nilsson, E. G. (2009), 'Design patterns for user interface for mobile applications', *Adv. Eng. Softw.* **40**(12), 1318–1328.
URL: <https://doi.org/10.1016/j.advengsoft.2009.01.017>
- Norman, D. A. (2013), *The design of everyday things*, revised and expanded edition, Basic Books, New York, New York.
- Norman, D. A. & Draper, S. W. (1986), *User-Centered System Design: New Perspectives on Human-Computer Interaction*, Lawrence Erlbaum Associates, Hillsdale, NJ.

- Osborn, A. (1953), *Applied Imagination: Principles and Procedures of Creative Problem Solving*.
- Ozok, A. A. & Salvendy, G. (2000), 'Measuring consistency of web page design and its effects on performance and satisfaction', *Ergonomics* **43**(4), 443–460. PMID: 10801079.
URL: <https://doi.org/10.1080/001401300184332>
- Pajamas Design System* (n.d.). Accessed: 21 August 2025.
URL: <https://design.gitlab.com/patterns/ai-human-interaction>
- Patrício, L., Fisk, R. P., Falcão e Cunha, J. & Constantine, L. (2011), 'Multilevel service design: From customer value constellation to service experience blueprinting', *Journal of Service Research* **14**(2), 180–200. (Original work published 2011).
- Polaris React* (n.d.). Accessed: 21 August 2025.
URL: <https://polaris-react.shopify.com/>
- Porsche Design System* (n.d.). Accessed: 21 August 2025.
URL: <https://designsystem.porsche.com/v3/patterns/forms/guidelines/>
- Ragab Khalaf, M. (2019), 'Responsive or adaptive educational mobile websites: The impact of different designs on students', *The Turkish Online Journal of Educational Technology* **18**(2). Accessed: 22 January 2026.
URL: <https://files.eric.ed.gov/fulltext/EJ1211207.pdf>
- Raneburger, D., Popp, R., Kaindl, H., Armbruster, A. & Šajatović, V. (2014), An iterative and incremental process for interaction design through automated GUI generation, in M. Kurosu, ed., 'Human-Computer Interaction. Theories, Methods, and Tools', Springer International Publishing, Cham, pp. 373–384.
- Roth, R. E. (2017), 'User interface and user experience (UI/UX) design', *The Geographic Information Science & Technology Body of Knowledge (2nd Quarter 2017 Edition)*.
- Roto, V., Väättäjä, H., Law, E. & Powers, R. (2016), Experience design for multiple customer touchpoints, in 'Proceedings of the 9th Nordic Conference on Human-Computer Interaction', NordiCHI '16, Association for Computing Machinery, New York, NY, USA.
URL: <https://doi.org/10.1145/2971485.2987685>
- Rowe, P. G. (1987), *Design thinking*, The MIT Press.
- Sangiorgi, U., Beuvers, F. & Vanderdonckt, J. (2012), 'User interface design by collaborative sketching', *Proceedings of the Designing Interactive Systems Conference, DIS '12*.

- Sangiorgi, U., Beuvers, F. & Vanderdonckt, J. (2025), 'Hierarchy in typography: An analytical review of visual dominance in poster and digital media', *Journal of Emerging Technologies and Innovative Research (JETIR)* **12**. Accessed: 23 January 2026.
URL: <https://www.jetir.org/papers/JETIR2508191.pdf>
- Seffah, A. (2010), 'The evolution of design patterns in HCI: from pattern languages to pattern-oriented design'.
- Seffah, A. (2015), *HCI Pattern Capture and Dissemination: Practices, Lifecycle, and Tools*, Springer International Publishing, Cham, pp. 219–242.
URL: https://doi.org/10.1007/978-3-319-15687-3_10
- Shah, H. (2023), 'Advancing web accessibility: A guide to transitioning design systems from WCAG 2.0 to WCAG 2.1'.
- Shneiderman, B. (1996), The eyes have it: a task by data type taxonomy for information visualizations, in 'Proceedings 1996 IEEE Symposium on Visual Languages', pp. 336–343.
- Snyder, C. (2003), *Paper Prototyping: The Fast and Easy Way to Design and Refine user Interfaces.*, Morgan Kaufmann Publishers.
- Spectrum - Adobe's Design System* (n.d.). Accessed: 21 August 2025.
URL: <https://spectrum.adobe.com/>
- Stanton, N. A. (2006), 'Hierarchical task analysis: Developments, applications, and extensions', *Applied Ergonomics* **37**(1), 55–79. Special Issue: Fundamental Reviews.
URL: <https://www.sciencedirect.com/science/article/pii/S0003687005000980>
- Stein, A. & Ramaseshan, B. (2016), 'Towards the identification of customer experience touch point elements', *Journal of Retailing and Consumer Services* **30**, 8–19.
URL: <https://www.sciencedirect.com/science/article/pii/S0969698915301855>
- Stoeva, M. (2025), The evolution of design systems and UI/UX techniques.
- Suarez, M., Anne, J., Sylor-Miller, K. & Stanfield, D. M. R. (2019), *Design Systems Handbook*, InVision. (2nd ed.).
- Sweller, J. (1988), 'Cognitive load during problem solving: Effects on learning', *Cognitive Science* **12**(2), 257–285.
- Thomas, P. (2025), 'The effect of component reuse on software quality and maintainability'.
- Tidwell, J. (1999), 'Common ground: A pattern language for human-computer interface design'. Accessed: 29 August 2025.
URL: https://www.mit.edu/~jtidwell/common_ground.html

- Tidwell, J. (2010), *Designing Interfaces*, O'Reilly Media, Inc.
- Touloum, K., Idoughi, D. & Seffah, A. (2018), 'Leveraging user experience and touchpoints analysis for services design: Case of crisis management', *Journal of Interaction Science* **6**.
- Toxboe, A. (2020), 'Design patterns'. Accessed: 28 August 2025.
URL: <https://ui-patterns.com/patterns>
- Vääätäjä, H., Savioja, P., Roto, V., Olsson, T. & Varsaluoma, J. (2015), User experience goals as a guiding light in design and development – early findings, in 'INTERACT 2015 Adjunct proceedings', University of Bamberg Press, pp. 521–527. International Conference on Human-Computer Interaction ; Conference date: 01-01-2015.
- Vaisala (n.d.a), 'Indigo500 Series Transmitters', <https://www.vaisala.com/en/products/systems/indoor-monitoring-systems/indigo500>. Accessed: October 27, 2025.
- Vaisala (n.d.b), 'Indigo80 Handheld Indicator Datasheet', <https://docs.vaisala.com/v/u/B212157EN-F/en-US>. Accessed: October 27, 2025.
- van Welie, M. (2001), Task-based User Interface Design, Phd-thesis - research and graduation internal, Vrije Universiteit Amsterdam.
- van Welie, M. & Trætteberg, H. (2000), Interaction patterns in user interfaces, in '7th. Pattern Languages of Programs Conference', pp. 13–16.
- van Welie, M. & van der Veer, G. C. (2003), Pattern languages in interaction design: Structure and organization, in 'Proceedings of interact', Vol. 3, pp. 1–5.
- van Welie, M., van der Veer, G. C. & Eliëns, A. (2001), Patterns as tools for user interface design, in J. Vanderdonckt & C. Farenc, eds, 'Tools for Working with Guidelines', Springer London, London, pp. 313–324.
- van Welie, M., van der Veer, G. & Eliëns, A. (1998), An ontology for task models, in 'Design, specification and verification of interaction systems '98', pp. 81–94.
- Verhoef, P. C., Kannan, P. & Inman, J. J. (2015), 'From multi-channel retailing to omni-channel retailing: Introduction to the special issue on multi-channel retailing', *Journal of Retailing* **91**(2), 174–181. Multi-Channel Retailing.
URL: <https://www.sciencedirect.com/science/article/pii/S0022435915000214>
- Vesselov, S. & Davis, T. (2019), *Building Design Systems: Unify User Experiences through a Shared Design Language*, 1st edn, APress.
- Waller, R. & Delin, J. (2010), 'Towards a pattern language approach to document description'.

- Weidig, J., Weippert, M. & Kuehn, C. (2024), 'Personalized touchpoints and customer experience: A conceptual synthesis', *Journal of Business Research* **177**, 114641.
URL: <https://www.sciencedirect.com/science/article/pii/S0148296324001450>
- Werle, D. R. (2021), Design systems: Definitions and main elements, Master's thesis, Tallin University, School of Digital Technologies, Tallin, Estonia. Accessed: 1 September 2025.
URL: <https://www.idmaster.eu/wp-content/uploads/2022/01/Daniel-Werle-thesis-Vdrw-2.pdf>
- World Wide Web Consortium (W3C) (n.d.), 'Web platform design principles'. Accessed: January 8, 2025.
URL: <https://www.w3.org/TR/design-principles/>
- Wu, L., Li, J. & Lei, T. (2016), Design entropy: A new approach for evaluating user experience in user interface design, in F. Rebelo & M. Soares, eds, 'Advances in Ergonomics in Design', Springer International Publishing, Cham, pp. 583–593.
- Wynn, D. & Eckert, C. (2017), 'Perspectives on iteration in design and development', *Research in Engineering Design* **28**.
- Yew, J., Convertino, G., Hamilton, A. & Churchill, E. (2020), Design systems: A community case study, in 'Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems', CHI EA '20, Association for Computing Machinery, New York, NY, USA, p. 1–8.
URL: <https://doi.org/10.1145/3334480.3375204>
- Yin, R. K. (2017), Case study research and applications, in 'Case Study Research and Applications', SAGE Publications, Incorporated, United States.
- Zhang, X. & Wakkary, R. (2014), Understanding the role of designers' personal experiences in interaction design practice, in 'Proceedings of the 2014 Conference on Designing Interactive Systems', DIS '14, Association for Computing Machinery, New York, NY, USA, p. 895–904.
URL: <https://doi.org/10.1145/2598510.2598556>

A Semi-Structured Interview Guide

1. Background / Warming Up (13 min)

A. Role & Workflow

1. Could you briefly describe your role and the kind of work you do daily?
2. Who do you collaborate with the most in your work?

B. Current Approach to Reuse & Decision-Making

3. Think of the last time you had to solve a recurring design problem. What was the first thing you did?
4. What sources do you use to decide how a particular design solution should work?
5. What tends to slow you down when making design decisions?

C. Design System (role-dependent section)

For Designers and Developers:

6. How often do you use the design system in your work?
7. For what kind of tasks do you use it for?
8. What things or parts of the design system do you use most?
9. Can you recall a recent moment when it didn't give you what you needed?

For Design System Developers:

6. What triggers an update to design system documentation today?
7. What information do you need before publishing new documentation?
8. What makes documentation hard to maintain over time?
9. What makes it easier for teams to adopt changes?

2. Main Questions (30 min)

Part 1: Prior Experience with Design Patterns

10. What comes to mind when you hear the term *design pattern*?
11. What about *UX design pattern*?
12. In academic literature, design patterns have been separated into UI and UX patterns. How would that work for you?

(a) How would that separation affect how you discuss patterns with others?

13. Have you used design patterns in your work before?

Part 2: Prior Assumptions of Ideal Documentation

14. What kind of information would you look for in pattern documentation?

15. How could pattern documentation help you?

16. For what problems would pattern documentation **not** help you?

17. What would make you ignore pattern documentation even if it existed?

Part 3: Show the Design Proposal & Gather Feedback

A. First Impressions

18. Without clicking anything yet, what do you think this page is?

19. What do you notice first?

20. What do you think is most important here?

21. What parts are unclear?

B. Content Quality & Decision Support

22. What information in the documentation helps you decide whether to use this pattern?

23. What information in the documentation helps you decide how to implement it?

24. What would you change in this documentation first?

(a) What would you remove?

(b) What would you add?

(c) What's missing?

C. Naming / Structure Terminology

25. If you had to rename these "components", what would you call them?

26. Which of these terms would feel most natural when searching later?

(a) building blocks

(b) organisms

(c) assembled solutions

D. Findability & Navigation

27. Scenario 1: Imagine you're working on a login screen for a new application. Where would you start in this documentation?
- (a) What made you choose that?
 - (b) What would you expect to see here?
28. Scenario 2: You need to check whether this Login pattern is appropriate for a specific context. Where would you look first?
- (a) What made you choose that?
 - (b) What would you expect to see here?
29. How would you make it better for navigating under high pressure?

E. Adoption Barriers & Motivation

30. In what situations do you think you would refer to this pattern documentation?
31. How do you think this pattern documentation would help others?

4. Wrap-up (3-4 min)

32. Is there anything we did not cover that feels important?
33. What advice would you give to someone creating pattern documentation?